WNDRVR
SUPPORT NETWORK

API REFERENCE: APPLICATION NETWORKING, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

API Reference: Application Networking, SR0650

20 July 2020

1. AFPACKETLIB

Name

afPacketLib - **AF_PACKET** socket domain library

Layer

RTNET

Routines

llFlowProtoSet( ) - set layer 2 protocol in socket flow description. **llFlowIfindexSet( )** - set ifindex in socket flow description
afPacketSockRx( ) - deliver a frame to all matching packets sockets **afPacketRx( )** - reception handler for **AF_PACKET** sockets
afPacketLibInit( ) - initialize **AF_PACKET** socket library

Description

Address domain specific functions for **AF_PACKET**. **AF_PACKET** provides a BSD socket abstraction on top of layer 2. As an example, the socket applicaiton will be working with raw Ethernet frames when transmitting/receiving to/from an Ethernet network interface.

The socket address type for **AF_PACKET** is **struct sockaddr_ll** with is defined in headerfile **packet.h** and has to following definition

```
struct sockaddr_ll
    {
    uint8_t     sll_len;        /* Length of structure, >=sizeof(struct sockaddr_ll) */
    uint8_t     sll_family;     /* Always AF_PACKET */
    uint16_t    sll_protocol;   /* Physical layer-protocol */
    uint32_t    sll_ifindex;    /* Network interface number */
    uint16_t    sll_hatype;     /* ARP hardware type, IFT_xxx */
    uint8_t     sll_pkttype;    /* Packet type, PACKET_xxx */
    uint8_t     sll_halen;      /* Length of physical-layer address */
    uint8_t     sll_addr[8];    /* Physical-layer address  */
    };
```

Sockets may be created either with type **SOCK_RAW** or **SOCK_DGRAM**. The differences between the two types is that the application is resposible for providing the full frame for **SOCK_RAW** while the network stack will add an apropritate layer 2 header when sending via a **SOCK_DGRAM** socket.

The full frame is always provided in receive operations, regardless of socket type.

An **AF_PACKET** socket will not match any ingress frames before it is bound with a call to **bind( )**. It is however possible to transmit frames on an unbound socket.

Only the sll_len, sll_family, sll_protocol and sll_ifindex fiels must be set when binding the socket. A non-zero protocol value means the socket only matches frames of that protocol, while zero matches all frame types. A non-zero sll_ifindex means only packets from that network interface matches the socket, while a zero value matches any interface.

An ingress frame may match multiple sockets, all matching sockets will receive a copy of the frame.

Note that this implementation of **AF_PACKET** sockets does not allow setting the IEEE 802.3 protocol number at the time of socket creation. The third parameter in the socket API must be zero like below examples:

packet_socket = socket (**AF_PACKET**, **SOCK_RAW**, 0); packet_socket = socket (**AF_PACKET**, **SOCK_DGRAM**, 0);

WNDRVR

Include files

afPacketLib.h

Name

llFlowProtoSet( ) - set layer 2 protocol in socket flow description.

Layer

RTNET

Synopsis

```
_WRS_INLINE void llFlowProtoSet
    (
    SYS_SOCK_FLOW * flow,   /**< flow description */
    UINT16BE        proto   /**< layer 2 protocol to set */
    )
```

Description

This routine updates the flow with the specified layer 2 protocol to make it possible to lookup matching **AF_PACKET** sockets.

Returns

N/A

Errno

N/A

See also

afPacketLib

Name

llFlowIfindexSet( ) - set ifindex in socket flow description

Layer

RTNET

Synopsis

```
_WRS_INLINE void llFlowIfindexSet
    (
    SYS_SOCK_FLOW * flow,   /**< flow description */
    IFI             ifi     /**< ifindex */
    )
```

Description

This routine updates the flow with the specified network interface index to make it possible to lookup matching **AF_PACKET** sockets.

Returns

N/A

Errno

N/A

See also

afPacketLib

Name

afPacketSockRx( ) - deliver a frame to all matching packets sockets

Description

This routine delivers an ingress frame to all matching **AF_PACKET** sockets.

Returns

N/A

Errno

N/A

See also

afPacketLib

Name

afPacketRx( ) - reception handler for **AF_PACKET** sockets

Layer

RTNET

Synopsis

```
void afPacketRx
    (
    const PKT_CTX * cFrame   /**< Ethernet frame */
    )
```

Description

This routine delivers the frame to all matching **AF_PACKET** socket instances.

Returns

N/A

Errno

N/A

See also

afPacketLib

Name

afPacketLibInit( ) - initialize **AF_PACKET** socket library

Layer

RTNET

Synopsis

```
STATUS afPacketLibInit (void)
```

Description

This routine register the the socket back-ends for **SOCK_RAW** and **SOCK_DGRAM** socket on the **AF_PACKET** domain.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOMEM

Cannot allocate memory.

See also

afPacketLib

2. ARGLIB

Name

argLib - argument parsing library

Layer

NET_BASE

Routines

argFind( ) - find matching argument definition **argInit( )** - initialize an argument context **argParse( )** - parse arguments **argIdxGet( )** - get the index of first unparsed argument **argErrGet( )** - get parse error reason **argDef( )** - define an argument instance **argBool( )** - create a boolean argument **argInt( )** - create an integer argument **argIntRange( )** - create an integer argument with a specific range **argChoice( )** - create a choice argument **argStr( )** - create a choice argument **argInAddr( )** - create an IPv4 address argument **argIn6Addr( )** - create an IPv6 address argument

Description

This module provides routes to help parsing command-line arguments passed to commands that are designed to be invoked by system users. Arguments are validated and stored in appropriate C datatypes.

The process is separated into two stages. The first stage defines all comman-line arguments, i.e. specifies the type, possibly the range and also associate a human readable description to the argument. This is done with the set of routines with names like **argIntDef( )**, **argStringDef( )**, etc.

The second phase is passing the sequence of command-line arguments to this module via the **argParse( )** routine. This part will try to parse all arguments with the limits gived by the argument definitions and store the parsed data into the C-variable associated with each argument.

The definition of the command-line argument is also used to generate a manual page, the ´-h´ and ´--help` option is always available to show the manual page generated from the argument definition(s).

Example

```
int main(int argc, char * argv[])
   {
   BOOL         isFoo;
   int          barLevel;
   const char * theString;
   ARG_CTX      ctx;
   ARG_DEF      argDefs[3];
   ARG_DEF *    def;

   /*
    * Initialize a local argument parsing context
    */
   argInit (&ctx);

   /*
    * Define the option '-f' and '--foo', define the default value to 'FALSE' and
    * associate the argument with the C boolean 'isFoo'.
    */
   def = argDef (&ctx, argDefs + 0, 'f', "foo", "enable the fooer");
```

```
   argBool (def, &isFoo, FALSE);

   /*
    * Define the option '-b <level>' and '--bar=<level>', define the default
    * value to '42' and associate the argument with the C integer 'barLevel'.
    */
   def = argDef (&ctx, argDefs + 1, 'b', "bar", "set the bar level");
   argInt (def, &barLevel, 42);

   /*
    * Define the option '-s <string>' and '--string=<string>', define the
    * default value to 'NULL' and associate the argument with the C integer
    * 'theString'.
    */
   def = argDef (&ctx, argDefs + 2, 's', "string", "specify the string");
   argStr (def, &theString, NULL);

   /*
    * Parse the arguments
    */
   if (argParse (&ctx, argc, argv))
       {
       /* argument parsing successful */
       }
   else
       {
       /* failed to parse the arguments */
       }
   ...
   }
```

Include files

argLib.h

Name

argFind( ) - find matching argument definition

Layer

NET_BASE

Synopsis

```
ARG_DEF * argFind
    (
    ARG_CTX *     ctx,       /**< argument context */
    const char *  args[],    /**< user arguments */
    int           argsLen,   /**< length of 'args' */
    int *         argNxt,    /**< current index, will gold next index */
    const char ** argVal     /**< will point to argument value */
    )
```

Description

This routine try to find the argument definition instance that matches the current user-argument, args[*argNxt].

Returns

Pointer to argument definition, or **NULL** if no match is found.

Errno

Not Available

See also

argLib, **argMatch( )**

Name

argInit( ) - initialize an argument context

Layer

NET_BASE

Synopsis

```
void argInit
    (
    ARG_CTX * ctx   /**< argument context */
    )
```

Description

This routine initialize an argument context, an argument to show a generated help page is added to the context.

Returns

N/A

Errno

N/A

See also

argLib

Name

argParse( ) - parse arguments

Layer

NET_BASE

Synopsis

```
BOOL argParse
    (
    ARG_CTX *     ctx,       /**< argument context */
    const char * args[],     /**< arguments to be parsed, argv[0] may be the program name */
    int           argLen     /**< number of elements in 'args' */
    )
```

Description

This routine parse arguments in the **args** array

Returns

TRUE if the arguments are successfully processed, or **FALSE** otherwise.

Errno

Not Available

See also

argLib, **argIdxGet( )**, **argErrGet( )**

Name

argIdxGet( ) - get the index of first unparsed argument

Layer

NET_BASE

Synopsis

```
int argIdxGet
    (
    const ARG_CTX * ctx    /**< argument context */
    )
```

Description

This routine return the index to the first unparsed argument, only meaningful to call after **argParse( )**.

Returns

An index

Errno

Not Available

See also

argLib, **argParse( )**

Name

argErrGet( ) - get parse error reason

Layer

NET_BASE

Synopsis

```
ARG_ERR argErrGet
    (
    const ARG_CTX * ctx   /**< argument context */
    )
```

Description

This routine return the reason why not all argument got parsed, only meaningful to call after **argParse( )**.

Returns

ARG_SUCCESS if all argumens is parsed, or **ARG_STOP** if all arguments
up to the first argument equal to "--" is successful, or
ARG_ERR_UNKNOWN if an unknown argument name is found, or
ARG_ERR_PARSE if the parsing of a value to an argument failed.

Errno

Not Available

See also

argLib, **argParse( )**

Name

argDef( ) - define an argument instance

Layer

NET_BASE

Synopsis

```
ARG_DEF * argDef
    (
    ARG_CTX *    ctx,        /**< argument context */
    ARG_DEF *    def,        /**< argument definition storage */
    char         shortOpt,   /**< single character identifier */
    const char * longOpt,    /**< descriptive string identifier */
    const char * doc         /**< argument documentation */
    )
```

Description

This routine define the generic parts for an argument instance, the argument type and default value must be set separately.

Returns

RET_DESC

Errno

Not Available

See also

argLib, **argBool( )**, **argInt( )**, **argIntRange( )**, **argStr( )**, **argInAddr( )**, **argIn6Addr( )**

Name

argBool( ) - create a boolean argument

Layer

NET_BASE

Synopsis

```
void argBool
    (
    ARG_DEF * def,         /**< argument defintion from argDef() */
    BOOL *    val,         /**< pointer to C varaible for argument */
    BOOL      defaultVal   /**< default value */
    )
```

Description

This routine create a boolean argument associated with the argument specification set with **argDef( )**. The first argument must be a pointer returned by **argDef( )**.

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

Name

argInt( ) - create an integer argument

Layer

NET_BASE

Synopsis

```
void argInt
    (
    ARG_DEF * def,          /**< argument defintion from argDef() */
    int *     val,          /**< pointer to C variable for argument */
    int       defaultVal    /**< default value */
    )
```

Description

This routine create an interger argument associated with the argument specification set with **argDef( )**.

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

Name

argIntRange( ) - create an integer argument with a specific range

Layer

NET_BASE

Synopsis

```
void argIntRange
    (
    ARG_DEF * def,          /**< argument defintion from argDef() */
    int *     val,          /**< pointer to C variable for argument */
    int       defaultVal,   /**< default value to assign 'val' */
    int       minVal,       /**< minimum allowed value, inclusive */
    int       maxVal        /**< maximum allowed value, inclusive */
    )
```

Description

This routine create an interger argument range associated with the argument specification set with **argDef( )**. Parsing will fail if the argument value can be parsed as an integer but the value is outside of the range.

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

Name

argChoice( ) - create a choice argument

Layer

NET_BASE

Synopsis

```
void argChoice
    (
    ARG_DEF *       def,          /**< argument definition from argDef() */
    int *           val,          /**< pointer to C variable for argument */
    int             defaultVal,   /**< default value to assign 'val' */
    const ARG_CHS * chl,          /**< list of allowed choices */
    int             chlLen        /**< number of elements in 'chl' array */
    )
```

Description

This routine create a argument definition that make it possible to pick one option out of a fixed set of options.

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

Name

argStr( ) - create a choice argument

Layer

NET_BASE

Synopsis

```
void argStr
    (
    ARG_DEF *     def,          /**< argument definition from argDef() */
    const char ** val,          /**< storage for used specified string */
    const char *  defaultStr    /**< default value for 'str' */
    )
```

Description

This routine create a argument definition capture an unparsed user-supplied string.

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

Name

argInAddr( ) - create an IPv4 address argument

Layer

NET_BASE

Synopsis

```
void argInAddr
    (
    ARG_DEF *        def,       /**< argument definition from argDef() */
    struct in_addr * val,       /**< storage for used specified string */
    const char *     dfltAddr   /**< default value for 'str' */
    )
```

Description

This routine create an argument definition for an IPv4 address

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

Name

argIn6Addr( ) - create an IPv6 address argument

Layer

NET_BASE

Synopsis

```
void argIn6Addr
    (
    ARG_DEF *         def,       /**< argument definition from argDef() */
    struct in6_addr * val,       /**< storage for used specified string */
    const char *      dfltAddr   /**< default value for 'str' */
    )
```

Description

This routine create an argument definition for an IPv6 address

Returns

N/A

Errno

Not Available

See also

argLib, **argDef( )**

3. ARP4LIB

Name

arp4Lib - address resolution protocol library

Layer

RTNET

Routines

arp4LibInit( ) - initialize the ARP library

Description

Implementation of the Address Resolution Protocol, ARP. This protocol provide means to find the Ethernet MAC address of an IPv4-node located on the same link.

Invoking **ip4LibInit( )** will result in a call to **arp4LibInit( )**, so no explicit initialization is required of this library when used in a system using **ip4Lib**.

This library implements the the following RFC /ts RFC | Title -----|------ 0826 | An Ethernet Address Resolution Protocol /te

Include files

arp4Lib.h

Name

arp4LibInit( ) - initialize the ARP library

Layer

RTNET

Synopsis

```
STATUS arp4LibInit(void)
```

Description

This routine initialize the ARP library by register it with the Ethernet layer to receive frames carrying ARP as payload.

Returns

OK if successful, **ERROR** otherwise

Errno

ENOMEM

Could not allocate memory.

See also

arp4Lib

4. ARPAINETLIB

Name

arpaInetLib - string = binary translation of IP addresses

Layer

NET_BASE

Routines

isPartCntValidInet6Ctx( ) - check if the number of hex-parts is valid **inet_pton( )** - convert a string representation of an Internet address to its **inet_aton( )** - convert a string to IPv4 address **inet_ntop( )** - convert Internet address to a string representation **inet_addr( )** - convert a dot notation Internet address to a long integer **inet_ntoa( )** - convert a network address to dotted decimal notation

Description

This module implements the Internet address string conversion routines defined by **arpa/inet.h**. All functions in the module are stateless, so no initialization routine exist and all functions are reentrant and thread safe.

This module implement the parts relevant for conversion between binary and string representation of Internet address:

/ts RFC | Title -----|----- 3493 | Basic Socket Interface Extensions for IPv6 5952 | A Recommendation for IPv6 Address Text Representation /te

Include files

arpa/inet.h

Name

isPartCntValidInet6Ctx( ) - check if the number of hex-parts is valid

Layer

NET_BASE

Synopsis

```
_WRS_INLINE BOOL isPartCntValidInet6Ctx
    (
    const INET6_PTON_CTX * ctx   /**< inet6 parser context */
    )
```

Description

This routine check if the number of hex-parts is a valid count for an IPv6 address.

Returns

TRUE if valid, or **FALSE** otherwise

Errno

N/A

See also

arpalnetLib

Name

inet_pton( ) - convert a string representation of an Internet address to its

Layer

NET_BASE

Synopsis

```
int inet_pton
    (
    int          family,   /**< address family, AF_INET, AF_INET6 or AF_LINK */
    const char * strPtr,   /**< string to parse */
    void *       addrPtr   /**< storage area for the binary */
                           /* representation of the address */
    )
```

Description

binary representation

This routine try to convert a string representation of an IPv4 (family==**AF_INET**) or IPv6 (family==**AF_INET6**) address to its binary representation.

Returns

1 if successful, or 0 if the string is not a valid string representation, or -1 if the address family is not supported

Errno

EAFNOSUPPORT

Specified address family is not supported by routine.

See also

arpalnetLib

Name

inet_aton( ) - convert a string to IPv4 address

Layer

NET_BASE

Synopsis

```
int inet_aton
    (
    const char *     cp,        /**< a text representation of an Internet */
                                /* address */
    struct in_addr * addrPtr    /**< storage for the binary representation of */
                                /* the address */
    )
```

Description

Try to convert the Internet host address **cp** from dotted quad notation into binary data and stores in in the structure that **addr** points to.

a.b.c.d Each of the four numeric parts specifies a byte of the address; the bytes are assigned in left-to-right order to produce the binary address. **a.b.c** Parts a and b specify the first two bytes of the binary address. Part c is interpreted as a 16-bit value that defines the rightmost two bytes of the binary address. This notation is suitable for specifying (out-moded) Class B network addresses. a.b Part a specifies the first byte of the binary address. Part b is interpreted as a 24-bit value that defines the rightmost three bytes of the binary address. This notation is suitable for specifying (outmoded) Class C network addresses. a The value a is interpreted as a 32-bit value that is stored directly into the binary address without any byte rearrangement.

In all of the above forms, components of the dotted address can be specified in decimal, octal (with a leading 0), or hexadecimal, with a leading 0X). Addresses in any of these forms are collectively termed IPV4 numbers-and-dots notation. The form that uses exactly four decimal numbers is referred to as IPv4 dotted-decimal notation (or sometimes: IPv4 dotted-quad notation).

Returns

non-zero(1) if successful, or 0 if the string is not a valid string representation

Errno

N/A

See also

arpalnetLib

Name

inet_ntop( ) - convert Internet address to a string representation

Layer

NET_BASE

Synopsis

```
const char * inet_ntop
    (
    int          family,    /**< network address family, AF_INET or */
```

```
                            /* AF_INET6 */
    const void * addrPtr,   /**< binary address of Internet address */
    char *       str,       /**< storage for the string representation */
    size_t       strLen     /**< the length of 'str' buffer */
    )
```

Description

This routine try to convert a binary representation of an IPv4 or IPv6 address to a corresponding string representation.

Returns

pointer to the string, or **NULL** if the routine failed

Errno

EAFNOSUPPORT

Specified address family is not supported by routine.

ENOSPC

Not enough buffer space for the string

See also

arpalnetLib

Name

inet_addr( ) - convert a dot notation Internet address to a long integer

Layer

NET_BASE

Synopsis

```
uint32_t inet_addr
    (
    const char * cp
    )
```

Description

This routine interprets an Internet address string, returning the corresponding 32-bit value in network byte order, or returning (uint32_t)(-1) if the string is not a valid internet address string.

This routine is obsolete; its return value does not distinguish between an error return and the valid **all ones** restricted IPv4 broadcast address 255.255.255.255. Instead of using this routine, use **inet_aton( )** or **inet_pton( )**.

This routine handles addresses of the forms

```
    a a in [0, 4294967295UL]
        x = a
    a.b       a in [0, 255]; b in [0, 16777215]
            x = (a << 24) + b
    a.b.c     a, b in [0, 255]; c in [0, 65535]
                x = (a << 24) + (b << 16) + c
    a.b.c.d   a, b, c, d in [0, 255]
                x = (a << 24) + (b << 16) + (c << 8) + d
```

where each of a, b, c, or d is a decimal, octal (starts with the digit 0) or hexidecimal (starts with 0x) number string; the corresponding numbers must be in the indicated ranges. Leading and embedded whitespace is not allowed, but any string character that is not a valid digit in the current numeric base other than **.** terminates any of the four forms, and additionally **.** will terminate the last form.

The return value is x converted to network byte order if the address string is acceptable.

Warning

Do not use this routine for new programs, use **inet_pton( )** instead.

Example

The following example returns 0x5a000002 on a big-endian target:

```
    inet_addr ("90.0.0.2");
```

Returns

The Internet address, or **INADDR_NONE**.

Errno

Not Available

See also

arpalnetLib, **inet_pton( )**

Name

inet_ntoa( ) - convert a network address to dotted decimal notation

Layer

NET_BASE

Synopsis

```
char * inet_ntoa
    (
    struct in_addr in
    )
```

Description

This routine converts an Internet address in network format to dotted decimal notation.

Warning

This routine is supplied for UNIX compatibility only. Previously, this routine allocates an 18 bytes of memory each time it is called and relies on the caller for the memory to be freed when done with the returned string. This is no longer the case with the current implementation. The string returned is a statically allocated buffer.

Do not use this routine for new programs, use **inet_ntop( )** instead.

Example

The following example returns a pointer to the string "90.0.0.2":

```
struct in_addr iaddr;
 ...
iaddr.s_addr = htonl(0x5a000002);
 ...
inet_ntoa (iaddr);
```

Returns

A pointer to the string version of an Internet address.

Errno

Not Available

See also

arpalnetLib, **inet_ntop( )**

5. CSUMOFLDLIB

Name

csumOfldLib - checksum offload library

Layer

RTNET

Routines

csumOfldTxGet( ) - get Internet checksum parameter **csumOfldPseudoHdr( )** - calculate pseudo header checksum
csumOfldTxTr( ) - perform Internet checksum on transport layer **muxIfTxCsum( )** - ensure that checksums are calculated

Description

This module is a helper function to setup Internet checksum calculations in drivers.

Include files

csumOfldLib.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

csumOfldTxGet( ) - get Internet checksum parameter

Layer

RTNET

Synopsis

```
BOOL csumOfldTxGet
    (
    CSUM_OFLD  * co,     /**< pointer to where to write the offload parameters */
    const void * frame   /**< Ethernet frame */
    )
```

Description

This routine get the offsets and other parameters required for network interface cards to calculate the Internet checksum of an egress Ethernet frame.

Returns

TRUE if this frame need checksum calculation, or **FALSE** otherwise

Errno

N/A

See also

csumOfldLib

Name

csumOfldPseudoHdr( ) - calculate pseudo header checksum

Layer

RTNET

Synopsis

```
UINT32 csumOfldPseudoHdr
    (
    const CSUM_OFLD * co,        /**< checksum offload parameters */
    void *            frame,     /**< Ethernet frame with IPv4/IPv6 packet */
    UINT32            frameLen   /**< length of the frame */
    )
```

Description

This routine calculate the pseudo header checksum for IPv4 or IPv6

Returns

intermediate checksum

Errno

N/A

See also

csumOfldLib

Name

csumOfldTxTr( ) - perform Internet checksum on transport layer

Layer

RTNET

Synopsis

```
_WRS_INLINE void csumOfldTxTr
    (
    const CSUM_OFLD * co,        /**< checksum offload parameters */
    void *            frame,     /**< Ethernet frame with IPv4/IPv6 packet */
    UINT32            frameLen   /**< length of the frame */
    )
```

Description

This routine calculate and write the Internet checksum for the transport layer.

Returns

N/A

Errno

Not Available

See also

csumOfldLib, **csumOfldTx( )**

Name

muxIfTxCsum( ) - ensure that checksums are calculated

Layer

RTNET

Synopsis

```
void csumOfldTx
    (
    const END_CAPABILITIES * cap,     /**< device capabilities */
    M_BLK_ID                 mBlkId   /**< buffer to setup checksum calc */
    )
```

Description

This routine check the capability of the device, Internet checksum is either calculated in this routine or the buffer is setup to calculate the checksum in the egress interface.

Returns

N/A

Errno

N/A

See also

csumOfldLib

6. DAEMON

Name

daemon - Module that implements the tNetTask

Layer

NET_BASE

Routines

netLibInitialize( ) - initialize the network package **netDaemonQ( )** - return the job queue ID of the specified network daemon **netDaemonQnum( )** - return the network job queue number of the specified queue **netDaemonsCount( )** - return the number of network daemons

Description

This library contains the network task that runs low-level network interface routines in a task context. The network task executes and removes routines that were added to the job queue. This facility is used by network interfaces in order to have interrupt-level processing at task level.

This library has been modified to use **jobQueueLib** as the underlying job engine. The tNetTask task now executes **jobProcess( )** to service jobs. The **netJobAdd( )** routine is still available as a macro implemented in terms of **jobQueueStdPost( )**. However, as discussed in the **jobQueueLib** documentation, **jobPost( )** is now the preferred API for deferring work.

The routine **netLibInit( )** initializes the network and spawns the network task **netTask( )**. This is done automatically when **INCLUDE_NET_DAEMON** is defined.

The routine **netHelp( )** in **usrLib** displays a summary of the network facilities available from the VxWorks shell.

Include files

netLib.h

See also

routeLib, **hostLib**, **netDrv**, **netHelp( )**, VxWorks Network Programmer's Guide

Name

netLibInitialize( ) - initialize the network package

Layer

NET_BASE

Synopsis

```
STATUS netLibInitialize
    (
    unsigned int              numDaemons,
    BOOL                      affinityLock,
```

```
    int (*netTaskFunc) (JOB_QUEUE_ID qId, int i),
    unsigned int              priorities_configured[],
    unsigned int              affinities_configured[]
    )
```

Description

This routine should be called only once to initialize the network. This is done automatically when **INCLUDE_NET_DAEMON** is defined. This routine is not protected against concurrent calls from multiple threads.

If numDaemons is zero, then for uniprocessor builds, it is first set instead to 1, while for SMP builds, it is set to the number of configured CPUs.

This routine creates a number of network daemon tasks to service network jobs.

Returns

OK, or **ERROR** if network support cannot be initialized.

Errno

Not Available

See also

daemon, usrConfig, **netTask( )**

Name

netDaemonQ( ) - return the job queue ID of the specified network daemon

Layer

NET_BASE

Synopsis

```
JOB_QUEUE_ID netDaemonQ
    (
    int i
    )
```

Description

i is the number of a particular network daemon, a non-negative integer less than the numDaemons parameter passed to **netLibInitialize( )**. This function returns the corresponding job queue ID.

Returns

NULL if i out of range; otherwise, the ith network daemon queue ID.

Errno

Not Available

See also

daemon, **netDaemonQnum( )**

Name

netDaemonQnum( ) - return the network job queue number of the specified queue

Layer

NET_BASE

Synopsis

```
int netDaemonQnum
    (
    JOB_QUEUE_ID qId
    )
```

Description

If qId is the **JOB_QUEUE_ID** of one of the network daemon job queues, this function returns the corresponding job queue number, an index in [0, netNumDaemons). Otherwise, this function returns -1.

This function is essentially the inverse of the function **netDaemonQ( )** (if the domain of both functions is suitably restricted).

Returns

The index of the network job queue corresponding to qId, or -1, if there is no corresponding network daemon job queue.

Errno

Not Available

See also

daemon, **netDaemonQ( )**

Name

netDaemonsCount( ) - return the number of network daemons

Layer

NET_BASE

Synopsis

```
unsigned int netDaemonsCount (void)
```

Description

This function returns the number of network daemons created at startup time by the **netLibInitialize( )** function. If additional daemons are created after start-up, they are not counted.

Returns

The number of network daemons created at startup time by the **netLibInitialize( )** function.

Errno

Not Available

See also

daemon

7. ENDIFLIB

Name

endIfLib - network interface for user-space END drivers

Layer

RTNET

Routines

No user-callable routines

Description

Include files

endIfLib.h

8. ETHLIB

Name

ethLib - Ethernet protocol library

Layer

RTNET

Routines

ethTypeToTblIdx( ) - map Ethernet frame type to RX function table index **ethTypeToFnIdx( )** - map Ethernet frame type to RX function index **ethLibInit( )** - initialize the Ethernet library **ethProbeTxSet( )** - set probe transmission handler **ethFrameRxSet( )** - set reception handler for a frame type **ethRx( )** - Ethernet reception handler **ethProbe( )** - initiate next-hop IP-address to Ethernet address mapping **ethNeighUpdate( )** - set the context data for a neighbor cache node

Description

Ethernet protocol implementation for the VxWorks Real-time network stack. This library only contain the multiplexing of incoming frames to the appropriate upper layer handler selected on Ethernet frame type. Upper layer handler is registered via the **ethFrameRxSet( )** routine.

This library implements the relevant parts of the following RFC. /ts RFC | Title -----|------ 0894 | A Standard for the Transmission of IP Datagrams over Ethernet Networks 1112 | Host Extensions for IP Multicasting 2464 | Transmission of IPv6 Packets over Ethernet Networks /te

Include files

ethLib.h

Name

ethTypeToTblIdx( ) - map Ethernet frame type to RX function table index

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT8 ethTypeToTblIdx
    (
    UINT16BE type
    )
```

Description

This routine map Ethernet frame type to RX function table index

Returns

And RX function table index between 0 and 255.

Errno

N/A

See also

ethLib

Name

ethTypeToFnIdx( ) - map Ethernet frame type to RX function index

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT8 ethTypeToFnIdx
    (
    UINT16BE type
    )
```

Description

This routine map Ethernet frame type to the RX function index

Returns

An RX function index between 0 and 255.

Errno

N/A

See also

ethLib

Name

ethLibInit( ) - initialize the Ethernet library

Layer

RTNET

Synopsis

```
STATUS ethLibInit (void)
```

Description

This routine initialize the Ethernet library to discard all frame types.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOMEM

Cannot allocate memory.

See also

ethLib

Name

ethProbeTxSet( ) - set probe transmission handler

Layer

RTNET

Synopsis

```
void ethProbeTxSet
    (
    BOOL          isV4,      /**< TRUE if IPv4, FALSE if IPv6 */
    ETH_PROBE_RTN probeRtn   /**< probe function to use */
    )
```

Description

This routine set probe transmission handler for IPv4 or IPv6

Returns

N/A

Errno

N/A

See also

ethLib

Name

ethFrameRxSet( ) - set reception handler for a frame type

Layer

RTNET

Synopsis

```
STATUS ethFrameRxSet
    (
    UINT16BE   type,   /**< Frame type */
    PKT_RX_RTN rxRtn   /**< Reception handler for frame type */
    )
```

Description

This routine set the reception handler for a specific frame type.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOMEM

Could not allocate memory.

See also

ethLib

Name

ethRx( ) - Ethernet reception handler

Layer

RTNET

Synopsis

```
void ethRx
    (
    PKT_CTX * frame   /**< Ethernet frame */
    )
```

Description

This routine process ingress Ethernet frames and dispatch them to the appropriate upper layer handler based on the frame type.

Returns

N/A

Errno

Not Available

See also

ethLib

Name

ethProbe( ) - initiate next-hop IP-address to Ethernet address mapping

Layer

RTNET

Synopsis

```
void ethProbe
    (
    NEIGH_DB                neighDb,          /**< neighbor database */
    BOOL                    useUnicastSol,    /**< TRUE if unicast */
                                              /* solicitation is preferred */
    const struct in6_addr * nh,               /**< next-hop address */
    const struct in6_addr * triggerSrc,       /**< source address that */
                                              /* triggered this probe */
    const void *            neighCtx          /**< ethernet header for */
                                              /* neighbor node */
    )
```

Description

This routine initiate the process to find a mapping from a next-hop/neighbor IP-address to the corresponding Ethernet address. This may include protocols like ARP/NDP but some mappings may also be computed.

Returns

N/A

Errno

N/A

See also

ethLib

Name

ethNeighUpdate( ) - set the context data for a neighbor cache node

Layer

RTNET

Synopsis

```
void ethNeighUpdate
    (
    NEIGH_DB                neighDb,    /**< neighbor database */
    void *                  neighCtx,   /**< neighbor entry context */
    const struct in6_addr * nh,         /**< next-hop IP-address */
    const void *            nhEthAddr   /**< next-hop Ethernet address */
    )
```

Description

This routine set the context data for a neighbor cache node

Returns

N/A

Errno

Not Available

See also

ethLib

9. FIBLIB

Name

fibLib - forward information base library

Layer

RTNET

Routines

fibInit( ) - initialize a forward information base instance **fibLookup( )** - try to lookup an node in a FIB instance **fibInsert( )** - try to insert a new node into a FIB **fibRemove( )** - Remove all FIB-nodes matching the contraint **fibForEach( )** - run a callback on each FIB node **fibAttrInit( )** - Initialize a new set of attributes. **fibAttrDstGet( )** - Get destination address attribute **fibAttrDstSet( )** - Set destination address attribute **fibAttrSrcGet( )** - Get source address attribute **fibAttrSrcSet( )** - Set source address attribute **fibAttrNhGet( )** - Get next-hop address attribute **fibAttrNhSet( )** - Set next-hop attribute **fibAttrIfiGet( )** - Get network interface index attribute **fibAttrIfiSet( )** - Set network interface index attribute **fibAttrPlenGet( )** - Get prefix length attribute **fibAttrPlenSet( )** - Set prefix length attribute **fibAttrNlenGet( )** - Get prefix length attribute **fibAttrNlenSet( )** - Set prefix length attribute

Description

Forward information base, sometimes also called "forwarding table", is optmized data structures for mapping an IP destination address to the egress network interface and the address of the next hop IP-node to deliver an IP-datagram to.

The most basic lookup would just specify the IP address of the final node, but it is possible to include additional restrictions in what is considered an acceptable match, for example a specific egress network interface might be requested.

Given this FIB configuration, where column `Nr` is added just to make it easier to reference to a specific row:

Nr	Dst[/Plen]	Ifindex	Next-hop
1	8.8.8.8	3	192.168.1.1
2	127.0.0.1	1	
3	10.0.0.0/8	3	
4	192.168.1.0/24	2	
5	0.0.0.0/0	2	10.1.1.1

a lookup will match the destination address from top to bottom in the table above. The FIB entries are ordered on prefix length first, entries with longer prefix length must be considered first. All bits are significant in entries without explicit prefix length, so the first two entries above has a prefix length of 32 as IPv4 addresses are 32-bit in size (and IPv6 addresses are 128-bit).

FIB-nodes with the same prefix length does not really have to be ordered as a specific destination address will still only match exactly one node within a group of entries sharing the same prefix length. The network stack will still order elements sharing the same prefix length in ascending order of the network address to make the FIB easier to read for humans.

The ifindex field specifies the egress network interface to use for matching destination addresses. The next-hop field specifies the IP-address of the intermediate IP-node IP-datagrams must be sent to, absents of a next-hop attribute means that all matching IP-nodes are link-local to the sender.

As an example, a lookup of address 192.168.1.2 will match row 4. The constraint argument passed to **fibLookup( )** must in this case have set the destination address using **fibAttrDstSet( )**. The returned result will have at least the destination address of the matching FIB-node set, 192.168.1.0, the egress network interface index, 2, and prefix length, 24. All successful lookup will also return a next-hop, it will be set to the IP-address of the next IP-node which in the case of link-local destination addresses is the address used in the lookup. So the next-hop is 192.168.1.2 in this case. It will be 10.1.1.1 if the destination address, for example, is 147.11.176.59 as row 5 defines a set of IP-addresses reachable through the gateway 10.1.1.1.

Insertion of new entries into the FIB, **fibInsert( )**, must include a destination address or they will fail. Insertions lacking next-hop is treated as link-local, it is explicitly allowed to add the default route, 0.0.0.0/0, without a specific next-hop, the result is that the local node will use ARP/NDP for every IP-node matching the default route.

Removal of an FIB-node will remove all nodes matching the set of attribute passed to **fibRemove( )**. That feature can be used to, for example, remove all FIB-entries using a specific network interface ID by specify the interface index and nothing else.

Example

Lookup based soley on destination address, this is the most common use-case for the FIB

```
STATUS lookupExample
    (
    FIB * fib,                  /* Forward information base database */
    struct in6_addr * dst,      /* Address to lookup */
    struct in6_addr * nh;       /* Next-hop address */
    IFI             * ifi;      /* Egress interface index */
    )
    {
    FIB_ATTR    attr;
    FIB_ATTR    result;
    STATUS      status = ERROR;

    fibAttrInit (&attr);
    fibAttrDstSet (&attr, dst);
    if (fibLookup (fib, &attr, &result) == OK
        && fibAttrNhGet (&result, nh) == OK
        && fibAttrIfiGet (&result, ifi) == TRUE)
        {
        status = OK;
        }

    return status;
    }
```

Insert FIB-node to network fc00::/64 via 3ffe::1 on egress interface index 2

```
STATUS insertExample
    (
    FIB * fib           /* FIB database where the node should be inserted */
    )
    {
    struct in6_addr     dst;
    struct in6_addr     nh;
    FIB_ATTR            attr;

    (void)inet_pton (AF_INET6, &dst, "fc00::");
    (void)inet_pton (AF_INET6, &nh, "3ffe::1");
```

```
    fibAttrInit (&attr);
    fibAttrDstSet (&attr, &dst);
    fibAttrPlenSet (&attr, 64);
    fibAttrIfiSet (&attr, 2);
    fibAttrNhSet (&attr, &nh);
    return fibInsert (fib, &attr);
    }
```

Include files

fibLib.h

Name

fibInit( ) - initialize a forward information base instance

Layer

RTNET

Synopsis

```
STATUS fibInit                                  /* req: net_fibInit_1 */
    (
    FIB *  fib,       /**< storage for a forward information base instance */
    UINT32 capacity   /**< capacity of FIB database */
    )
```

Description

This routine initializes a forward information base instance. Created FIB cannot be destroyed and a single instance must not be initialized multiple times.

Returns

OK, or **ERROR** if there is not enough memory.

Errno

S_memLib_NOT_ENOUGH_MEMORY

There is not enough memory to allocate the FIB data-base.

S_semLib_NOT_ENOUGH_MEMORY

Not enough memory to initialize the generation lock.

See also

fibLib, **calloc( )**, **genInit( )**

Name

fibLookup( ) - try to lookup an node in a FIB instance

Layer

RTNET

Synopsis

```
STATUS fibLookup                                /* fib: net_fibLookup_1 */
    (
    FIB *            fib,          /**< FIB used for the lookup */
    const FIB_ATTR * constraint,   /**< Attributes that must match */
    FIB_ATTR *       result        /**< Attributes of the matching FIB node */
    )
```

Description

This routine tries to find an node inside [fib] matching the contraints supplied.

Returns

OK, or **ERROR** with `errno` set.

Errno

ENETUNREACH

No matching node found.

See also

fibLib

Name

fibInsert( ) - try to insert a new node into a FIB

Layer

RTNET

Synopsis

```
STATUS fibInsert                                /* req: fibInsert_1 */
    (
    FIB *            fib,   /**< FIB to insert into */
    const FIB_ATTR * attr   /**< Attributes of the node to insert */
    )
```

Description

This routine try to insert a new node into a FIB.

Returns

OK, or **ERROR** with `errno` set if the insert failed.

Errno

EEXIST

The FIB already contain something that matches attr.

EINVAL

The attr parameter does not include a destination address.

ENOMEM

The FIB is at maximum capacity.

See also

fibLib

Name

fibRemove( ) - Remove all FIB-nodes matching the contraint

Layer

RTNET

Synopsis

```
UINT32 fibRemove                                /* req: net_fibRemove_1 */
    (
    FIB *            fib,         /**< FIB to remove from */
    const FIB_ATTR * constraint   /**< Constraint FIB-nodes must match to be */
                                  /* removed */
    )
```

Description

This routine removes all FIB-nodes that matches the contraint.

Returns

The number of FIB-nodes removed.

Errno

Not Available

See also

fibLib

Name

fibForEach( ) - run a callback on each FIB node

Layer

RTNET

Synopsis

```
void fibForEach                                /* req: fibForEach_1 */
    (
    FIB *            fib,   /**< FIB to traverse */
    FIB_FOR_EACH_RTN rtn,   /**< routine to call for each FIB-node */
    void *           ctx    /**< user data to pass to 'rtn' */
    )
```

Description

This routine take an user-supplied routine and a user-supplied argument and call that routine with the argument and a FIB node for every FIB node in the database.

Returns

N/A

Errno

N/A

See also

fibLib

Name

fibAttrInit( ) - Initialize a new set of attributes.

Layer

RTNET

Synopsis

```
void fibAttrInit                               /* req: fibAttrInit_1 */
    (
    FIB_ATTR * attr   /**< Attribute to initialize */
    )
```

Description

This routine initialize a new set of attributes.

Returns

N/A

Errno

Not Available

See also

fibLib

Name

fibAttrDstGet( ) - Get destination address attribute

Layer

RTNET

Synopsis

```
const struct in6_addr * fibAttrDstGet           /* req: net_fibAttrDstGet_1 */
    (
    const FIB_ATTR * attr   /**< Set of attributes */
    )
```

Description

This routine try to get the destination address attribute.

Returns

Pointer to destination address, or **NULL** if the attribute is not set.

Errno

Not Available

See also

fibLib

Name

fibAttrDstSet( ) - Set destination address attribute

Layer

RTNET

Synopsis

```
void fibAttrDstSet                              /* req: net_fibAttrDstSet_1 */
    (
    FIB_ATTR *              attr,   /**< Set of attributes */
    const struct in6_addr * dst     /**< New destination address */
    )
```

Description

This routine set the destination address attribute.

Returns

N/A

Errno

Not Available

See also

fibLib

Name

fibAttrSrcGet( ) - Get source address attribute

Layer

RTNET

Synopsis

```
const struct in6_addr * fibAttrSrcGet           /* req: net_fibAttrSrcGet_1 */
    (
    const FIB_ATTR * attr   /**< Set of attributes */
    )
```

Description

This routine try to get the source address attribute.

Returns

Pointer to source address, or **NULL** if the attribute is not set.

Errno

Not Available

See also

fibLib

Name

fibAttrSrcSet( ) - Set source address attribute

Layer

RTNET

Synopsis

```
void fibAttrSrcSet                              /* req: net_fibAttrSrcSet_1 */
    (
    FIB_ATTR *              attr,   /**< Set of attributes */
    const struct in6_addr * src     /**< New destination address */
    )
```

Description

This routine set the source address attribute.

Returns

N/A

Errno

Not Available

See also

fibLib

Name

fibAttrNhGet( ) - Get next-hop address attribute

Layer

RTNET

Synopsis

```
const struct in6_addr * fibAttrNhGet            /* req: net_fibAttrNhGet_1 */
    (
    const FIB_ATTR * attr   /**< Set of attributes */
    )
```

Description

This routine try to get the next-hop address attribute.

Returns

Pointer to next-hop, or **NULL** if the attribute is not set.

Errno

Not Available

See also

fibLib

Name

fibAttrNhSet( ) - Set next-hop attribute

Layer

RTNET

Synopsis

```
void fibAttrNhSet                                /* req: net_fibAttrNhSet_1 */
    (
    FIB_ATTR *              attr,   /**< Set of attributes */
    const struct in6_addr * nh      /**< New next-hop address */
    )
```

Description

This routine set the next-hop attribute.

Returns

N/A

Errno

Not Available

See also

fibLib

Name

fibAttrIfiGet( ) - Get network interface index attribute

Layer

RTNET

Synopsis

```
BOOL fibAttrIfiGet                              /* req: net_fibAttrIfiGet_1 */
    (
    const FIB_ATTR * attr,   /**< Set of attributes */
    IFI *            ifi     /**< Pointer to interface index storage */
    )
```

Description

This routine try to get the network interface attribute.

Returns

TRUE, or **FALSE** if the attribute is not present.

Errno

Not Available

See also

fibLib

Name

fibAttrIfiSet( ) - Set network interface index attribute

Layer

RTNET

Synopsis

```
void fibAttrIfiSet                              /* req: net_fibAttrIfiSet_1 */
    (
    FIB_ATTR * attr,   /**< Set of attributes */
    IFI        ifi     /**< New network interface index attribute */
    )
```

Description

This routine set the network interface index attribute.

Returns

N/A

Errno

Not Available

See also

fibLib

Name

fibAttrPlenGet( ) - Get prefix length attribute

Layer

RTNET

Synopsis

```
BOOL fibAttrPlenGet                             /* req: net_fibAttrPlenGet_1 */
    (
    const FIB_ATTR * attr,   /**< Set of attributes */
    UINT8 *          plen    /**< Pointer to prefix length storage */
    )
```

Description

This routine try to get the prefix length attribut.

Returns

TRUE, or **FALSE** if the attribute set does not contain prefix length.

Errno

Not Available

See also

fibLib

Name

fibAttrPlenSet( ) - Set prefix length attribute

Layer

RTNET

Synopsis

```
void fibAttrPlenSet                             /* req: net_fibAttrPlenSet_1 */
    (
```

```
    FIB_ATTR * attr,    /**< Set of attributes */
    UINT8      plen     /**< New prefix length attribute */
    )
```

Description

This routine set the prefix length attribute.

Returns

N/A

Errno

Not Available

See also

fibLib

Name

fibAttrNlenGet( ) - Get prefix length attribute

Layer

RTNET

Synopsis

```
BOOL fibAttrNlenGet                             /* req: net_fibAttrNlenGet_1 */
    (
    const FIB_ATTR * attr,    /**< Set of attributes */
    UINT8 *          nlen     /**< Pointer to prefix length storage */
    )
```

Description

This routine try to get the prefix length attribut.

Returns

TRUE, or **FALSE** if the attribute set does not contain prefix length.

Errno

Not Available

See also

fibLib

Name

fibAttrNlenSet( ) - Set prefix length attribute

Layer

RTNET

Synopsis

```
void fibAttrNlenSet                             /* req: net_fibAttrNlenSet_1 */
    (
    FIB_ATTR * attr,   /**< Set of attributes */
    UINT8      nlen    /**< New prefix length attribute */
    )
```

Description

This routine set the prefix length attribute.

Returns

N/A

Errno

Not Available

See also

fibLib

10. FIBSHOW

Name

fibShow - C-interpreter FIB show command

Layer

RTNET

Routines

fibShow( ) - show all FIB entries. **routeShow( )** - show all route entries

Description

This module contain a routine that print a list of the IPv4 FIB-entries that the network stack is currently using.

Include files

N/A

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

fibShow( ) - show all FIB entries.

Layer

RTNET

Synopsis

```
INT32 fibShow (void)
```

Description

This routine shows all FIB entries in the main-table.

Returns

Number of entries printed.

Errno

N/A

See also

fibShow

Name

routeShow( ) - show all route entries

Layer

RTNET

Synopsis

```
INT32 routeShow (void)
```

Description

This routine shows all route entries in the main-table.

Returns

Number of entries printed.

Errno

N/A

See also

fibShow

11. FTPLIB

Name

ftpLib - File Transfer Protocol (FTP) library

Layer

IPNET_COREIP

Routines

ftpLibInit( ) - initialize the **ftpLib( )** utility **ftpCommand( )** - send an FTP command and get the reply **ftpCommandEnhanced( )** - send an FTP command and get the complete RFC reply code **ftpXfer( )** - initiate a transfer via FTP **ftpReplyGet( )** - get an FTP command reply **ftpReplyGetEnhanced( )** - get an FTP command reply **ftpHookup( )** - get a control connection to the FTP server on a specified host **ftpLogin( )** - log in to a remote FTP server **ftpDataConnInitPassiveMode( )** - initialize an FTP data connection using PASV mode **ftpDataConnInit( )** - initialize an FTP data connection using PORT mode **ftpDataConnGet( )** - get a completed FTP data connection **ftpLs( )** - list directory contents via FTP **ftpLibDebugOptionsSet( )** - set the debug level of the ftp library routines **ftpTransientConfigSet( )** - set parameters for host **FTP_TRANSIENT** responses **ftpTransientConfigGet( )** - get parameters for host **FTP_TRANSIENT** responses **ftpTransientFatalInstall( )** - set applette to stop FTP transient host responses

Description

This library provides facilities for transferring files to and from a host via File Transfer Protocol (FTP). This library implements only the "client" side of the FTP facilities.

Ftp in vxworks

For most purposes, you should access the services of **ftpLib** by means of **netDrv**, a VxWorks I/O driver that supports transparent access to remote files by means of standard I/O system calls. Before attempting to access **ftpLib** services directly, you should check whether **netDrv** already provides the same access for less trouble.

High-level interface

The routines **ftpXfer( )** and **ftpReplyGet( )** provide the highest level of direct interface to FTP. The routine **ftpXfer( )** connects to a specified remote FTP server, logs in under a specified user name, and initiates a specified data transfer command. The routine **ftpReplyGet( )** receives control reply messages sent by the remote FTP server in response to the commands sent.

Low-level interface

The routines **ftpHookup( )**, **ftpLogin( )**, **ftpDataConnInit( )**, **ftpDataConnGet( )**, **ftpCommand( )**, **ftpCommandEnhanced( )** provide the primitives necessary to create and use control and data connections to remote FTP servers. The following example shows how to use these low-level routines. It implements roughly the same function as **ftpXfer( )**.

```
char *host, *user, *passwd, *acct, *dirname, *filename;
int ctrlSock = ERROR; /* This is the control socket file descriptor */
int dataSock = ERROR; /* This is the data path socket file descriptor */

if (((ctrlSock = ftpHookup (host)) == ERROR)                                  ||
    (ftpLogin (ctrlSock, user, passwd, acct) == ERROR)                        ||
    (ftpCommand (ctrlSock, "TYPE I", 0, 0, 0, 0, 0, 0) != FTP_COMPLETE)       ||
    (ftpCommand (ctrlSock, "CWD %s", dirname, 0, 0, 0, 0, 0) != FTP_COMPLETE) ||
    ((dataSock = ftpDataConnInit (ctrlSock)) == ERROR)                        ||
```

```
    (ftpCommand (ctrlSock, "RETR %s", filename, 0, 0, 0, 0, 0) != FTP_PRELIM) ||
    ((dataSock = ftpDataConnGet (dataSock)) == ERROR))
    {
    /* an error occurred; close any open sockets and return */

    if (ctrlSock != ERROR)
        close (ctrlSock);
    if (dataSock != ERROR)
        close (dataSock);
    return (ERROR);
    }
```

For even lower-level access, please note that the sockets provided by **ftpHookup( )** and **ftpDataConnInit( )** are standard TCP/IP sockets. Developers may implement **read( )**, **write( )** and **select( )** calls using these sockets for maximum flexibility.

To use this feature, include the following component: **INCLUDE_FTP**

Tuning for multiple file access

Please note that accessing multiple files simultaneously may require increasing the memory available to the network stack. You can examine memory requirements by using **netStackSysPoolShow( )** and **netStackDataPoolShow( )** before opening and after closing files.

You may need to modify the following macro definitions according to your specific memory requirements:

NUM_64 NUM_128 NUM_256 NUM_512 NUM_1024 NUM_2048 NUM_SYS_64 NUM_SYS_128 NUM_SYS_256 NUM_SYS_512 NUM_SYS_1024 NUM_SYS_2048

Please also note that each concurrent file access requires three file descriptors (File, Control and Socket). The following macro definition may need modification per your application: **NUM_FILES**

Developers are encouraged to enable the error reporting facility during debugging using the function **ftpLibDebugOptionsSet( )**. The output is displayed via the logging facility.

Include files

ftpLib.h

See also

netDrv, **logLib**, VxWorks 7 File Systems and I/O Programmer's Guide, VxWorks 7 Platform Programmer's Guide, VxWorks 7 FTP Programmer's Guide

Name

ftpLibInit( ) - initialize the **ftpLib( )** utility

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpLibInit
    (
```

```
    long timeout
    )
```

Description

This routine allocates resources used by the **ftpLib** utility. It is called automatically when **INCLUDE_FTP** is defined.

Returns

OK

Errno

Not Available

See also

ftpLib

Name

ftpCommand( ) - send an FTP command and get the reply

Layer

IPNET_COREIP

Synopsis

```
int ftpCommand
    (
    int           ctrlSock,   /* fd of control connection socket */
    char          *fmt,       /* format string of command to send */
    _Vx_usr_arg_t arg1,       /* first of six args to format string */
    _Vx_usr_arg_t arg2,
    _Vx_usr_arg_t arg3,
    _Vx_usr_arg_t arg4,
    _Vx_usr_arg_t arg5,
    _Vx_usr_arg_t arg6
    )
```

Description

This command has been superceded by **ftpCommandEnhanced( )**

This routine sends the specified command on the specified socket, which should be a control connection to a remote FTP server. The command is specified as a string in **printf( )** format with up to six arguments.

After the command is sent, **ftpCommand( )** waits for the reply from the remote server. The FTP reply code is returned in the same way as in **ftpReplyGet( )**.

Example

```
ftpCommand (ctrlSock, "TYPE I", 0, 0, 0, 0, 0, 0);      /* image-type xfer */
ftpCommand (ctrlSock, "STOR %s", file, 0, 0, 0, 0, 0); /* init file write */
```

Returns

1 = **FTP_PRELIM** (positive preliminary) 2 = **FTP_COMPLETE** (positive completion)
3 = **FTP_CONTINUE** (positive intermediate) 4 = **FTP_TRANSIENT** (transient negative completion)
5 = **FTP_ERROR** (permanent negative completion)

ERROR if there is a read/write error or an unexpected **EOF**.

Errno

Not Available

See also

ftpLib, **ftpReplyGet( )**

Name

ftpCommandEnhanced( ) - send an FTP command and get the complete RFC reply code

Layer

IPNET_COREIP

Synopsis

```
int ftpCommandEnhanced
    (
    int           ctrlSock,           /* fd of control connection socket */
    char          *fmt,               /* format string of command to send */
    _Vx_usr_arg_t arg1,               /* first of six args to format string */
    _Vx_usr_arg_t arg2,
    _Vx_usr_arg_t arg3,
    _Vx_usr_arg_t arg4,
    _Vx_usr_arg_t arg5,
    _Vx_usr_arg_t arg6,
    char          *replyString,       /* storage for the last line of the server response or N
ULL */
    int           replyStringLength   /* Maximum character length of the replyString */
    )
```

Description

This command supercedes **ftpCommand( )**

This routine sends the specified command on the specified socket, which should be a control connection to a remote FTP server. The command is specified as a string in **printf( )** format with up to six arguments.

After the command is sent, **ftpCommand( )** waits for the reply from the remote server. The FTP reply code is returned in the same way as in **ftpReplyGetEnhanced( )**.

Example

```
ftpCommandEnhanced (ctrlSock, "TYPE I", 0, 0, 0, 0, 0, 0, 0, 0);     /* image-type xfer */
ftpCommandEnhanced (ctrlSock, "STOR %s", file, 0, 0, 0, 0, 0, 0, 0); /* init file write */
ftpCommandEnhanced (ctrlSock, "PASV", file, 0, 0, 0, 0, 0, reply, rplyLen); /* Get port */
```

Returns

The complete FTP response code (see RFC #959)

ERROR if there is a read/write error or an unexpected **EOF**.

Errno

Not Available

See also

ftpLib, **ftpReplyGetEnhanced( )**, **ftpReplyGet( )**

Name

ftpXfer( ) - initiate a transfer via FTP

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpXfer
    (
    char *host,         /* name of server host */
    char *user,         /* user name for host login */
    char *passwd,       /* password for host login */
    char *acct,         /* account for host login */
    char *cmd,          /* command to send to host */
    char *dirname,      /* directory to 'cd' to before sending command */
    char *filename,     /* filename to send with command */
    int  *pCtrlSock,    /* where to return control socket fd */
    int  *pDataSock     /* where to return data socket fd, */
                        /* (NULL == don't open data connection) */
    )
```

Description

This routine initiates a transfer via a remote FTP server in the following order:

(1) Establishes a connection to the FTP server on the specified host.

(2) Logs in with the specified user name, password, and account, as necessary for the particular host.

(3) Sets the transfer type to image by sending the command "TYPE I".

(4) Changes to the specified directory by sending the command "CWD dirname".

(5) Sends the specified transfer command with the specified filename as an argument, and establishes a data connection. Typical transfer commands are "STOR %s", to write to a remote file, or "RETR %s", to read a remote file.

The resulting control and data connection file descriptors are returned via pCtrlSock and pDataSock, respectively.

After calling this routine, the data can be read or written to the remote server by reading or writing on the file descriptor returned in pDataSock. When all incoming data has been read (as indicated by an **EOF** when reading the data socket) and/or all outgoing data has been written, the data socket fd should be closed. The routine **ftpReplyGet( )** should then be called to receive the final reply on the control socket, after which the control socket should be closed.

If the FTP command does not involve data transfer, pDataSock should be **NULL**, in which case no data connection will be established. The only FTP commands supported for this case are DELE, RMD, and MKD.

Example

The following code fragment reads the file "/usr/fred/myfile" from the host "server", logged in as user "fred", with password "magic" and no account name.

```
#include "vxWorks.h"
#include <ftpLib.h>

int       ctrlSock;
int       dataSock;
char      buf [512];
int       nBytes;
STATUS    status;

if (ftpXfer ("server", "fred", "magic", "",
             "RETR %s", "/usr/fred", "myfile",
             &ctrlSock, &dataSock) == ERROR)
    return (ERROR);

while ((nBytes = read (dataSock, buf, sizeof (buf))) > 0)
    {
    ...
    }

close (dataSock);

if (nBytes < 0)             /* read error? */
    status = ERROR;

if (ftpReplyGet (ctrlSock, TRUE) != FTP_COMPLETE)
    status = ERROR;

if (ftpCommand (ctrlSock, "QUIT", 0, 0, 0, 0, 0, 0) != FTP_COMPLETE)
```

```
        status = ERROR;

    close (ctrlSock);
```

Returns

OK, or **ERROR** if any socket cannot be created or if a connection cannot be made.

Errno

Not Available

See also

ftpLib, **ftpReplyGet( )**

Name

ftpReplyGet( ) - get an FTP command reply

Layer

IPNET_COREIP

Synopsis

```
int ftpReplyGet
    (
    int  ctrlSock,    /* control socket fd of FTP connection */
    BOOL expecteof    /* TRUE = EOF expected, FALSE = EOF is error */
    )
```

Description

This routine has been superceded by **ftpReplyGetEnhanced( )**

This routine gets a command reply on the specified control socket.

The three-digit reply code from the first line is saved and interpreted. The left-most digit of the reply code identifies the type of code (see RETURNS below).

If the reply code indicates an error, the entire reply is printed if the ftp error printing is enabled (see the manual entry for **ftpLibDebugOptionsSet( )**).

If an **EOF** is encountered on the specified control socket, but no **EOF** was expected (expecteof == **FALSE**), then **ERROR** is returned.

Returns

1 = **FTP_PRELIM** (positive preliminary) 2 = **FTP_COMPLETE** (positive completion)
3 = **FTP_CONTINUE** (positive intermediate) 4 = **FTP_TRANSIENT** (transient negative completion)
5 = **FTP_ERROR** (permanent negative completion)

ERROR if there is a read/write error or an unexpected **EOF**.

Errno

Not Available

See also

ftpLib

Name

ftpReplyGetEnhanced( ) - get an FTP command reply

Layer

IPNET_COREIP

Synopsis

```
int ftpReplyGetEnhanced
    (
    int  ctrlSock,          /* control socket fd of FTP connection */
    BOOL expecteof,         /* TRUE = EOF expected, FALSE = EOF is error */
    char *replyString,      /* Location to store text of reply, or NULL */
    int  stringLengthMax    /* Maximum length of reply (not including NULL) */
    )
```

Description

This routine supercedes **ftpReplyGet( )**

This routine gets a command reply on the specified control socket.

The three-digit reply code from the first line is saved and interpreted. The left-most digit of the reply code identifies the type of code (see RETURNS below).

If the reply code indicates an error, the entire reply is printed if the ftp error printing is enabled (see the manual entry for **ftpLibDebugOptionsSet( )**).

The last line of text retrieved from the servers response is stored in the location specified by replyString. If replyString is **NULL** the parameter is ignored.

If an **EOF** is encountered on the specified control socket, but no **EOF** was expected (expecteof == **FALSE**), then **ERROR** is returned.

Returns

The complete FTP response code (see RFC #959)

ERROR if there is a read/write error or an unexpected **EOF**.

Errno

Not Available

See also

ftpLib

Name

ftpHookup( ) - get a control connection to the FTP server on a specified host

Layer

IPNET_COREIP

Synopsis

```
int ftpHookup
    (
    char *host   /* server host name or inet address */
    )
```

Description

This routine establishes a control connection to the FTP server on the specified host. This is the first step in interacting with a remote FTP server at the lowest level. (For a higher-level interaction with a remote FTP server, see the manual entry for **ftpXfer( )**.)

Returns

The file descriptor of the control socket, or **ERROR** if the Internet address or the host name is invalid, if a socket could not be created, or if a connection could not be made.

Errno

Not Available

See also

ftpLib, **ftpLogin( )**, **ftpXfer( )**

Name

ftpLogin( ) - log in to a remote FTP server

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpLogin
    (
    FAST int ctrlSock,   /* fd of login control socket */
    char     *user,      /* user name for host login */
    char     *passwd,    /* password for host login */
```

```
    char     *account    /* account for host login */
    )
```

Description

This routine logs in to a remote server with the specified user name, password, and account name, as required by the specific remote host. This is typically the next step after calling **ftpHookup( )** in interacting with a remote FTP server at the lowest level. (For a higher-level interaction with a remote FTP server, see the manual entry for **ftpXfer( )**).

Returns

OK, or **ERROR** if the routine is unable to log in.

Errno

Not Available

See also

ftpLib, **ftpHookup( )**, **ftpXfer( )**

Name

ftpDataConnInitPassiveMode( ) - initialize an FTP data connection using PASV mode

Layer

IPNET_COREIP

Synopsis

```
int ftpDataConnInitPassiveMode
    (
    int ctrlSock   /* fd of associated control socket */
    )
```

Description

This routine sets up the client side of a data connection for the specified control connection. It issues a PASV command and attempts to connect to the host-specified port. If the host responds that it can not process the PASV command (command not supported) or fails to recognize the command, it will return **ERROR**.

This routine must be called before the data-transfer command is sent; otherwise, the server's connect may fail.

This routine is called after **ftpHookup( )** and **ftpLogin( )** to establish a connection with a remote FTP server a low level. (For a higher-level interaction with a remote FTP server, see **ftpXfer( )**.)

This function is preferred over **ftpDataConnInit( )** because the remote system must preserve old port connection pairs even if the target system suffers from a reboot (2MSL). Using PASV we encourage the host's selection of a fresh port.

Returns

The file descriptor of the data socket created, or **ERROR**.

Errno

Not Available

See also

ftpLib, **ftpHookup( )**, **ftpLogin( )**, **ftpCommandEnhanced( )**, **ftpXfer( )**, **ftpConnInit( )**

Name

ftpDataConnInit( ) - initialize an FTP data connection using PORT mode

Layer

IPNET_COREIP

Synopsis

```
int ftpDataConnInit
    (
    int ctrlSock   /* fd of associated control socket */
    )
```

Description

This routine sets up the client side of a data connection for the specified control connection using the PORT command. It creates the data port, informs the remote FTP server of the data port address, and listens on that data port. The server will then connect to this data port in response to a subsequent data-transfer command sent on the control connection (see the manual entry for **ftpCommand( )**).

This routine must be called before the data-transfer command is sent; otherwise, the server's connect may fail.

This routine is called after **ftpHookup( )** and **ftpLogin( )** to establish a connection with a remote FTP server at the lowest level. (For a higher-level interaction with a remote FTP server, see **ftpXfer( )**.)

Please note that **ftpDataConnInitPassiveMode( )** is recommended instead of **ftpDataConnInit( )**.

Returns

The file descriptor of the data socket created, or **ERROR**.

Errno

Not Available

See also

ftpLib, **ftpDataConnInitPassiveMode( )**, **ftpHookup( )**, **ftpLogin( )**, **ftpCommand( )**, **ftpXfer( )**

Name

ftpDataConnGet( ) - get a completed FTP data connection

Layer

IPNET_COREIP

Synopsis

```
int ftpDataConnGet
    (
    int dataSock   /* fd of data socket on which to await connection */
    )
```

Description

This routine completes a data connection initiated by a call to **ftpDataConnInit( )**. It waits for a connection on the specified socket from the remote FTP server. The specified socket should be the one returned by **ftpDataConnInit( )**. The connection is established on a new socket, whose file descriptor is returned as the result of this function. The original socket, specified in the argument to this routine, is closed.

Usually this routine is called after **ftpDataConnInit( )** and **ftpCommand( )** to initiate a data transfer from/to the remote FTP server.

Returns

The file descriptor of the new data socket, or **ERROR** if the connection failed.

Errno

Not Available

See also

ftpLib, **ftpDataConnInit( )**, **ftpCommand( )**

Name

ftpLs( ) - list directory contents via FTP

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpLs
    (
    char * dirName   /* name of directory to list */
    )
```

Description

This routine lists the contents of a directory. The content list is obtained via an NLST FTP transaction.

The local device name must be the same as the remote host name with a colon ":" as a suffix. (For example "wrs:" is the device name for the "wrs" host.)

Returns

OK, or **ERROR** if could not open directory.

Errno

Not Available

See also

ftpLib

Name

ftpLibDebugOptionsSet( ) - set the debug level of the ftp library routines

Layer

IPNET_COREIP

Synopsis

```
void ftpLibDebugOptionsSet
    (
    UINT32 debugLevel
    )
```

Description

This routine enables the debugging of ftp transactions using the ftp library.

Debugging Level	Meaning
FTPL_DEBUG_OFF	No debugging messages.
FTPL_DEBUG_INCOMING	Display all incoming responses.
FTPL_DEBUG_OUTGOING	Display all outgoing commands.
FTPL_DEBUG_ERRORS	Display warnings and errors

Example

```
ftpLibDebugOptionsSet (FTPL_DEBUG_ERRORS);     /* Display any runtime errors */
ftpLibDebugOptionsSet (FTPL_DEBUG_OUTGOING);   /* Display outgoing commands */
ftpLibDebugOptionsSet (FTPL_DEBUG_INCOMING);   /* Display incoming replies */
ftpLibDebugOptionsSet (FTPL_DEBUG_INCOMING |   /* Display both commands and */
                       FTPL_DEBUG_OUTGOING);   /*         replies */
```

Returns

N/A

Errno

Not Available

See also

ftpLib

Name

ftpTransientConfigSet( ) - set parameters for host **FTP_TRANSIENT** responses

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpTransientConfigSet
    (
    UINT32 maxRetryCount,   /* The maximum number of attempts to retry */
    UINT32 retryInterval    /* time (in system clock ticks) between retries */
    )
```

Description

This routine adjusts the delay between retries in response to receiving **FTP_PRELIM** and the maximum retry count permitted before failing.

Returns

OK

Errno

Not Available

See also

ftpLib

Name

ftpTransientConfigGet( ) - get parameters for host **FTP_TRANSIENT** responses

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpTransientConfigGet
    (
    UINT32 *maxRetryCount,   /* The maximum number of attempts to retry */
    UINT32 *retryInterval    /* time (in system clock ticks) between retries */
    )
```

Description

This routine retrieves the delay between retries in response to receiving **FTP_TRANSIENT** and the maximum retry count permitted before failing.

Returns

OK

Errno

Not Available

See also

ftpLib, ftpTransientConfigSet, **tickLib**

Name

ftpTransientFatalInstall( ) - set applette to stop FTP transient host responses

Layer

IPNET_COREIP

Synopsis

```
STATUS ftpTransientFatalInstall
    (
    FUNCPTR pApplette   /* function that returns TRUE or FALSE */
    )
```

Description

The routine installs a function which will determine if a transient response should be fatal. Some ftp servers incorrectly use **transient** responses instead of **error** to describe conditions such as **disk full**.

Returns

OK if the installation is successful, or **ERROR** if the installation fails.

Errno

Not Available

See also

ftpLib, **ftpTransientConfigSet( )**, **ftpTransientFatal( )** in, **target/config/comps/src/net/usrFtp.c**

12. GENLIB

Name

genLib - generation lock library

Layer

RTNET

Routines

genInit( ) - initialize a generation lock **genRdCtxInit( )** - initializes a generation lock context for reader **genRdBegin( )** - begin a read transaction **genRdFinish( )** - try to finish a read transaction **genWrTake( )** - take a generation lock for writing **genWrForceNext( )** - move to next generation with write lock held **genWrGive( )** - give a generation lock held for writing

Description

Generation locks are a very specialized type of synchronization highly optimized for data structures that are read far more often than they are modified.

Several restriction applies to data structures that should be protected by a generation lock. First one is that no pointers must be returned that points to any of the protected data, all returned data must be copied by-value into an caller-private area supplied by the caller. Second restriction is that the memory area holding the protected data must be valid to read even after it has been replaced by a new generation. The benefit one get from living with all those restrictions is a lock mechanism that does not modify any of the cache-lines that must be shared between CPU-cores for them to share the lock and the data the lock protects as long as the data is only read.

Example of how the second restriction can be satisfied is by using two memory areas, one represents the current generation and the other represent the next generation. A reader might fetch data from an old generation, something that is then detected at the end of the protected read block. All read data must be discared at that point and the read operation must be restarted.

A read context is created for every read operation, the context keep track of whether the read operation has been restarted, the second attempt is a lot more expensive but is guaranteed to succeed to avoid live-lock of readers.

Writers get exclusive access to the lock with respect to other writer, but may run concurrently with readers. The generation is incremented when the lock is released and the generation ID must be used as the selected for determine which memory region that hold the current generation of data as that is the only way to guarantee that generation switche is atomic. The generation ID is also guaranteed to increment sequentually by one each time the write lock is released.

The functions **genRdBegin( )** and **genWrTake( )** are acquire barriers or stronger. The functions **genRdFinish( )** and **genWrGive( )** are release barriers or stronger.

Example

Reader example, the returned value is a snapshot of what was stored in the **nth** position at the time of calling. The operation has to be restarted if the generation changed since the value read at that point might never existed in any generation:

```
LOCAL UINT32    data[2][10]; /* two areas of 10 integers */
LOCAL GEN_LOCK  lock;        /* protects [data] */

UINT32 readNth
    (
    UINT32 nth,    /* element to read */
```

```
    )
    {
    UINT32              val;
    GENERATION          gen;
    GEN_LOCK_RD_CTX     ctx;

    genRdCtxInit (&ctx, &lock);

    do
        {
            gen = genRdBegin (&ctx);
            val = data [gen & 1] [nth];
        }
        while (!genRdFinish (&ctx));

    return val;
    }
```

Writers will receive the current generation ID when the write lock is acquired. Updating the **nth** element in the same example used above might look like this

```
LOCAL UINT32    data[2][10]; /* two areas of 10 integers */
LOCAL GEN_LOCK  lock;        /* protects [data] */

void writeNth
    (
    UINT32 nth,    /* element to read */
    UINT32 newVal, /* new value to at the [nth] positiion */
    )
    {
    UINT32      curIdx;         /* [data] index for current generation */
    UINT32      nextIdx;        /* [data] index for next generation */

    curIdx = genWrTake (&lock) & 1;

    nextIdx = curIdx ^ 1;
    memcpy (data [nextIdx], data [curIdx], sizeof data[0]);

    data [nextIdx] [nth] = newVal;

    genWrGive(&lock, TRUE);
    }
```

The benefits of the generation lock are perfect scaling on SMP as long as a protected structure is only read, readers do not cause any cache-line bouncing when acquire and releasing the lock. Another benefit is that a high priority task that want to read data will never be blocked, not even in the event of concurrent writers if running on a uni-processor system.

Tasks are protected from deletion while holding the lock for writing. Acquiring the lock for writing is protected from priority inversion.

Reusing a node

A cache, like the neighbor- and peer-cache, using the generation lock might have to reuse an existing node. One very critical step in the reuse process is to ensure that no existing node get its state modified before it is certain that no reader concurrently read the state being modified and also accept it as valid when calling **genRdFinish( )**. A possible algorithm to support reuse is

to first mark the to-be-reused node as free, move to the next generation, then initialize the entry, add it and move to yet another generation where the new entry is present.

One case that would potentially return incorrect data without the intermediate generation where the node is removed from the cache is were a reader started a read transaction, got hold of the node that just happen to be the one just about to be reused. The writer could then start to modify the node after the reader found it but before the relevant data is copied. The reader could then finish the read before the generation is increase, which would accept the read data as valid. The intermediate generation guarantee that the node state is either not modified, i.e. reader finishes before the intermediate generation is visible, or the potentially inconsistent data is not accepted as the generation has changed before the read transaction finish.

In order to prevent a race-condition where the writer frees a node but some other task grab the write lock and the node before the write lock can be re-acquired, the function **genWrForceNext( )** exist to move to the new generation in a sequentially consistent way while holding the write lock.

Include files

genLib.h

Name

genInit( ) - initialize a generation lock

Layer

RTNET

Synopsis

```
STATUS genInit                                  /* req: net_genInit_1 */
    (
    GEN_LOCK * lock   /**< generation lock to initialize */
    )
```

Description

This routine initializes a generation lock. The lock cannot be destroyed and must not be initialized multiple times.

Returns

OK, or **ERROR** if the lock cannot be initialized.

Errno

S_semLib_NOT_ENOUGH_MEMORY

Not enough memory available to create the lock.

See also

genLib

Name

genRdCtxInit( ) - initializes a generation lock context for reader

Layer

RTNET

Synopsis

```
void genRdCtxInit                                /* req: net_genRdCtxInit_1 */
    (
    GEN_LOCK_RD_CTX * ctx,    /**< storage for callee read context */
    GEN_LOCK *        lock    /**< lock to associate with [ctx] */
    )
```

Description

This routine initializes a generation lock context for a task that want to read a snapshot of the data protected by the lock.

Returns

N/A

Errno

Not Available

See also

genLib

Name

genRdBegin( ) - begin a read transaction

Layer

RTNET

Synopsis

```
GENERATION genRdBegin                            /* req: net_getRdBegin_1 */
    (
    GEN_LOCK_RD_CTX * ctx    /**< reader context for generation lock */
    )
```

Description

This routine begin a read transaction on data protected by a generation lock.

Returns

The current generation

Errno

Not Available

See also

genLib

Name

genRdFinish( ) - try to finish a read transaction

Layer

RTNET

Synopsis

```
BOOL genRdFinish                                /* req: net_genRdFinish_1 */
    (
    GEN_LOCK_RD_CTX * ctx   /**< reader context for generation lock */
    )
```

Description

This routine try to finish a read transaction on data protected by a generation lock.

Returns

TRUE if the generation was stable throughout the transaction, or **FALSE** if the generation changed which means that the transaction must be restarted and the read data must be discarded.

Errno

Not Available

See also

genLib

Name

genWrTake( ) - take a generation lock for writing

Layer

RTNET

Synopsis

```
GENERATION genWrTake
    (
```

```
    GEN_LOCK * lock   /**< generation lock to take for writing */
    )
```

Description

This routine take a generation lock for writing. The write lock is exclusive with respect to other writers, but the writer may run concurrently with readers.

Returns

The current generation

Errno

Not Available

See also

genLib

Name

genWrForceNext( ) - move to next generation with write lock held

Layer

RTNET

Synopsis

```
GENERATION genWrForceNext
    (
    GEN_LOCK * lock   /**< Generation lock to give */
    )
```

Description

This routine must be called with the write lock held, it move the lock to the next generation with sequentially consistent semantic.

Returns

The new generation

Errno

Not Available

See also

genLib

Name

genWrGive( ) - give a generation lock held for writing

Layer

RTNET

Synopsis

```
void genWrGive
    (
    GEN_LOCK * lock,     /**< Generation lock to give */
    BOOL       newGen    /**< TRUE if there is a new generation of the */
                         /* protected data, FALSE otherwise */
    )
```

Description

This routine give a generation lock held for writing, readers that run concurrently with this call will fail their read transaction and restart to get the new state of the protected data.

Returns

N/A

Errno

Not Available

See also

genLib

13. GETADDRINFO

Name

getaddrinfo - Hostname-Internet address-Translation library

Layer

IPNET_USRSPACE

Routines

getaddrinfo( ) - node name-to-address translation **gai_strerror( )** - convert a numeric error code into a string **freeaddrinfo( )** - free allocated memory for structure addrinfo

Description

This module provides a backwards compatible routine **getaddrinfo( )**, a function that provides for IP node name-address-translation.

Include files

netdb.h

See also

VxWorks 7 Network Stack Programmer's Guide

Name

getaddrinfo( ) - node name-to-address translation

Layer

IPNET_USRSPACE

Synopsis

```
int getaddrinfo
    (
    const char *            pHostname,
    const char *            pServname,
    const struct addrinfo * pHints,
    struct addrinfo **      ppRes
    )
```

Description

This routine needs an IP node name (for example, the host or router node), an IP address in hostname, and/or a service name (such as "ftp") in servname. The routine provides a linked list of **addrinfo** structures in res. res must be freed with **freeaddrinfo** after use to avoid memory leaks. The address information is returned in network byte order.

The search can be narrowed by specifying different flags for hints or can be **NULL**. (See below). The search starts in the host table; if the search cannot be satisfied there, a DNS request is carried out (if a DNS server has been initialized).

The **addrinfo** structure is as follows:

```
struct addrinfo
   {
   int     ai_flags;         -> AI_PASSIVE, ...
   int     ai_family;        -> AF_INET, AF_INET6, AF_UNSPEC
   int     ai_socktype;      -> SOCK_STREAM, SOCK_DGRAM, SOCK_RAW, ANY
   int     ai_protocol;      -> IPPROTO_TCP, IPPROTO_ICMP, ...
   size_t  ai_addrlen;       -> length of ai_addr
                                     Don't use for hints!
   char   *ai_canonname;     -> ptr to canonical name of host
                                     Don't use for hints!
   struct sockaddr *ai_addr; -> ptr to socket address structure
                                     Don't use for hints!
   struct addrinfo *ai_next; -> ptr to next structure in linked list
                                     Don't use for hints!
   };
```

The following flags are defined:

AI_PASSIVE

Specifies the returned address when the hostname is null. If this flag is specified the returned address must be suitable for use in a call to **bind( )** by a server accepting incoming connections; the address will be **INADDR_ANY** or IN6ADDR_ANY_INIT in this case. Otherwise, the returned address must be suitable for a call to **connect( )** (or **sendto( )** or **sendmsg( )** for a connectionless protocol), and the address in this case will be the loopback address. This flag is ignored if the hostname is not null.

AI_CANONNAME

If this flag is set and hostname is not **NULL**, then the ai_canonname member of the first addrinfo returned will point to a string containing the canonical name corresponding to hostname. However, if the canonical name cannot be located or hostname is a numeric host string, then ai_canonname will be hostname or a string with the same contents.

AI_NUMERICHOST

When this flag is set and a non-**NULL** hostname is provided, the hostname must be a numeric host address string, or **EAI_NONAME** will be returned. This flag prevents using any type of name resolution service (for example, DNS).

AI_V4MAPPED

If no IPv6 record is found in either the host table or DNS, a request for IPv4 records is sent and the found IPv4 address is returned as IPv4-mapped IPv6 address. Used for IPv6 (**AF_INET6**) only.

AI_ADDRCONFIG

If this flag is specified, an IPv4 address is returned only if an IPv4 address is configured for the local system, while an IPv6 address is returned only if at least one non-loopback IPv6 address is locally configured.

Example

```
struct addrinfo hints, *res;

bzero ((char *) &hints, sizeof (hints));
hints.ai_flags = AI_V4MAPPED;
hints.ai_family = AF_INET6;
```

```
getaddrinfo ("alpha","ftp",&hints,&res);
\...
freeaddrinfo (res);
```

Behaviour changes

The following flags are not supported:

AI_NUMERICSERV

The following error codes are not currently returned.

EAI_BADHINTS

EAI_BADFLAGS

Returns

The return value is zero if the call is successful. Otherwise, a numeric error code is returned that can be converted into a string with **gai_strerror( )** (see **netdb.h**).

EAI_BADFLAGS

This error code is currently not returned.

EAI_FAIL

A non-recoverable failure in name resolution occurred.

EAI_FAMILY

The specified **ai_family** is not supported.

EAI_MEMORY

A memory allocation failure occurred.

EAI_NONAME

No hostname or servname is provided, or the provided one is not known.

EAI_SERVICE

The provided servname is not supported for **ai_socktype**.

EAI_SOCKTYPE

The value of **ai_socktype** is not supported.

EAI_SYSTEM

A system error was returned in **errno**.

0

Returns zero if **OK**.

Errno

N/A

See also

getaddrinfo

Name

gai_strerror( ) - convert a numeric error code into a string

Layer

IPNET_USRSPACE

Synopsis

```
char * gai_strerror
    (
    int ecode
    )
```

Description

This routine uses **ai_errlist** to convert an error code returned by **getaddrinfo( )** into a string.

Returns

The error string.

Errno

N/A

See also

getaddrinfo, **netdb.h**

Name

freeaddrinfo( ) - free allocated memory for structure addrinfo

Layer

IPNET_USRSPACE

Synopsis

```
void freeaddrinfo
    (
    struct addrinfo * pAi
    )
```

Description

This routine frees the allocated memory for the linked list of addrinfo structures and for the canonical name string. ai should point to the first element of a list of addrinfo structures returned by **getaddrinfo( )**.

Returns

N/A

Errno

Not Available

See also

getaddrinfo

14. GETHOSTBYADDR

Name

gethostbyaddr - address-to-name translation library

Layer

IPNET_USRSPACE

Routines

gethostbyaddr( ) - get the name of a host, given the host address

Description

This library supplies the **gethostbyaddr( )** routine.

To use this feature, include the following component: **INCLUDE_GETNAMEINFO**.

Include files

none

Name

gethostbyaddr( ) - get the name of a host, given the host address

Layer

IPNET_USRSPACE

Synopsis

```
struct hostent * gethostbyaddr
    (
    const char *addr,
    int        len,
    int        type
    )
```

Description

This routine performs a host table lookup and possibly a DNS search to query the given host address, addr. It returns a pointer to a hostent structure containing the corresponding host name (if found), otherwise it returns **NULL**. addr points to a protocol address, either **struct in_addr** or **struct in6_addr** depending on the address family, type, which must be either **AF_INET** or **AF_INET6**. Note that the address is specified in network byte order.

Note

This routine allocates memory for the **hostent** structure. The caller must call **hostentFree( )** to return the allocated memory.

Returns

A pointer to a **struct hostent**, upon success; or **NULL**, if an error occurred.

Errno

Not Available

See also

gethostbyaddr

15. GETHOSTBYNAME

Name

gethostbyname - Hostname-Internet address-Translation library

Layer

IPNET_USRSPACE

Routines

gethostbyname( ) - get the address of a host, given the host name

Description

This library supplies the **gethostbyname( )** routine.

Include files

none

Name

gethostbyname( ) - get the address of a host, given the host name

Layer

IPNET_USRSPACE

Synopsis

```
struct hostent * gethostbyname
    (
    const char *name
    )
```

Description

This routine performs a host table lookup and possibly a DNS search to query the given host name. It returns the internet address (in network byte order) and the alias (if any) for this host name.

Note

This routine allocates memory for the **hostent** structure. The caller must call **hostentFree( )** to free the allocated memory.

Returns

A pointer to a **struct hostent** upon success; or **NULL**, if an error occurred.

Errno

Not Available

See also

gethostbyname

16. GETNAMEINFO

Name

getnameinfo - Hostname-Internet address-Translation library

Layer

IPNET_USRSPACE

Routines

getnameinfo( ) - translates a socket address to a node name and service name

Description

This module provides a backwards compatible routine **getnameinfo( )**, a function that provides for IP node name-address-translation.

Include files

wrapper/wrapperHostLib.h

Name

getnameinfo( ) - translates a socket address to a node name and service name

Layer

IPNET_USRSPACE

Synopsis

```
int getnameinfo
    (
    const struct sockaddr * pSa,       /* socket address to be translated */
    socklen_t               salen,     /* depends on AF_INET(6) */
    char *                  pHost,     /* node name or address string */
    size_t                  hostlen,   /* length of node name buffer */
    char *                  pServ,     /* service name or port number string */
    size_t                  servlen,   /* length of service name buffer */
    int                     flags
    )
```

Description

* This function needs a socket address and provides an IP node name and/or service name. The caller allocates space for host with hostlen (e.g. **NI_MAXHOST** + 1) and serv with servlen (e.g. **NI_MAXSERV** + 1). If the caller doesn't want the host name or service name he must specify a hostlen or servlen of 0. The routine accesses the host table and/or DNS.

The following flags are defined:

NI_NUMERICHOST

The numeric form of the host's address is returned instead of its name, under all circumstances.

NI_NAMEREQ

An error is returned if the host's name can't be located.

NI_NUMERICSERV

The numeric form of the service address is returned instead of its name, under all circumstances.

NI_DGRAM

Indicates that the service is a datagram service (**SOCK_DGRAM**). Default behavior is to assume the service is a stream service (**SOCK_STREAM**).

Example

```
char                hostName[NI_MAXHOST + 1];
char                servName[NI_MAXSERV + 1];
int                 flags;
struct sockaddr_in  sockaddr4;

bzero((char *) &sockaddr4, sizeof(sockaddr4));
sockaddr4.sin_len = 16;
sockaddr4.sin_family = AF_INET;
sockaddr4.sin_port = htons(69);            ->  TFTP
sockaddr4.sin_addr.s_addr = inet_addr("192.168.0.1");
flags = NI_DGRAM;

getnameinfo((sockaddr *) &sockaddr4, sockaddr4.sin_len,
            hostName, NI_MAXHOST, servName, NI_MAXSERV, flags);
```

Behaviuor changes

The followinf flags are not currently supported.

NI_NUMERICSCOPE

Returns

0

Returns zero, if **OK**.

EAI_AGAIN

The name could not be resolved at this time. Future attemts may succeed.

EAI_FAIL

A non-recoverable error occured

EAI_MEMORY

There was a memory allocation error

EAI_NONAME

The name does not resolve for the supplied parameters. **NI_NAMEREQD** is set and the host name cannot be located, or both nodename and servname were null.

EAI_SYSTEM

A system error occured. The error code can be found in errno.

Errno

N/A

See also

getnameinfo

17. GETPROTOBY

Name

getprotoby - getprotobyname and getprotobynumber

Layer

NET_BASE

Routines

getprotobyname( ) - returns protocol information matching name **getprotobynumber( )** - returns protocol information matching numeric value

Description

This module implements the POSIX defined functions **getprotobyname( )** and **getprotobynumber( )**.

Include files

netdb.h

Name

getprotobyname( ) - returns protocol information matching name

Layer

NET_BASE

Synopsis

```
struct protoent * getprotobyname
    (
    const char *name   /**< IP-protocol name */
    )
```

Description

This routine returns the protocol entry matching the passed name.

Returns

pointer to protocol entry, or **NULL**

Errno

N/A

See also

getprotoby

Name

getprotobynumber( ) - returns protocol information matching numeric value

Layer

NET_BASE

Synopsis

```
struct protoent * getprotobynumber
    (
    int proto   /**< IP-protocol number */
    )
```

Description

This routine returns the protocol entry matching the passed numeric protocol number.

Returns

pointer to protocol entry, or **NULL**

Errno

N/A

See also

getprotoby

18. GETSERVBY

Name

getservby - getservbyname and getservbyport

Layer

NET_BASE

Routines

getservbyname( ) - get service description by name **getservbyport( )** - get service description by port

Description

This module implements the POSIX defined functions **getservbyname( )** and **getservbyport( )**.

Include files

netdb.h

Name

getservbyname( ) - get service description by name

Layer

NET_BASE

Synopsis

```
struct servent * getservbyname
    (
    const char * name,   /**< service name */
    const char * proto   /**< transport protocol, or NULL for any protocol */
    )
```

Description

This routine returns a description of a service matching the name and optional transport protocol.

Returns

service description, or **NULL**

Errno

N/A

See also

getservby

Name

getservbyport( ) - get service description by port

Layer

NET_BASE

Synopsis

```
struct servent * getservbyport
    (
    int          port,   /**< service port number in network byte order */
    const char * proto   /**< transport protocol, or NULL for any protocol */
    )
```

Description

This routine returns a description of a service matching the name and optional transport protocol.

Returns

service description, or **NULL**

Errno

N/A

See also

getservby

19. HOSTLIB

Name

hostLib - host table subroutine library

Layer

IPNET_USRSPACE

Routines

hostAdd( ) - add a host to the host table **hostDelete( )** - delete a host from the host table **hostGetByName( )** - look up a host in the host table by its name **hostGetByAddr( )** - look up a host in the host table by its Internet address **hostTblInit( )** - initialize the network host table

Description

This library provides routines to store and access the network host database. The host table contains information regarding the known hosts on the local network. The host table (displayed with **hostShow( )**) contains the Internet address, the official host name, and aliases.

By convention, network addresses are specified in dotted (".") decimal notation. The library **inetLib** contains Internet address manipulation routines. Host names and aliases may contain any printable character.

Before any of the routines in this module can be used, the library must be initialized by **hostTblInit( )**. This is done automatically if **INCLUDE_HOST_TBL** is defined.

Include files

hostLib.h

See also

inetLib, VxWorks 7 Network Stack Programmer's Guide

Name

hostAdd( ) - add a host to the host table

Layer

IPNET_USRSPACE

Synopsis

```
STATUS hostAdd
    (
    char *hostName,    /* host name */
    char *hostAddr     /* host addr in standard Internet format */
    )
```

Description

This routine adds a host name to the local host table. This must be called before sockets on the remote host are opened, or before files on the remote host are accessed via **netDrv** or **nfsDrv**.

The host table has one entry per Internet address. More than one name may be used for an address. Additional host names are added as aliases.

IPv6 link local addresses must be specified with scope delimiter % and corresponding interface id. The same is ignored and set to 0 for other addresses.

Example

```
IPv4 only

   -> hostAdd "wrs", "90.0.0.2"
   -> hostShow
   hostname         inet address       aliases
   --------         ------------       -------
   localhost        127.0.0.1
   yuba             90.0.0.3
   wrs              90.0.0.2
   value = 12288 = 0x3000 = _bzero + 0x18

IPv4/6

   -> hostAdd "wrs", "90.0.0.2"
   -> hostAdd "wrs", "FE80::1%eth0"
   -> hostAdd "wrs", "3FFE::2000:1"
   -> hostShow
   hostname         inet address                     alias
   --------         ------------                     -----
   localhost        127.0.0.1
   localhost        ::1
   yuba             90.0.0.3
   wrs              90.0.0.2
   wrs              FE80::1%eth0
   wrs              3FFE::2000:1
```

Returns

OK, or **ERROR** if the host table is full, the host name/inet address pair is already entered, the Internet address is invalid, or memory is insufficient.

Errno

S_hostLib_INVALID_PARAMETER

hostName or hostAddr are invalid

S_hostLib_HOST_ALREADY_ENTERED

The entry already exists in table

See also

hostLib, **netDrv**, **nfsDrv**

Name

hostDelete( ) - delete a host from the host table

Layer

IPNET_USRSPACE

Synopsis

```
STATUS hostDelete
    (
    char *name,    /* host name or alias */
    char *addr     /* host addr in standard Internet format */
    )
```

Description

This routine deletes a host name from the local host table. If name is a host name, the host entry is deleted. If name is a host name alias, the alias is deleted.

Returns

OK, or **ERROR** if the parameters are invalid or the host is unknown.

Errno

S_hostLib_INVALID_PARAMETER S_hostLib_UNKNOWN_HOST

See also

hostLib

Name

hostGetByName( ) - look up a host in the host table by its name

Layer

IPNET_USRSPACE

Synopsis

```
int hostGetByName
    (
    char *name    /* name of host */
    )
```

Description

This routine returns the Internet address of a host that has been added to the host table by **hostAdd( )**. If the DNS resolver library **resolvLib** has been configured in the vxWorks image, a query for the host IP address is sent to the DNS server, if the name was not found in the local host table. It supports IPv4 only (IPv6 entries are ignored). **getaddrinfo( )** supports both IPv4 and IPv6 and should be used for name resolution.

Returns

The Internet address in network byte order (as an integer), or **ERROR** if the host is unknown.

Errno

S_hostLib_INVALID_PARAMETER S_hostLib_UNKNOWN_HOST

See also

hostLib

Name

hostGetByAddr( ) - look up a host in the host table by its Internet address

Layer

IPNET_USRSPACE

Synopsis

```
STATUS hostGetByAddr
    (
    int  addr,   /* inet address of host */
    char *name   /* buffer to hold name */
    )
```

Description

This routine finds the host name by its Internet address and copies it to name. addr must be in network byte order. The buffer name should be preallocated with (MAXHOSTNAMELEN + 1) bytes of memory and is **NULL**-terminated unless insufficient space is provided. If the buffer name is smaller than (MAXHOSTNAMELEN + 1) then it is possible that it is not large enough for the host name. In this case there might be buffer overflow. Therefore it is very important to allocate sufficient memory as indicated above. If the DNS resolver library **resolvLib** has been configured in the vxWorks image, a query for the host name is sent to the DNS server, if the name was not found in the local host table. It supports IPv4 only (IPv6 entries are ignored). **getnameinfo( )** supports both IPv4 and IPv6 and should be used for address resolution.

Warning

This routine does not look for aliases. Host names are limited to MAXHOSTNAMELEN (from **hostLib.h**) characters.

Returns

OK, or **ERROR** if buffer is invalid or the host is unknown.

Errno

S_hostLib_INVALID_PARAMETER S_hostLib_UNKNOWN_HOST

See also

hostLib, **hostGetByName( )**

Name

hostTblInit( ) - initialize the network host table

Layer

IPNET_USRSPACE

Synopsis

```
STATUS hostTblInit (void)
```

Description

This routine initializes the host list data structure used by routines throughout this module. It should be called before any other routines in this module. This is done automatically if **INCLUDE_HOST_TBL** is defined.

Returns

OK, or **ERROR** if the initialization fails

Errno

Not Available

See also

hostLib, usrConfig

20. HOSTUTILS

Name

hostUtils - Utility routines for host table

Layer

IPNET_USRSPACE

Routines

hostentAlloc( ) - Allocate a hostent structure **hostentFree( )** - Frees the memory associated with a hostent structure

Description

This file provides utility routines for the host to name mapping functions.

Include files

hostLib.h

Name

hostentAlloc( ) - Allocate a hostent structure

Layer

IPNET_USRSPACE

Synopsis

```
struct hostent * hostentAlloc
    (
    void
    )
```

Description

This routine allocates a hostent structure. The memory allocated here must be freed up by a call to **hostentFree( )**. The h_name member of the returned struct hostent points to a buffer of size **HOSTENT_ALLOC_MAXNAME**, while h_addr_list[0] points to a buffer of size **HOSTENT_ALLOC_MAXADDRS**.

Returns

A pointer to a struct hostent, or **NULL** if there was insufficient memory to do the allocations.

Errno

Not Available

See also

hostUtils

Name

hostentFree( ) - Frees the memory associated with a hostent structure

Layer

IPNET_USRSPACE

Synopsis

```
void hostentFree
    (
    struct hostent * hp   /* the hostent struct to be freed */
    )
```

Description

This routine must be called in conjunction with a call to **gethostbyname( )**. It frees up the memory allocated by **hostentAlloc( )**.

Returns

N/A

Errno

Not Available

See also

hostUtils

21. HTTPBASELIB

Name

httpBaseLib - Hyper Text Transfer Protocol module

Layer

NET_BASE

Routines

httpBufLenGet( ) - get length of buffer **httpBufIsEmpty( )** - check if buffer is empty **httpBufAvailGet( )** - get available space in buffer **httpBufFrontGet( )** - get a pointer to HTTP buffer **httpBufBackGet( )** - get pointer to first non used byte in buffer **httpBufLenAdd( )** - add the length of a HTTP buffer **httpCreate( )** - create HTTP session **httpDelete( )** - delete a HTTP session context **httpReqFinish( )** - finish a request **httpResInit( )** - initialize response context **strstrNeverNull( )** - coverity workaround **httpProcess( )** - update state of the HTTP session **httpDoneSet( )** - mark session as done **httpIsSessionActive( )** - check if the HTTP session state is active **httpRdCheck( )** - check if the session may become ready for read **httpWrCheck( )** - check if the session is chocked **httpErrnoGet( )** - get last error **httpSockGet( )** - get socket descriptor **httpUsrCtxGet( )** - get user defined context **httpUsrCtxSet( )** - set user context **httpMethodGet( )** - get request method **httpRequestTargetGet( )** - get request target **httpRequestTargetFormGet( )** - get request target form **httpVersionGet( )** - get HTTP version **httpHeaderFieldGet( )** - get HTTP header field **httpPayloadGet( )** - get request payload **httpIdGet( )** - get HTTP session context **httpVersionSet( )** - set HTTP protocol version **httpStatusCodeSet( )** - set HTTP status code **httpCloseRtnSet( )** - set response close routine **httpFinishRtnSet( )** - set response finish routine **httpHeaderFieldSet( )** - set header field value **httpWrite( )** - write data with chunked encoding **httpEnd( )** - end chunked encoding response **httpRawWrite( )** - write HTTP response

Description

This module implement the HTTP protocol as described in RFC7230. This module is not a webserver, the module implement the details of the Hyper Text Transfer Protocol and keep enough state to receive and send complete HTTP messages. Creation of TCP socket and establish a connection to a peer is out of scope for this module, the socket passed to the **httpCreate( )** routine must already be in connected state.

This module support both blocking and non-blocking sockets. The **httpProcess( )** routine will return **ERROR** and **httpErrnoGet( )** will return **EWOULDBLOCK** if the socket is in non-blocking mode and no more data is available on the socket.

The HTTP request **close( )** routine is called if the TCP session is finished or aborted before all pending data is transmitted.

The HTTP request **finish( )** routine is called after the **httpWrite( )** routine returned **FALSE** and all data queued on the HTTP context is instead queued on the socket. Note that the **httpWrite( )** routine will never return **FALSE** if the socket is in blocking mode.

Example:

Wait for HTTP request, if the request is a GET and the reqeust target is "/", then respond with the plain text string

```
"Hello World! My payload includes a trailing CRLF.\r\n"
```

the TCP session is kept open if allowed by the client.

```
static const char msg[] = "Hello World! My payload includes a trailing CRLF.\r\n";

LOCAL void reqRtn
    (
    const HTTP_REQ req,
```

```
    HTTP_RES res
    )
    {
    if (httpMethodGet (req) == HTTP_M_GET
        && strcmp (httpRequestTargetGet (req), "/") == 0)
        {
        httpHeaderFieldSet (res,
                            HTTP_HF_CONTENT_TYPE,
                            "text/plain;charset=\"utf-8\"");
        httpWriteAndEnd (res, msg, sizeof msg);
        }
    }

   ...

    {
    /*
     * 'sock' must be a connected TCP socket and it is assumed here that the
     * socket is in blocking mode
     */

    HTTP_ID session = httpCreate (sock, reqRtn);

    do
        {
        httpProcess (session);
        }
    while (httpIsSessionActive (session));
    }
```

Include files

httpLib.c

Name

httpBufLenGet( ) - get length of buffer

Layer

NET_BASE

Synopsis

```
_WRS_INLINE size_t httpBufLenGet
    (
    const HTTP_BUF * buf   /**< HTTP buffer */
    )
```

Description

This routine get the number of bytes used in the buffer.

Returns

Length of buffer

Errno

N/A

See also

httpBaseLib

Name

httpBufIsEmpty( ) - check if buffer is empty

Layer

NET_BASE

Synopsis

```
_WRS_INLINE BOOL httpBufIsEmpty
    (
    const HTTP_BUF * buf   /**< HTTP buffer */
    )
```

Description

This routine check if a HTTP buffer is empty

Returns

TRUE if empty, or **FALSE** otherwise

Errno

N/A

See also

httpBaseLib

Name

httpBufAvailGet( ) - get available space in buffer

Layer

NET_BASE

Synopsis

```
_WRS_INLINE size_t httpBufAvailGet
    (
    const HTTP_BUF * buf   /**< HTTP data buffer */
    )
```

Description

This routine get the number of bytes that can be added to the buffer before reaching the current capacity limit.

Returns

Number of available bytes

Errno

N/A

See also

httpBaseLib

Name

httpBufFrontGet( ) - get a pointer to HTTP buffer

Layer

NET_BASE

Synopsis

```
_WRS_INLINE char * httpBufFrontGet
    (
    HTTP_BUF * buf   /**< HTTP data buffer */
    )
```

Description

This routine return a pointer to the front of the HTTP buffer.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpBufBackGet( ) - get pointer to first non used byte in buffer

Layer

NET_BASE

Synopsis

```
_WRS_INLINE char * httpBufBackGet
    (
    HTTP_BUF * buf   /**< HTTP data buffer */
    )
```

Description

This routine get a

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpBufLenAdd( ) - add the length of a HTTP buffer

Layer

NET_BASE

Synopsis

```
_WRS_INLINE void httpBufLenAdd
    (
    HTTP_BUF * buf,   /**< HTTP buffer */
    size_t     cnt    /**< number of bytes that has been added to the end */
                      /* of this buffer */
    )
```

Description

This routine is used in conjunction with **httpBufBackGet( )** and **httpBufAvailGet( )** to allow reading an unkown about of data of size zero to **httpBufAvailGet( )** into the buffer, this routine is used to adjust the length with the actual number of bytes added at the end.

Returns

N/A

Errno

Not Available

See also

httpBaseLib, **httpBufBackGet( )**, **httBufAvailGet( )**

Name

httpCreate( ) - create HTTP session

Layer

NET_BASE

Synopsis

```
HTTP_ID httpCreate
    (
    int          sock,    /**< connected TCP socket */
    HTTP_REQ_RTN reqRtn   /**< HTTP request handler */
    )
```

Description

This routine create a HTTP session context

Returns

A new HTTP context, or **NULL** if not enough memory is available.

Errno

N/A

See also

httpBaseLib

Name

httpDelete( ) - delete a HTTP session context

Layer

NET_BASE

Synopsis

```
void httpDelete
    (
    HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine free all resources allocated by the HTTP session context, however it does not close socket passed to **httpCreate( )**.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpReqFinish( ) - finish a request

Layer

NET_BASE

Synopsis

```
_WRS_INLINE void httpReqFinish
    (
    HTTP_REQ req   /**< HTTP request */
    )
```

Description

This routine do post cleanup after a HTTP request is emitted.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpResInit( ) - initialize response context

Layer

NET_BASE

Synopsis

```
_WRS_INLINE void httpResInit
    (
    HTTP_RES       res,    /**< HTTP response context */
    const HTTP_REQ req     /**< HTTP request context */
    )
```

Description

This routine initialize the response context to using the same HTTP version as the request and with status code set to 200 **OK**.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

strstrNeverNull( ) - coverity workaround

Layer

NET_BASE

Synopsis

```
_WRS_INLINE char * strstrNeverNull
    (
```

```
    char *       haystack,   /**< string to search in */
    const char * needle,     /**< substring to look for */
    char *       fallback    /**< fallback string to return if no match */
    )
```

Description

This routine provides a workaround for Coverity complaining about not checking **strstr( )** for **NULL** return in **httpHeaderFieldsParse( )** where it is impossible for **strstr( )** to every return **NULL** so the unit-tests cannot get to 100 % line coverage.

Returns

pointer to the needle in the haystack

Errno

N/A

See also

httpBaseLib

Name

httpProcess( ) - update state of the HTTP session

Layer

NET_BASE

Synopsis

```
void httpProcess
    (
    HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine ensure that any pending HTTP request is queued for transmission and that any newly received data is parsed and the HTTP request routine is called if a complete request is received.

Returns

OK if all data pending on the socket is received and successfully
process, or **ERROR** if some data could be queued for transmission

Errno

N/A

See also

httpBaseLib

Name

httpDoneSet( ) - mark session as done

Layer

NET_BASE

Synopsis

```
extern void httpDoneSet
    (
    HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine mark a HTTP session as done, which mean that **httpProcess( )** no longer check for new requests.

Returns

N/A

Errno

Not Available

See also

httpBaseLib, **httpProcess( )**

Name

httpIsSessionActive( ) - check if the HTTP session state is active

Layer

NET_BASE

Synopsis

```
BOOL httpIsSessionActive
    (
    const HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine check if the HTTP session is a state where forward progress can be done, that is the socket session is still active and no other persistent errors is pending.

Returns

TRUE if the state is **OK**, or **FALSE** otherwise

Errno

N/A

See also

httpBaseLib

Name

httpRdCheck( ) - check if the session may become ready for read

Layer

NET_BASE

Synopsis

```
extern BOOL httpRdCheck
    (
    const HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine check if the HTTP session is in a state where any user should check the underlying socket for readability.

Returns

TRUE of the underlying socket should be checked for readability, or **FALSE** if no more HTTP requests are expected.

Errno

N/A

See also

httpBaseLib

Name

httpWrCheck( ) - check if the session is chocked

Layer

NET_BASE

Synopsis

```
extern BOOL httpWrCheck
    (
    const HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine check if the transmission buffer of the underlying socket is full, if so then the socket must be checked for write.

Returns

TRUE if the socket must be checked for ready for write, or **FALSE**
if data may still be queued on the socket for transmission.

Errno

N/A

See also

httpBaseLib

Name

httpErrnoGet( ) - get last error

Layer

NET_BASE

Synopsis

```
int httpErrnoGet
    (
    const HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine get the last error encountered in this HTTP session, note that **EWOULDBLOCK** is expected and non-fatal if the socket passed to **httpCreate( )** is in non-blocking mode.

Returns

Last errno.

Errno

N/A

See also

httpBaseLib

Name

httpSockGet( ) - get socket descriptor

Layer

NET_BASE

Synopsis

```
int httpSockGet
    (
    const HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine get the socket descriptor assocated with the HTTP session through the **httpCreate( )** routine.

Returns

socket descriptor

Errno

N/A

See also

httpBaseLib

Name

httpUsrCtxGet( ) - get user defined context

Layer

NET_BASE

Synopsis

```
void * httpUsrCtxGet
    (
```

```
    HTTP_ID session   /**< HTTP session context */
    )
```

Description

This routine get the user defined context data

Returns

N/A

Errno

Not Available

See also

httpBaseLib, **httpUsrCtxSet( )**

Name

httpUsrCtxSet( ) - set user context

Layer

NET_BASE

Synopsis

```
void httpUsrCtxSet
    (
    HTTP_ID session,   /**< HTTP session context */
    void *  usrCtx     /**< pointer to used defined data */
    )
```

Description

This routine let an user store a pointer to private data that can be fetched via the **httpUsrSessionGet( )** routine. This module will not use or interpret the user defined context in any way.

Returns

N/A

Errno

Not Available

See also

httpBaseLib, **httpUsrCtxGet( )**

Name

httpMethodGet( ) - get request method

Layer

NET_BASE

Synopsis

```
HTTP_MTD httpMethodGet
    (
    const HTTP_REQ req   /**< HTTP request context */
    )
```

Description

This routine get the HTTP request method

Returns

HTTP request method

Errno

N/A

See also

httpBaseLib

Name

httpRequestTargetGet( ) - get request target

Layer

NET_BASE

Synopsis

```
const char * httpRequestTargetGet
    (
    const HTTP_REQ req   /**< HTTP request context */
    )
```

Description

This routine get the HTTP request target string

Returns

HTTP request target

Errno

N/A

See also

httpBaseLib

Name

httpRequestTargetFormGet( ) - get request target form

Layer

NET_BASE

Synopsis

```
HTTP_REQ_TGT_FRM httpRequestTargetFormGet
    (
    const HTTP_REQ req   /**< HTTP request context */
    )
```

Description

This routine get the HTTP request target form

Returns

HTTP request target form

Errno

N/A

See also

httpBaseLib

Name

httpVersionGet( ) - get HTTP version

Layer

NET_BASE

Synopsis

```
HTTP_VER httpVersionGet
    (
    const HTTP_REQ req   /**< HTTP request context */
    )
```

Description

This routine get the HTTP version used in the HTTP request.

Returns

HTTP protocol version

Errno

N/A

See also

httpBaseLib

Name

httpHeaderFieldGet( ) - get HTTP header field

Layer

NET_BASE

Synopsis

```
const char * httpHeaderFieldGet
    (
    const HTTP_REQ req,   /**< HTTP request context */
    HTTP_HF        hf     /**< HTTP header field to get */
    )
```

Description

This routine get a specific HTTP header field from the request context

Returns

header field value or **NULL** if the request did not contain the specified header field.

Errno

N/A

See also

httpBaseLib

Name

httpPayloadGet( ) - get request payload

Layer

NET_BASE

Synopsis

```
const void * httpPayloadGet
    (
    const HTTP_REQ req,         /**< HTTP request context */
    size_t *       payloadLen   /**< pointer to where the length of the */
                                /* payload is stored */
    )
```

Description

This routine get the request payload

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpIdGet( ) - get HTTP session context

Layer

NET_BASE

Synopsis

```
HTTP_ID httpIdGet
    (
    HTTP_RES res   /**< HTTP response context */
    )
```

Description

This routine get the HTTP session context associated with the specified HTTP request.

Returns

HTTP context

Errno

N/A

See also

httpBaseLib

Name

httpVersionSet( ) - set HTTP protocol version

Layer

NET_BASE

Synopsis

```
void httpVersionSet
    (
    HTTP_RES         res,    /**< HTTP response contet */
    const HTTP_VER * ver     /**< HTTP protocol version to set */
    )
```

Description

This routine set the HTTP protocol version to be used in the HTTP response.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpStatusCodeSet( ) - set HTTP status code

Layer

NET_BASE

Synopsis

```
void httpStatusCodeSet
    (
    HTTP_RES res,   /**< HTTP response context */
    HTTP_SC  sc     /**< HTTP status code */
    )
```

Description

This routine set the HTTP status code to use in the HTTP status line

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpCloseRtnSet( ) - set response close routine

Layer

NET_BASE

Synopsis

```
void httpCloseRtnSet
    (
    HTTP_RES       res,       /**< HTTP response context */
    HTTP_CLOSE_RTN closeRtn   /**< user-defined close routine */
    )
```

Description

This routine set a callback routine that is run if the underlying socket is closed or fail before the whole HTTP response message is queued to the underlying socket send buffer.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpFinishRtnSet( ) - set response finish routine

Layer

NET_BASE

Synopsis

```
void httpFinishRtnSet
    (
    HTTP_RES        res,        /**< HTTP response context */
    HTTP_FINISH_RTN finishRtn   /**< user-defined finish routine */
    )
```

Description

This routine set a callback routine that is run if/when the whole HTTP response message is queued on the underlying socket send buffer. Note that this does not guarantee that the peer actually received the response at the time when routine is run.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpHeaderFieldSet( ) - set header field value

Layer

NET_BASE

Synopsis

```
void httpHeaderFieldSet
    (
    HTTP_RES     res,   /**< HTTP response context */
    HTTP_HF      hf,    /**< HTTP header field ID */
    const char * val    /**< HTTP header field value */
    )
```

Description

This routine set a header field value on the HTTP response context.

Returns

N/A

Errno

N/A

See also

httpBaseLib

Name

httpWrite( ) - write data with chunked encoding

Layer

NET_BASE

Synopsis

```
BOOL httpWrite
    (
    HTTP_RES     res,      /**< HTTP response context */
    const void * data,     /**< data to write */
    size_t       dataLen   /**< length of data */
    )
```

Description

This routine write a HTTP response using chunked encoding, the response is not completed until the **httpEnd( )** is called.

Returns

TRUE if all response data is flushed to underlying socket, or **FALSE** of at least some of the data is buffered.

Errno

Not Available

See also

httpBaseLib, **httpEnd( )**

Name

httpEnd( ) - end chunked encoding response

Layer

NET_BASE

Synopsis

```
BOOL httpEnd
    (
    HTTP_RES res   /**< HTTP response context */
    )
```

Description

This routine end a HTTP response that uses chunked encoding

Returns

N/A

Errno

Not Available

See also

httpBaseLib, **httpWrite( )**

Name

httpRawWrite( ) - write HTTP response

Layer

NET_BASE

Synopsis

```
BOOL httpWriteAndEnd
    (
    HTTP_RES     res,      /**< HTTP response context */
    const void * data,     /**< data to write */
    size_t       dataLen   /**< length of data */
    )
```

Description

This routine write a HTTP response with Content-Length, i.e. in a single message without chunked encoding.

Returns

TRUE if all response data is flushed to underlying socket, or **FALSE** of at least some of the data is buffered.

Errno

N/A

See also

httpBaseLib

22. ICMPLIB

Name

icmpLib - internet control message protocol library

Layer

RTNET

Routines

icmpTx( ) - transmit ICMP packet **icmpEchoRx( )** - handle echo request **icmpDstUnreachTx( )** - transmit ICMP destination unreachable message **icmpLibInit( )** - intialize ICMP library

Description

Implementation of the IPv4 Internet control message protocol as described by the RFC listed below.

This library is initialized, which include register a handler for **IPPROTO_ICMP** with **ip4Lib**, by the **icmpLibInit( )** routine. The icmpLibInit

Libraries that need notification about received ICMP messages must add a source to the **NS_NCT_ICMP** chain in **netnsLib**. The context information passed to the chain action handler is passed an **ICMP_CTX** instance with information about the ICMP message.

/ts RFC | Title -----|----- 0792 | Internet control message protocol 6633 | Deprecation of ICMP Source Quench Messages 6918 | Formally Deprecating Some ICMPv4 Message Types /te

Include files

icmpLib.h

See also

ip4Lib, **netnsLib**, VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

icmpTx( ) - transmit ICMP packet

Layer

RTNET

Synopsis

```
void icmpTx
    (
    PKT_CTX * pkt   /**< ICMP packet */
    )
```

Description

This routine calculates the ICMP checksum and deliver the packet to the IPv4 layer for transmission.

Returns

N/A

Errno

N/A

See also

icmpLib

Name

icmpEchoRx( ) - handle echo request

Layer

RTNET

Synopsis

```
void icmpEchoRx
    (
    PKT_CTX *        pkt,    /**< ICMP packet */
    const ICMP_HDR * ih      /**< ICMP header */
    )
```

Description

This routine transmit a echo reply matching the echo request

Returns

N/A

Errno

N/A

See also

icmpLib

Name

icmpDstUnreachTx( ) - transmit ICMP destination unreachable message

Layer

RTNET

Synopsis

```
void icmpDstUnreachTx
    (
    PKT_CTX * trigger,   /**< IP-datagram that trigger this ICMP */
    UINT8     code       /**< destination unreachable message code */
    )
```

Description

This routine tramsit an ICMP destination unreachable message, including part of the IP-datagram that triggered transmission the ICMP message.

Returns

N/A

Errno

N/A

See also

icmpLib

Name

icmpLibInit( ) - intialize ICMP library

Layer

RTNET

Synopsis

```
STATUS icmpLibInit
    (
    NETNS * ns   /**< network namespace */
    )
```

Description

This routine initialize the ICMP library by registering a handler for the ICMP protocol with **ip4Lib**

Returns

Always **OK**

Errno

N/A

See also

icmpLib

23. IFNAMELIB

Name

ifNameLib - network interface name and index library

Layer

NET_BASE

Routines

if_indextoname( ) - get network interface name from index **if_nametoindex( )** - get network interface index from name

Description

This library implements the routines **if_nametoindex( )**, **if_indextoname**, **if_nameindex( )** and **if_freenameindex( )**. All four functions are defined by POSIX.1-2008.

Include files

net/if.h

Name

if_indextoname( ) - get network interface name from index

Layer

NET_BASE

Synopsis

```
char * if_indextoname
    (
    unsigned ifindex,   /**< network interface index to map */
    char *   ifname     /**< buffer of at least size IFNAMSIZ */
    )
```

Description

This routine tries to map a network interface index to the name given to that interface. ifname must point to a buffer of at least IFNAMSIZ bytes. If there is a network interface with index ifindex, the interface name string is copied starting at ifname. In this case, the return value is ifname; otherwise, the routine fails and the return value is **NULL**.

Returns

Pointer to the name, or **NULL** otherwise.

Errno

ENXIO if an interface with the specified index does not exist.

See also

ifNameLib

Name

if_nametoindex( ) - get network interface index from name

Layer

NET_BASE

Synopsis

```
unsigned if_nametoindex
    (
    const char * ifname
    )
```

Description

This routine tries to find a network interface with the specified name, the network interface index is returned if a match is found.

Returns

network interface index, or 0 if the operation is unsuccessful.

Errno

ENXIO if an interface with the specified name does not exist.

See also

ifNameLib

24. INCSUMLIB

Name

incsumLib - Internet checksum library

Layer

RTNET

Routines

incsumCompute( ) - compute Internet checksum **incsumFold( )** - fold a partial Internet checksum

Description

This library implements generation/verification of the Internet checksum as described by RFC1071. Generation of the checksum is split into two parts, one is generating an intermediate checksum from the content of some buffer. This is done via the **incsumCompute( )** routine, result from multiple buffers can be concatenated by addition of the individual intermediate checksums.

The Internet checksum is generated by passing aggregate intermediate checksum to the **incsumFold( )** routine. This routine always return zero when used for verification and the checksum is correct.

/ts RFC | Title -----|----- 1071 | Computing the Internet Checksum /te

Include files

incsumLib.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

incsumCompute( ) - compute Internet checksum

Layer

RTNET

Synopsis

```
UINT32 incsumCompute
    (
    const void * ptr,   /**< buffer to compute a checksum on */
    UINT32       len    /**< length of buffer */
    )
```

Description

This routine compute the Internet checksum for a specified number of bytes of a buffer.

Returns

Unfolded Internet checksum

Errno

Not Available

See also

incsumLib, **incsumFold( )**

Name

incsumFold( ) - fold a partial Internet checksum

Layer

RTNET

Synopsis

```
UINT16BE incsumFold
    (
    UINT32 sum
    )
```

Description

This routine finish the Internet checksum calculation by folding the partial checksum.

Returns

Internet checksum

Errno

Not Available

See also

incsumLib, **incsumCompute( )**

25. INETLIB

Name

inetLib - Internet address manipulation routines

Layer

IPNET_USRSPACE

Routines

inet_lnaof( ) - get the local address (host number) from the Internet address **inet_makeaddr_b( )** - form an Internet address from network and host numbers **inet_makeaddr( )** - form an Internet address from network and host numbers **inet_netof( )** - return the network number from an Internet address **inet_netof_string( )** - extract the network address in dot notation **inet_network( )** - convert an Internet network number from string to address **inet_ntoa_b( )** - convert an network address to dot notation, store it in a buffer

Description

This library provides routines for manipulating Internet addresses, including the UNIX BSD 4.3 **inet_** routines. It includes routines for converting between character addresses in Internet standard dotted decimal notation and integer addresses, routines for extracting the network and host portions out of an Internet address, and routines for constructing Internet addresses given the network and host address parts.

All Internet addresses are returned in network order (bytes ordered from left to right). All network numbers and local address parts are returned as machine format integer values.

Internet addresses

Internet addresses are typically specified in dotted decimal notation or as a 4-byte number. Values specified using the dotted decimal notation take one of the following forms:

```
a.b.c.d
a.b.c
a.b
a
```

If four parts are specified, each is interpreted as a byte of data and assigned, from left to right, to the four bytes of an Internet address. Note that when an Internet address is viewed as a 32-bit integer quantity on any MC68000 family machine, the bytes referred to above appear as "a.b.c.d" and are ordered from left to right.

If a three-part address is specified, the last part is interpreted as a 16-bit quantity and placed in the right-most two bytes of the network address. This makes the three-part address format convenient for specifying Class B network addresses as "128.net.host".

If a two-part address is supplied, the last part is interpreted as a 24-bit quantity and placed in the right-most three bytes of the network address. This makes the two-part address format convenient for specifying Class A network addresses as "net.host".

If only one part is given, the value is stored directly in the network address without any byte rearrangement.

Although dotted decimal notation is the default, it is possible to use the dot notation with hexadecimal or octal numbers. The base is indicated using the same prefixes as are used in C. That is, a leading 0x or 0X indicates a hexadecimal number. A leading 0 indicates an octal number. If there is no prefix, the number is interpreted as decimal.

To use this feature, include the following component: **INCLUDE_INETLIB**

Include files

inetLib.h, **inet.h**

See also

UNIX BSD 4.3 manual entry for inet(3N), "VxWorks 7 Network Stack Programmer's Guide"

Name

inet_lnaof( ) - get the local address (host number) from the Internet address

Layer

IPNET_USRSPACE

Synopsis

```
int inet_lnaof
    (
    int inetAddress   /* inet addr from which to extract local portion */
    )
```

Description

This routine returns the local network address portion of an Internet address. The routine handles class A, B, and C network number formats. inetAddress must be in network order.

Example

The following example returns 2:

```
    int retValue;
    retValue = inet_lnaof (inet_addr("90.0.0.2"));
```

Returns

The local address portion of inetAddress.

Errno

Not Available

See also

inetLib

Name

inet_makeaddr_b( ) - form an Internet address from network and host numbers

Layer

IPNET_USRSPACE

Synopsis

```
void inet_makeaddr_b
    (
    int            netAddr,     /* network part of the inet address */
    int            hostAddr,    /* host part of the inet address */
    struct in_addr *pInetAddr   /* where to return the inet address */
    )
```

Description

This routine constructs the Internet address from the network number and local host address. This routine is identical to the UNIX **inet_makeaddr( )** routine except that you must provide a buffer for the resulting value.

Example

The following copies the address 0x5a000002 to the location pointed to by pInetAddr:

```
    inet_makeaddr_b (0x5a, 2, pInetAddr);
```

Returns

N/A

Errno

Not Available

See also

inetLib

Name

inet_makeaddr( ) - form an Internet address from network and host numbers

Layer

IPNET_USRSPACE

Synopsis

```
struct in_addr inet_makeaddr
    (
    int netAddr,   /* network part of the address */
    int hostAddr   /* host part of the address */
    )
```

Description

This routine constructs the Internet address from the network number and local host address.

This routine is supplied for UNIX compatibility only.

Example

The following example returns the address 0x5a000002 to the structure **in_addr**:

```
    inet_makeaddr (0x5a, 2);
```

Returns

The network address in an **in_addr** structure.

Errno

Not Available

See also

inetLib, **inet_makeaddr_b( )**

Name

inet_netof( ) - return the network number from an Internet address

Layer

IPNET_USRSPACE

Synopsis

```
int inet_netof
    (
    struct in_addr inetAddress   /* inet address */
    )
```

Description

This routine extracts the network portion of an Internet address. IP address must be in network order.

Example

The following example returns 0x5a:

```
    struct in_addr inetAddress;

    inetAddress.s_addr = inet_addr ("90.0.0.2");
    inet_netof (inetAddress);
```

Returns

The network portion of inetAddress.

Errno

Not Available

See also

inetLib

Name

inet_netof_string( ) - extract the network address in dot notation

Layer

IPNET_USRSPACE

Synopsis

```
void inet_netof_string
    (
    char *inetString,   /* inet addr to extract local portion from */
    char *netString     /* net inet address to return */
    )
```

Description

This routine extracts the network Internet address from a host Internet address (specified in dotted decimal notation). The routine handles class A, B, and C network addresses. The buffer netString should be **INET_ADDR_LEN** bytes long.

Note

This is the only routine in **inetLib** that considers configured subnet masks. The IP address provided is used to calculate a classful network mask, and the corresponding (classful) network address. The interface address list is searched for an address matching that classful network address. If a matching interface IP address is found, the associated subnet mask from the (first) match is used with the original IP address to obtain the subnet address returned as a string in netString. If no match is found, the string form of the classful network address is returned in netString.

This process is unreliable if there are two configured interface addresses belonging to different subnets with different subnet masks within the same classful network.

Example

The following example copies "90.0.0.0" to netString:

```
    char netString [INET_ADDR_LEN];
    ...
    inet_netof_string ("90.0.0.2", netString);
```

Returns

N/A

Errno

S_inetLib_ILLEGAL_INTERNET_ADDRESS

See also

inetLib

Name

inet_network( ) - convert an Internet network number from string to address

Layer

IPNET_USRSPACE

Synopsis

```
u_long inet_network
    (
    register char *inetString   /* string version of inet addr */
    )
```

Description

This routine forms a network address from an ASCII string containing an Internet network number.

Example

The following example returns 0x5a:

```
    inet_network ("90");
```

Returns

The Internet address for an ASCII string, or (u_long)**ERROR** if invalid.

Errno

S_inetLib_ILLEGAL_NETWORK_NUMBER

See also

inetLib

Name

inet_ntoa_b( ) - convert an network address to dot notation, store it in a buffer

Layer

IPNET_USRSPACE

Synopsis

```
void inet_ntoa_b
    (
    struct in_addr inetAddress,   /* inet address */
    char           *pString       /* where to return ASCII string */
    )
```

Description

This routine converts an Internet address in network format to dotted decimal notation.

This routine is identical to the UNIX **inet_ntoa( )** routine except that you must provide a buffer of size **INET_ADDR_LEN**.

Example

The following example copies the string "90.0.0.2" to pString:

```
    struct in_addr iaddr;
     ...
    iaddr.s_addr = htonl(0x5a000002);
     ...
    inet_ntoa_b (iaddr, pString);
```

Returns

N/A

Errno

Not Available

See also

inetLib

26. IP4LIB

Name

ip4Lib - Internet protocol version 4 library

Layer

RTNET

Routines

ip4IsFragment( ) - check if IP-datagram is a fragment **ip4TransProtoRxSet( )** - set a transport protocol reception handler **ip4LibInit( )** - initialize the IPv4 library **rtnetStackIp4Ctor( )** - constructor for IPv4 network stack **rtnetStackIp4Enable( )** - link safe network IPv4 stack into an image or RTP

Description

Implementation of the Internet protocol version 4 for the VxWorks RTNET stack. This network stack is mainly a end-host network stack.

This library will register protocol handler for IPv4 with the link layer libraries. The handlers are registed via the **ip4LibInit( )** function.

Network layer protocols using IPv4 should call **ip4Tx( )** to add an IPv4 header and perform proper protocol processing to egress packets.

This library implements the relevant parts of the following RFCs /ts RFC | Title -----|------ 0791 | Internet Protocol 1122 | Requirements for Internet Hosts -- Communication Layers 1191 | Path MTU Discovery 6864 | Updated Specification of the IPv4 ID Field /te

Include files

ip4Lib.h

Name

ip4IsFragment( ) - check if IP-datagram is a fragment

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL ip4IsFragment
    (
    const IP4_HDR * hdr   /**< IPv4 header */
    )
```

Description

This routine check if an IP-datagram is a fragment of a larger datagram.

Returns

TRUE if it is a fragment, or **FALSE** otherwise

Errno

N/A

See also

ip4Lib

Name

ip4TransProtoRxSet( ) - set a transport protocol reception handler

Layer

RTNET

Synopsis

```
void ip4TransProtoRxSet
    (
    UINT8      ipProto,    /**< IP-protocol number */
    PKT_RX_RTN rxRtn       /**< Protocol handler */
    )
```

Description

This routine set a transport protocol reception handler for a the specified protocol number.

Returns

N/A

Errno

Not Available

See also

ip4Lib

Name

ip4LibInit( ) - initialize the IPv4 library

Layer

RTNET

Synopsis

```
STATUS ip4LibInit(void)
```

Description

This routine initialize the Internet protocol version 4 library

Returns

OK if successful, **ERROR** otherwise

Errno

ENOMEM

Not enough memory to register the IPv4 reception handler in the link layer

See also

ip4Lib

Name

rtnetStackIp4Ctor( ) - constructor for IPv4 network stack

Layer

RTNET

Synopsis

```
_WRS_CONSTRUCTOR
    (
    rtnetStackIp4Ctor,
    92
    )
```

Description

This routine initialize the user-space IPv4 network stack

Returns

N/A

Errno

N/A

See also

ip4Lib

Name

rtnetStackIp4Enable( ) - link safe network IPv4 stack into an image or RTP

Layer

RTNET

Synopsis

```
void rtnetStackIp4Enable (void)
```

Description

This routine does nothing. It may be called to link the core components of the safe network IPv4 stack into an RTP.

Returns

N/A

Errno

N/A

See also

ip4Lib

27. IP4SOCKOPTLIB

Name

ip4SockOptLib - ipv4 socket option library

Layer

RTNET

Routines

ip4SockOptLibInit( ) - initialize IPv4 sock option library

Description

Implementation of **IPPROTO_IP** level socket options for the VxWorks RTNET stack.

Include files

ip4SockOptLib.h

Name

ip4SockOptLibInit( ) - initialize IPv4 sock option library

Layer

RTNET

Synopsis

```
STATUS ip4SockOptLibInit(void)
```

Description

This routine will initialize IPv4 sock option library

Returns

always **OK**

Errno

N/A

See also

ip4SockOptLib

28. IP6LIB

Name

ip6Lib - Internet protocol version 6 library

Layer

RTNET

Routines

ip6ChecksumSet( ) - Set upper layer checksum for an ipv6 packet **ip6LibInit( )** - initialize the IPv6 library **rtnetStackIp6Ctor( )** - constructor for IPv6 network stack **rtnetStackIp6Enable( )** - link safe network IPv6 stack into an image or RTP

Description

Implementation of the Internet protocol version 6 for the VxWorks RTNET stack.

This library will register protocol handler for IPv6 with the link layer libraries. The handlers are registed via the **ip4LibInit( )** function.

Network layer protocols using IPv6 should call **ip6Tx( )** to add an IPv6 header and perform proper protocol processing to egress packets.

Include files

ip6Lib.h

Name

ip6ChecksumSet( ) - Set upper layer checksum for an ipv6 packet

Layer

RTNET

Synopsis

```
void ip6ChecksumSet
    (
    PKT_CTX *               pkt,       /**< ipv6 socket */
    UINT16                  ipProto,   /**< upper layer protocol */
    const struct in6_addr * src,       /**< ip6 src address */
    const struct in6_addr * dst        /**< ip6 dst address */
    )
```

Description

This routine set upper layer checksum for an ipv6 packet

Returns

N/A.

Errno

N/A

See also

ip6Lib

Name

ip6LibInit( ) - initialize the IPv6 library

Layer

RTNET

Synopsis

```
STATUS ip6LibInit (void)
```

Description

This routine initialize the Internet protocol version 6 library

Returns

OK if successful, **ERROR** otherwise

Errno

ENOMEM

Not enough memory to register the IPv6 reception handler in the link layer

See also

ip6Lib

Name

rtnetStackIp6Ctor( ) - constructor for IPv6 network stack

Layer

RTNET

Synopsis

```
_WRS_CONSTRUCTOR
    (
    rtnetStackIp6Ctor,
    92
    )
```

Description

This routine initialize the user-space IPv6 network stack

Returns

N/A

Errno

N/A

See also

ip6Lib

Name

rtnetStackIp6Enable( ) - link safe network IPv6 stack into an image or RTP

Layer

RTNET

Synopsis

```
void rtnetStackIp6Enable (void)
```

Description

This routine does nothing. It may be called to link the core components of the safe network IPv6 stack into an RTP.

Returns

N/A

Errno

N/A

See also

ip6Lib

29. IP6SOCKOPTLIB

Name

ip6SockOptLib - ipv6 socket option library

Layer

RTNET

Routines

ip6SockOptLibInit( ) - initialize IPv6 sock option library

Description

Implementation of **IPPROTO_IPV6** level socket options for the VxWorks RTNET stack.

Include files

ip6SockOptLib.h

Name

ip6SockOptLibInit( ) - initialize IPv6 sock option library

Layer

RTNET

Synopsis

```
STATUS ip6SockOptLibInit(void)
```

Description

This routine will initialize IPv6 sock option library

Returns

always **OK**

Errno

N/A

See also

ip6SockOptLib

30. IPSHOW

Name

ipShow - C-interpreter IP-address show commands

Layer

RTNET

Routines

ipShowWithFilter( ) - show all link/IP-addresses matching the filter **linkShow( )** - show information about all attached network devices **ip4Show( )** - show all IPv4-addresses **ip6Show( )** - show all IPv6-addresses

Description

This module contain a routine, **linkShow( )**, that print information about all attached network devices, the operational state of the network devices and transmission/reception statistics.

This module also contain routines, **ip4Show( )** and **ip6Show( )**, that print the IP-addresses assigned to the network devices attached to the system.

Include files

N/A

Name

ipShowWithFilter( ) - show all link/IP-addresses matching the filter

Layer

RTNET

Synopsis

```
INT32 ipShowWithFilter
    (
    IP_SHOW_CTX * ctx   /**< counter and filter */
    )
```

Description

This routine show all link/IP-addresses matching the filter.

Returns

Number of links/IP-addresses printed.

Errno

N/A

See also

ipShow

Name

linkShow( ) - show information about all attached network devices

Layer

RTNET

Synopsis

```
INT32 linkShow (void)
```

Description

This routine show information about all attached network devices.

Returns

Number of network devices printed.

Errno

N/A

See also

ipShow

Name

ip4Show( ) - show all IPv4-addresses

Layer

RTNET

Synopsis

```
INT32 ip4Show (void)
```

Description

This routine show all IPv4-addresses assigned on all network interfaces.

Returns

Number of IPv4-addresses printed, or -1 if operation failed.

Errno

N/A

See also

ipShow

Name

ip6Show( ) - show all IPv6-addresses

Layer

RTNET

Synopsis

```
INT32 ip6Show (void)
```

Description

This routine show all IPv6-addresses assigned on all network interfaces.

Returns

Number of IPv6-addresses printed, or -1 if operation failed.

Errno

N/A

See also

ipShow

31. JSONNETCFGLIB

Name

jsonNetCfgLib - network stack JSON-configuration library file

Layer

RTNET

Routines

jsonNetCfgIsCompact( ) - check if JSON-tree is in compact format **jsonNetCfgTopicGet( )** - get network config for topic **jsonNetCfgFileGet( )** - get configuration from file **jsonNetCfgGet( )** - get network configuration

Description

This module get networks stack configuration data from file and/or from environment variables. The configuration must be structured as JSON-text with the following formats.

Compact format only allow setting a single gateway, IP-address and prefix length similar to what is passed from the boot monitor to the kernel network stack. All keys are optional, the keys listed are the one current used by the this library. It is only possible to configure a single network interface with this format, that is the second interface to be attached (the first is always loopback).

If **devname** is specified, then that device class, or device instance if unit number is also specified, will be used for the network interface. This parameter is not required if the RTP only got access to a single network device.

```
    {
        "device": {
            "name": "<driver name>",
            "unit"  : <device unit number as integer>,
            "mux" : "<mux name>"
            "id" : "<VxBus device identifier>"
            "mac" : "<device MAC address>"
        },
        "inet": {
            "address"  : "<dotted-quad>",
            "prefixlen": <prefix length as an integer 1-32>,
            "gateway"  : "<dotted-quad>"
        },
        "inet6": {
            "address"  : "<colon delmitered IPv6>",
            "prefixlen": <prefix length as an integer 1-128>,
            "gateway"  : "<colon delmitered IPv6>"
        }
    }
```

The normal format allow setting all the network stack parameters, it also allow configuration of more than one network interface.

```
    {
        "task": {
            "priority": <priority 1-255 for the network task>,
```

```
            "options" : <VxWorks task options as integer>,
            "stacksz" : <network task stack size in bytes>
        },
        "fib": {
            "capacity": <maximum number of FIB entries as integer>
        },
        "peerinfo": {
            "capacity": <maximum number of peers as integer>,
            "lifetime": <lifetime in seconds for peer entries>
        },
        "reassembly": {
            "capacity": <maximum number of fragment chains as integer>,
            "lifetime": <duration in seconds before a partial IP-datagram is discarded>
        },
        "protocol": {
            "portlo": <lowest ephemeral port, integer between 1-65535>,
            "porthi": <highest ephemeral port, integer between 1-65535>,
            "inet"  : <true|false, whether IPv4 is enabled or not>
            "inet6" : <true|false, whether IPv6 is enabled or not>
            "tcp"   : <true|false, whether TCP is enabled or not>
            "udp"   : <true|false, whether UDP is enabled or not>
        },
        "netifs": [
            <one or more JSON-object in compact format described above>
        ]
    }
```

Include files

jsonNetCfgLib.h

Name

jsonNetCfgIsCompact( ) - check if JSON-tree is in compact format

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL jsonNetCfgIsCompact
    (
    json_t * root   /**< root of parsed JSON-tree */
    )
```

Description

This routine check if the JSON-tree is in compact format or in complete format.

Returns

TRUE if in compact format, or **FALSE** otherwise

Errno

N/A

See also

jsonNetCfgLib

Name

jsonNetCfgTopicGet( ) - get network config for topic

Description

This routine get network configuration for a specific topic

Returns

RET_DESC

Errno

EINVAL

The JSON token for the topic is not of type "object"

See also

jsonNetCfgLib

Name

jsonNetCfgFileGet( ) - get configuration from file

Layer

RTNET

Synopsis

```
STATUS jsonNetCfgFileGet
    (
    RTNET_STACK_CFG * cfg,       /**< network configuration */
    const char *      fileName   /**< configuration file name */
    )
```

Description

This routine get the network stack configuration from a file

Returns

OK if successful, or **ERROR** otherwise.

Errno

same as **jsonNetCfgGet( )**

See also

jsonNetCfgLib

Name

jsonNetCfgGet( ) - get network configuration

Layer

RTNET

Synopsis

```
STATUS jsonNetCfgGet
    (
    RTNET_STACK_CFG * cfg   /**< network configuration */
    )
```

Description

This routine get network configuration defined as a JSON-text according to the format described above.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOENT

The no file with the file name exist.

EINVAL

The JSON-text is not well formed.

ENOMEM

Out-of-memory

E2BIG

Too many elements (more than **JSON_NET_CFG_TOKENS_CNT**) in the JSON file.

See also

jsonNetCfgLib

32. NEIGHLIB

Name

neighLib - neighbor cache library file

Layer

RTNET

Routines

binarySearch( ) - binary search for neighbor **neighDbCreate( )** - create a neighbor database **neighDbInit( )** - initialize a neighbor database **neighTx( )** - transmit a packet **neighUpdate( )** - update the state of a neighbor node **neighForEach( )** - call an user-defined function for each neighbor node **neighNetifGet( )** - get network interface **neighFlush( )** - clear all neighbor entries **neighStateUpdateRegister( )** - register neighbor state handler **neighNaRx( )** - neighbor advertisement

Description

The neighbor cache is a data-structure that hold a mapping from IP-address to link-layer address of IP-nodes that is directly reachable from the local IP-node. It is a cache as only the most recently used IP-address tend to be present in the neighbor cache, neighbor nodes may be evicted at any time.

The IPv4/IPv6 layers use the neighbor library when they need to transmit an IP-datagram to the next-hop node returned by a forward information base lookup or specified explicitly by an application. The neighbor library add the link-layer header and deliver the packet to the driver layer if a valid mapping exist, or request a mapping to be created otherwise.

Neighbor state machine

The neighbor library does not implement protocols like address resolution protocol, ARP, or neighbor discovery protocol, NDP. The neighbor library implement the state-machine as described by RFC4861, the library contain hooks that allow specific protocol handlers to perform request for IP-address to link-layer mapping.

The possible state a neighbor node can have are

#unused#

This is not an actual state, it just represent a non-existing instance for a specific IP-address in the picture below

INCOMPLETE

Address resolution is in progress and the link-layer address of the neighbor has not yet been determined.

REACHABLE

Roughly speaking, the neighbor is known to have been reachable recently (within tens of seconds ago).

STALE

The neighbor is no longer known to be reachable but until traffic is sent to the neighbor, no attempt should be made to verify its reachability.

DELAY

The neighbor is no longer known to be reachable, and traffic has recently been sent to the neighbor. Rather than probe the neighbor immediately, however, delay sending probes for a short while in order to give upper-layer protocols a chance to provide reachability confirmation. OAOA\i PROBE The neighbor is no longer known to be reachable, and unicast Neighbor Solicitation probes are being sent to verify reachability.

The possible transitions between the state are visualized in this picture

```
                    +------------+
            +------->  #unused#  |
            |       +-----+------+
            |             |
            |          pkt tx
            |             |
        probe fail  +-----v------+
            +------<| INCOMPLETE |
            |       +-----+------+
            |             |
            |        probe reply
            |             |
            |       +-----v------+
            |       | REACHABLE  <-------+
            |       +-----+------+       |
            |             |              |
            |          timeout           |
            |             |              |
            |       +-----v------+       |
            |       |   STALE    |       |
            |       +-----+------+       |
            |             |              |
            |           pkt tx           |
            |             |              |
            |       +-----v------+ probe request
            |       |   DELAY    |>------+
            |       +-----+------+       |
            |             |              |
            |          timeout           |
            |             |              |
        probe fail  +-----v------+  probe reply
            +------<|   PROBE    |>------+
                    +------------+
```

Integration with arp/ndp

The **neighDbInit( )** routine take pointers to two functions, the [probeRtn] and [updateRtn] functions must be implemented by protocol handlers like ARP/NDP. [probeRtn] is called when an IP-address to link-layer address request should be transmitted, the neighbor library provide the IP-address of the next-hop, the IP-source address of the packet that need the neighbor information and a hint whether the the request should be unicast or multicast.

The resolution protocol handler should call **neighUpdate( )** for each request- and response-packet received, the neighbor library use that information to update the state of the corresponding neighbor node.

The neighbor library will call [updateRtn] as part of updating a neighbor node in **neighUpdate( )**, [updateRtn] must store the link-layer header that will be added to a packet matching this neighbor node. As an example, ARP for Ethernet would store the full Ethernet header in [updateRtn].

Eviction policy

There are two cases where an existing neighbor node may be reused as there is no unused neighbor node instances available. The first case is when **neighTx( )** is called with a next-hop IP-node not present in the cache, the eviction policy is to reuse the oldest neighbor node not in state INCOMPLETE. Nodes in INCOMPLETE state is not reused as it could result in not a single data packet being transmitted if the local node constantly tries to send packets to more next-hop nodes than the capacity of the neighbor cache.

The other case is when **neighUpdate( )** is called for a next-hop node not present in the cache. The oldest node, regardless of state, is reused. The rationale is that **neighUpdate( )** will move the entry to a state where packets might be transmitted without additional communication.

Pending ip-datagram

RFC4861 require that at least one packet must be queued while waiting for IP-address to link-layer resolution and that the newer packets replace older packets if there is a fixed capacity of the queue. This neighbor library will store exactly one packet and keep the newest.

This library implements state machine for neighbor discovery described in the following RFC /ts RFC | Title -----|------ 4861 | Neighbor Discovery for IP version 6 (IPv6) /te

Include files

neighLib.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

binarySearch( ) - binary search for neighbor

Description

This routine use binary search in the neighbor database to locate a specific neighbor node.

Returns

Pointer to matching neighbor node, or **NULL** if no match exist.

Errno

Not Available

See also

neighLib

Name

neighDbCreate( ) - create a neighbor database

Layer

RTNET

Synopsis

```
NEIGH_DB neighDbCreate
    (
```

```
    TMO_DB tmoDb   /**< Timeout database */
    )
```

Description

This routine creates a new neighbor database and reserve timeout handles for each neighbor node.

Returns

A new neighbor database, or **NULL** if not enough memory.

Errno

Not Available

See also

neighLib

Name

neighDbInit( ) - initialize a neighbor database

Layer

RTNET

Synopsis

```
STATUS neighDbInit
    (
    NEIGH_DB         db,          /**< neighbor database */
    NETIF            netif,       /**< network interface to associate with this */
                                  /* database */
    NEIGH_PROBE_RTN  probeRtn,    /**< probe function */
    NEIGH_UPDATE_RTN updateRtn,   /**< update function */
    UINT32           nodeCtxLen   /**< length of the link-layer context data */
    )
```

Description

This routine initialize a neighbor database, every neighbor node is assigned a timeout handle.

Returns

OK, or **ERROR** if the neighbor database cannot be initialized.

Errno

ENOMEM

Not possible to bind all timeouts.

S_semLib_NOT_ENOUGH_MEMORY

Not enough memory available to initiaize the generation lock.

See also

neighLib

Name

neighTx( ) - transmit a packet

Layer

RTNET

Synopsis

```
STATUS neighTx
    (
    NEIGH_DB  db,       /**< Neighbor database */
    PKT_CTX * pktCtx    /**< Packet context */
    )
```

Description

This routine tries to transmit a packet to a next-hop IP-address stored in the packet context. Node context data stored by the [updateRtn] passed to **neighDbCreate( )** is prepended to the packet and the packet is then passed to [txRtn] if there is a valid neighbor entry. If there is not a valid neighbor entry, the packet is queued and the [probeRtn] passed to **neighDbCreate( )** is called to request link layer resolution of the node owning the next-hop IP-address.

Returns

OK if the packet was successfully delivered to the driver layer or if the packet was successfully queued, or **ERROR** otherwise.

Errno

EINVAL

No next-hop IP-address is set on the packet context.

ENOBUF

A packet was dropped while waiting for IP-address-to-link-address. resolution

See also

neighLib, **neighDbCreate( )**

Name

neighUpdate( ) - update the state of a neighbor node

Layer

RTNET

Synopsis

```
void neighUpdate
    (
    NEIGH_DB                 db,          /**< Neighbor database */
    const struct in6_addr *  nh,          /**< Next-hop IP-address */
    const void *             nhLaddr,     /**< Next-hop link-address */
    BOOL                     solicited,   /**< Solicited update */
    BOOL                     override,    /**< Override link-address */
    NEIGH_ROUTER_FLAG_ACTION isRouter     /**< Router flag update */
    )
```

Description

This routine updates the state of a neighbor node as described in appendix C in RFC 4861.

Returns

N/A

Errno

Not Available

See also

neighLib

Name

neighForEach( ) - call an user-defined function for each neighbor node

Layer

RTNET

Synopsis

```
void neighForEach
    (
    NEIGH_DB          db,        /**< Neighbor database */
    NEIGH_FOREACH_RTN rtn,       /**< User-defined function */
    void *            userData   /**< User supplied context data */
    )
```

Description

This routine call an user-defined function for each neighbor node in the neighbor database.

Returns

N/A

Errno

Not Available

See also

neighLib

Name

neighNetifGet( ) - get network interface

Layer

RTNET

Synopsis

```
NETIF neighNetifGet
    (
    NEIGH_DB db
    )
```

Description

This routinte get the network interface associated with the neighbor database.

Returns

Network interface

Errno

N/A

See also

neighLib

Name

neighFlush( ) - clear all neighbor entries

Layer

RTNET

Synopsis

```
void neighFlush(void)
```

Description

This routine try to clear all neighbor entries.

Returns

N/A

Errno

Not Available

See also

neighLib

Name

neighStateUpdateRegister( ) - register neighbor state handler

Layer

RTNET

Synopsis

```
void neighStateUpdateRegister
    (
    NEIGH_STATE_UPDATE_RTN HandlerRtn   /**< probe function to use */
    )
```

Description

This routine try to register neighbor state change callback function.

Returns

N/A

Errno

Not Available

See also

neighLib

Name

neighNaRx( ) - neighbor advertisement

Layer

RTNET

Synopsis

```
BOOL neighNaRx
    (
    NEIGH_DB                 db,          /**< Neighbor database */
    const struct in6_addr *  nh,          /**< Next-hop IP-address */
    const void *             nhLaddr,     /**< Next-hop link-address */
    BOOL                     solicited,   /**< Solicited update */
    BOOL                     override,    /**< Override link-address */
    NEIGH_ROUTER_FLAG_ACTION isRouter     /**< Router flag update */
    )
```

Description

This routine checks the neighbor state when receiving neighbor advertisement as 7.2.5 in RFC 4861.

Returns

TRUE if the neighbor advertisement is ok, **FALSE** discards the neighbor advertisement.

Errno

Not Available

See also

neighLib

33. NEIGHSHOW

Name

neighShow - C-interpreter neighbor show commands

Layer

RTNET

Routines

neighShow( ) - show all neighbors **arp4Show( )** - show all IPv4 neighbors **ndpShow( )** - show all IPv6 neighbors

Description

This module contain routines that print neighbor entries in the system. Neighbor entries are mapping from an IPv4 or IPv6 address to the link-layer address of the next-hop node a packet must be delivered to in order to reach the destination IP-address. The next-hop node is always located on the same network link as this node, hence the name "neighbor".

Include files

N/A

Name

neighShow( ) - show all neighbors

Layer

RTNET

Synopsis

```
INT32 neighShow
    (
    int family
    )
```

Description

This routine show all neighbors, both **AF_INET** and **AF_INET6**.

Returns

Number of neighbors printed.

Errno

N/A

See also

neighShow

Name

arp4Show( ) - show all IPv4 neighbors

Layer

RTNET

Synopsis

```
INT32 arp4Show (void)
```

Description

This routine show all IPv4 neighbors.

Returns

Number of neighbors printed.

Errno

N/A

See also

neighShow

Name

ndpShow( ) - show all IPv6 neighbors

Layer

RTNET

Synopsis

```
INT32 ndpShow (void)
```

Description

This routine show all IPv6 neighbors.

Returns

Number of neighbors printed.

Errno

N/A

See also

neighShow

34. NETBUFADVLIB

Name

netBufAdvLib - network buffer library advanced routines

Layer

NET_BASE

Routines

netBufAdvLibInit( ) - initialize **netBufAdvLib** **netPoolAttach( )** - attach to the named pool **netPoolDetach( )** - detach and decrement usage reference **netPoolBind( )** - bind a child pool to a parent netPool, allow resource sharing **netPoolUnbind( )** - unbind two pools **netTupleListFree( )** - frees a Tuple list to the netPool **netTupleListGet( )** - get a list of mBlk-clBlk-cluster Tuples **netTuplePhysAddrMap( )** - translate a Tuple chain's buffers into physical address

Description

This library implements advanced features of the network buffer library, allowing network buffer pools to be shared, to enter parent-child, and to maintain management information about the properties of the pools.

Also, this library provides routines to allocate lists of tuples, and obtain the virtual and physical address of the buffers described by a chain of tuples.

This module is separated from **netBufLib** for better scalability when the routines in this library are not used.

To use this feature, include the following component: **INCLUDE_NETBUFADVLIB**

Include files

netBufLib.h

See also

netBufLib, **netPoolShow**, VxWorks 7 Network Stack Programmer's Guide

Name

netBufAdvLibInit( ) - initialize **netBufAdvLib**

Layer

NET_BASE

Synopsis

```
STATUS netBufAdvLibInit (void)
```

Description

This routine executes during system startup if **INCLUDE_NETWORK** is defined when the image is built. It links the network buffer library into the image.

Returns

OK or **ERROR**.

Errno

See also

netBufAdvLib

Name

netPoolAttach( ) - attach to the named pool

Layer

NET_BASE

Synopsis

```
NET_POOL_ID netPoolAttach
    (
    const char *pName   /* Pool name */
    )
```

Description

This routine allows a driver or kernel task to attach to an existing netBuf pool. Once attached, the calling task may call the existing API. Attaching to a pool is allowed only if the netBuf pools attributes are set to sharable. Pools with attached drivers cannot be deleted.

Returns

NET_POOL_ID or **NULL** on error

Errno

S_netBufLib_NETPOOL_INVALID

Returned if pool is invalid or **NULL**

See also

netBufAdvLib, **netPoolCreate( ) netPoolDetach( )**

Name

netPoolDetach( ) - detach and decrement usage reference

Layer

NET_BASE

Synopsis

```
STATUS netPoolDetach
    (
    NET_POOL_ID pNetPool   /* pointer to a net pool */
    )
```

Description

This routine decrements the attachment count created via a call to **netPoolAttach( )**. A netPool must be free of attachments or bindings before the netPool can be deleted.

Returns

OK or **ERROR**

Errno

S_netBufLib_NETPOOL_INVALID

Returned if pool is invalid or **NULL**

See also

netBufAdvLib, **netPoolCreate( ) netPoolAttach( )**

Name

netPoolBind( ) - bind a child pool to a parent netPool, allow resource sharing

Layer

NET_BASE

Synopsis

```
STATUS netPoolBind
    (
    NET_POOL_ID pNetPoolChild,   /* Child Pool name */
    const char  *pParentName     /* name of Parent Pool */
    )
```

Description

This routine binds the pNetPoolChild pool to the named parent. Once a bind is successful, the child pool may borrow resources from the parent pool. A bind succeeds only for netPools created with **netPoolCreate( )** and the pools must have the same attributes.

Returns

OK or **ERROR**

Errno

S_netBufLib_NETPOOL_INVALID

Returned if pool is invalid or **NULL**

S_netBufLib_DUPLICATE_POOL

The bind request failed because the child netPool was found within the upper hierarchy.

See also

netBufAdvLib, **netPoolCreate( ) netPoolUnbind( )**

Name

netPoolUnbind( ) - unbind two pools

Layer

NET_BASE

Synopsis

```
STATUS netPoolUnbind
    (
    NET_POOL_ID pNetPool   /* pointer to a net pool */
    )
```

Description

This routine removes a hierarchical bind between a child and parent netPool.

Returns

OK or **ERROR**

Errno

S_netBufLib_NETPOOL_INVALID

Returned if pool is invalid or **NULL**

See also

netBufAdvLib, **netPoolBind( )** **netPoolDestroy( )** **netPoolRelease( )**

Name

netTupleListFree( ) - frees a Tuple list to the netPool

Layer

NET_BASE

Synopsis

```
STATUS netTupleListFree
    (
    M_BLK_ID pMblk   /* Queue of mBlk to free */
    )
```

Description

This routine returns the Tuple list to the pool. All Tuples on the mNext and mNextPkt links are freed.

Returns

OK or **ERROR**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

See also

netBufAdvLib, **netTupleGet( ) netTupleListGet( )**

Name

netTupleListGet( ) - get a list of mBlk-clBlk-cluster Tuples

Layer

NET_BASE

Synopsis

```
int netTupleListGet
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    CL_POOL_ID  pClPool,    /* ptr to the cluster pool */
    int         count,      /* Number of tuples to get */
    int         type,       /* mBlk Type */
    M_BLK_ID *  ppMblk      /* OUT: Container to place tuple list */
    )
```

Description

This routine attempts to allocate count number of mBlk-clBlk-cluster triplets (tuples) from the specified pool, pNetPool. These tuples are joined together in a list using the **mBlkHdr.mNextPkt** pointer in the **mBlk** structure. The actual number of tuples allocated is returned; this may be less than the requested count if the pool becomes exhausted. Also *ppMBlk is updated with the list head, unless **ERROR** is returned.

Returns

The number of Tuples in the list, or **ERROR** for invalid arguments.

Errno

S_netBufLib_INVALID_ARGUMENT

One or more of the arguments is invalid.

See also

netBufAdvLib, **netTupleListFree( )**

Name

netTuplePhysAddrMap( ) - translate a Tuple chain's buffers into physical address

Layer

NET_BASE

Synopsis

```
STATUS netTuplePhysAddrMap
    (
    M_BLK_ID   pMblk,       /* pointer to the source mBlk chain */
    BUF_VTOP * pVtopTbl,    /* Virtual to Physical table */
    int *      pVtopCount   /* IN: Array elements count */
                            /* OUT: translation count */
    )
```

Description

This routine walks down a Tuple chain and returns the Physical address and length for each buffer segment. The information is placed into the **BUF_VTOP** table pVtopTbl. If the table is not large enough pVtopCount, and **ERROR** is returned. On return, pVtopCount is updated with the number of table entries that were updated.

netTuplePhysAddrMap( ) is intended for use by drivers and DMA engines.

BUF_VTOP TABLE:

pMBlk

Associated mBlk. In some situations, an mBlk may span multiple physical address.

pVAddr

Virtual Address

pPAddr

Physical Address

length

Length of segment

Returns

OK or **ERROR**

Errno

None

See also

netBufAdvLib

35. NETBUFLIB

Name

netBufLib - network buffer library

Layer

NET_BASE

Routines

netBufLibInitialize( ) - initialize **netBufLib** **netBufLibInit( )** - initialize **netBufLib** (deprecated) **netPoolInit( )** - initialize a **netBufLib**-managed memory pool **netPoolKheapInit( )** - kernel heap version of **netPoolInit( )** **netPoolDelete( )** - delete a memory pool **netMblkFree( )** - free an **mBlk** back to its memory pool **netClBlkFree( )** - free a **clBlk**-cluster construct back to the memory pool **netClFree( )** - free a cluster back to the memory pool **netMblkClFree( )** - free an **mBlk**-`clBlk'-cluster construct **netMblkClChainFree( )** - free a chain of **mBlk**-`clBlk'-cluster constructs **netMblkGet( )** - get an **mBlk** from a memory pool **netClBlkGet( )** - get a **clBlk** **netClusterGet( )** - get a cluster from the specified cluster pool **netMblkClGet( )** - get a **clBlk**-cluster and join it to the specified **mBlk** **netTupleGet( )** - get an **mBlk**-`clBlk'-cluster **netTupleFree( )** - free an **mBlk**-`clBlk'-cluster **netClBlkJoin( )** - join a cluster to a **clBlk** structure **netMblkClJoin( )** - join an **mBlk** to a **clBlk**-cluster construct **netClPoolIdGet( )** - return a **CL_POOL_ID** for a specified buffer size **netMblkToBufCopy( )** - copy data from an **mBlk** to a buffer **netMblkDup( )** - duplicate an **mBlk** **netMblkChainDup( )** - duplicate an **mBlk** chain **netPoolCreate( )** - create a buffer pool **netPoolRelease( )** - release a network buffer pool **netPoolIdGet( )** - returns net pool ID by name **netPoolNameGet( )** - returns netPool name

Description

This library contains routines that you can use to organize and maintain a memory pool that consists of pools of **mBlk** structures, pools of **clBlk** structures, and pools of clusters. The **mBlk** and **clBlk** structures are used to manage the clusters. The clusters are containers for the data described by the **mBlk** and **clBlk** structures.

These structures and the various routines of this library constitute a buffering API that has been designed to meet the needs both of network protocols and network device drivers.

The **mBlk** structure is the primary vehicle for passing data between a network driver and a protocol. However, the **mBlk** structure must first be properly joined with a **clBlk** structure that was previously joined with a cluster. Thus, the actual vehicle for passing data is not merely an **mBlk** structure but an **mBlk**-`clBlk'-cluster construct.

The **netBufLib** API is backed by a set of underlying methods that perform most of its intrinsic operations (initializing a pool, allocating and releasing buffers, statistics management). The default set of methods allocate **mBlk,** `clBlk' and clusters from individual pools, and require all three to be joined together to form a tuple. A second set of methods is available which allows pre-joined tuples to be allocated in a single call. To use these methods, one needs to pass **_pLinkPoolFuncTbl** as the pFuncTbl argument to the **netPoolInit( )** routine.

To use this feature, include the following component: **INCLUDE_NETBUFLIB**

Include files

netBufLib.h

See also

netBufAdvLib, **netPoolShow**. VxWorks 7 Network Stack Programmer's Guide

Name

netBufLibInitialize( ) - initialize **netBufLib**

Layer

NET_BASE

Synopsis

```
STATUS netBufLibInitialize
    (
    int drvClOffset   /* default cluster offset for 'driver' pools */
    )
```

Description

This routine executes during system startup if **INCLUDE_NETBUFLIB** is defined when the image is built. It links the network buffer library into the image, and also specifies the default leading cluster pad space to use for subsequently created **driver** pools, i.e. those having only one cluster size, and that cluster size at least 1500 bytes. The default leading cluster pad space value is modified only if the specified argument drvClOffset is non-negative.

It is recommended that components needing the facilities of this library establish a dependency upon **INCLUDE_NETBUFLIB** rather than calling **netBufLibInitialize( )** directly.

Returns

OK or **ERROR**.

Errno

See also

netBufLib

Name

netBufLibInit( ) - initialize **netBufLib** (deprecated)

Layer

NET_BASE

Synopsis

```
STATUS netBufLibInit (void)
```

Description

This routine is deprecated; it simply calls **netBufLibInitialize( )** with an argument of -1, to avoid modifying the default leading cluster pad space value.

Do not use this routine, as it will be removed in a subsequent release.

Returns

OK or **ERROR**.

Errno

See also

netBufLib

Name

netPoolInit( ) - initialize a **netBufLib**-managed memory pool

Layer

NET_BASE

Synopsis

```
STATUS netPoolInit
    (
    NET_POOL_ID   pNetPool,          /* pointer to a net pool */
    M_CL_CONFIG * pMclBlkConfig,     /* pointer to a mBlk configuration */
    CL_DESC *     pClDescTbl,        /* pointer to cluster desc table */
    int           clDescTblNumEnt,   /* number of cluster desc entries */
    POOL_FUNC *   pFuncTbl           /* pointer to pool function table */
    )
```

Description

Call this routine to set up a **netBufLib**-managed memory pool. Within this pool, **netPoolInit( )** organizes several sub-pools: one for **mBlk** structures, one for **clBlk** structures, and as many cluster sub-pools are there are cluster sizes. As input, this routine expects the following parameters:

pNetPool

Expects a **NET_POOL_ID** that points to a previously allocated **NET_POOL** structure. You need not initialize any values in this structure. That is handled by **netPoolInit( )**.

pMclBlkConfig

Expects a pointer to a previously allocated and initialized **M_CL_CONFIG** structure. Within this structure, you must provide four values: **mBlkNum**, a count of **mBlk** structures; **clBlkNum**, a count of **clBlk** structures; **memArea**, a pointer to an area of memory that can contain all the **mBlk** and **clBlk** structures; and **memSize**, the size of that memory area. For example, you can set up an **M_CL_CONFIG** structure as follows:

```
    M_CL_CONFIG mClBlkConfig = /* mBlk, clBlk configuration table */
        {
        mBlkNum     clBlkNum        memArea         memSize
        ----------  ----            -------         -------
        400,        245,            0xfe000000,     21260
        };
```

You can calculate the **memArea** and **memSize** values. Such code could first define a table as shown above, but set both **memArea** and **memSize** as follows:

```
mClBlkConfig.memSize = (mClBlkConfig.mBlkNum * (M_BLK_SZ + sizeof(void *))) +
                       (mClBlkConfig.clBlkNum * CL_BLK_SZ);
```

You can set the memArea value to a pointer to private memory, or you can reserve the memory with a call to **malloc( )**. For example:

```
    mClBlkConfig.memArea = malloc(mClBlkConfig.memSize);
```

The **netBufLib.h** file defines **M_BLK_SZ** as:

```
    sizeof(struct mBlk)
```

Currently, this evaluates to 32 bytes. Likewise, this file defines **CL_BLK_SZ** as:

```
    sizeof(struct clBlk)
```

Currently, this evaluates to 32 bytes.

When choosing values for **mBlkNum** and **clBlkNum**, remember that you need as many **clBlk** structures as you have clusters (data buffers). You also need at least as many **mBlk** structures as you have **clBlk** structures, but you will most likely need more. That is because **netBufLib** shares buffers by letting multiple **mBlk** structures join to the same **clBlk** and thus to its underlying cluster. The **clBlk** keeps a count of the number of **mBlk** structures that reference it.

pClDescTbl

Expects a pointer to a table of previously allocated and initialized **CL_DESC** structures. Each structure in this table describes a single cluster pool. You need a dedicated cluster pool for each cluster size you want to support. Within each **CL_DESC** structure, you must provide four values: **clusterSize**, the size of a cluster in this cluster pool; **num**, the number of clusters in this cluster pool; **memArea**, a pointer to an area of memory that can contain all the clusters; and **memSize**, the size of that memory area.

Thus, if you need to support six different cluster sizes, this parameter must point to a table containing six **CL_DESC** structures. For example, consider the following:

```
    CL_DESC clDescTbl [] =   /* cluster descriptor table */
        {
        /*
        clusterSize        num     memArea         memSize
        ----------         ----    -------         -------
        */
        {64,               100,    0x10000,        6800},
        {128,              50,     0x20000,        6600},
        {256,              50,     0x30000,        13000},
        {512,              25,     0x40000,        12900},
        {1024,             10,     0x50000,        10280},
        {2048,             10,     0x60000,        20520}
        };
```

As with the **memArea** and **memSize** members in the **M_CL_CONFIG** structure, you can set these members of the **CL_DESC** structures by calculation after you create the table. The formula would be as follows, for a pool using the default function table:

```
    clDescTbl[n].memSize =
        clDescTbl[n].num * (clDescTbl[n].clusterSize + sizeof (CL_POOL_ID));
```

This legacy formula assumes only that each cluster size and cluster start address is a multiple of 4 bytes. The network stack system and data pools are allocated with more stringent alignment requirements, and so the memory sizes for those pools are a bit bigger.

The formula for a pool using the function table at _pLinkPoolFuncTbl is different and is given below.

The **memArea** member can point to a private memory area that you know to be available for storing clusters, or you can use **malloc( )**.

```
clDescTbl[n].memArea =  malloc( clDescTbl[n].memSize );
```

Valid cluster sizes range from 16 to 65536 bytes on 4-byte alignment. Supported buffer sizes within a **netBufLib** pool must be unique within a size range that is bounded by powers of two. The bottom power of two is included in the range. The upper power of two is in the next range. There are 13 supported ranges:

16	<= cluster size <=	31	
32	<= cluster size <=	63	
64	<= cluster size <=	127	
128	<= cluster size <=	255	
256	<= cluster size <=	511	
512	<= cluster size <=	1023	
1024	<= cluster size <=	2047	
2048	<= cluster size <=	4095	
4096	<= cluster size <=	8191	
8192	<= cluster size <=	16347	
16348	<= cluster size <=	32767	
32768	<= cluster size <=	65535	
65536	= cluster size		

clDescTblNumEnt

Expects a count of the elements in the **CL_DESC** table referenced by the pClDescTbl parameter. This is a count of the number of cluster pools. You can get this value using the NELEMENTS macro defined in **vxWorks.h**. For example:

```
int clDescTblNumEnt = (NELEMENTS(clDescTbl));
```

pFuncTbl

Expects a **NULL** or a pointer to a function table. This table contains pointers to the functions used to manage the buffers in this memory pool. Using a **NULL** for this parameter tells **netBufLib** to use its default function table. If you opt for the default function table, every **mBlk** and every cluster is prepended by a 4-byte header (which is why the size calculations above for clusters and **mBlk** structures contained an extra **sizeof(long)**). However, users need not concern themselves with this header when accessing these buffers. The returned pointers from functions such as **netClusterGet( )** return pointers to the start of data, which is just after the header.

If pFuncTbl is set to **_pLinkPoolFuncTbl, netBufLib** will use an alternate function table which is designed mainly for use by device drivers. These functions manage buffers as fully assembled tuples which do not need to be broken up once released. This reduces the amount of time needed to allocate and free them. Also, these functions insure that **mBlk** and **clBlk** structures, as well as clusters, are all aligned on cache line boundaries. Note however that there currently is no consistent way to determine the correct cache line size for a given architecture when building the network code, so the default alignment used internally by the library is 64. This will satisfy architectures that require 64, 32 or 16 byte alignment.

When using the alternate function set, the following restrictions apply:

Each pool can have only one size of cluster (clDescTblNumEnt must be 1). Only the first cluster pool (**clDescTbl[0]**) is valid.

There must be an equal number of **mBlks,** `clBlks' and clusters: **mBlkNum,** 'clBlkNum' and **clDescTbl[0].num** must be the same.

The amount of memory for **memArea** in pMclBlkConfig should be computed as follows:

```
mClBlkConfig.memSize = (mClBlkConfig.mBlkNum *
                       ROUND_UP((sizeof(M_LINK) + sizeof(NET_POOL_ID)),
                                NETBUF_ALIGN)) + NETBUF_ALIGN;
```

The amount of memory for **memSize** in pClDescTbl should be computed as follows:

```
    clDescTbl[0].memSize =
        (clDescTbl[0].num *
         ROUND_UP(clDescTbl[0].clusterSize, NETBUF_ALIGN) +
         NETBUF_ALIGN;
```

Tuples must be allocated using the **netTupleGet( )** or **netMblkGet( )** routines. The underlying methods for **netClBlkGet( )** and **netClusterGet( )** are not implemented and always return **NULL**, hence these API calls should not be used.

Assuming you have set up the configuration tables as shown above, a typical call to **netPoolInit( )** would be as follows:

```
    int clDescTblNumEnt = (NELEMENTS(clDescTbl));
    NET_POOL  netPool;
    NET_POOL_ID       pNetPool = &netPool;

    if (netPoolInit (pNetPool, &mClBlkConfig, &clDescTbl [0], clDescTblNumEnt,
        NULL) != OK)
        return (ERROR);
```

The memory resources for netPools created using **netPoolInit( ) netPoolKheapInit( )** are not managed by the netBuf library. Such netPools have limitations regarding object management. Therefore, these netPools can not be destroyed gracefully via a call to **netPoolRelease( )**. Use **netPoolCreate( )** to create netPools that are fully managed by **netBufLib**.

Returns

OK or **ERROR**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_MEMSIZE_INVALID

Invalid netPool configuration.

S_netBufLib_CLSIZE_INVALID

Invalid netPool configuration.

S_netBufLib_NO_SYSTEM_MEMORY

Out of Memory.

S_netBufLib_MEM_UNALIGNED

Invalid netPool configuration.

S_netBufLib_MEMSIZE_UNALIGNED

Invalid netPool configuration.

S_netBufLib_MEMAREA_INVALID

Invalid netPool configuration.

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

See also

netBufLib, **netPoolDelete( )** or **netPoolCreate( )**

Name

netPoolKheapInit( ) - kernel heap version of **netPoolInit( )**

Layer

NET_BASE

Synopsis

```
STATUS netPoolKheapInit
    (
    NET_POOL_ID   pNetPool,          /* pointer to a net pool */
    M_CL_CONFIG * pMclBlkConfig,     /* pointer to a mBlk configuration */
    CL_DESC *     pClDescTbl,        /* pointer to cluster desc table */
    int           clDescTblNumEnt,   /* number of cluster desc entries */
    POOL_FUNC *   pFuncTbl           /* pointer to pool function table */
    )
```

Description

This initializes a **netBufLib**-managed memory pool from Kernel heap. See **netPoolInit( )** for more detail.

The memory resources for netPools created using **netPoolInit( )** and **netPoolKheapInit( )** are not managed by the netBuf library. Such netPools have limitations regarding object management. Therefore, these netPools can not be destroyed via a call to **netPoolDestroy( )**. Use **netPoolCreate( )** to create netPools that are fully managed by **netBufLib**.

Returns

OK or **ERROR**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_MEMSIZE_INVALID

Invalid netPool configuration.

S_netBufLib_CLSIZE_INVALID

Invalid netPool configuration.

S_netBufLib_NO_SYSTEM_MEMORY

Out of Memory.

S_netBufLib_MEM_UNALIGNED

Invalid netPool configuration.

S_netBufLib_MEMSIZE_UNALIGNED

Invalid netPool configuration.

S_netBufLib_MEMAREA_INVALID

Invalid netPool configuration.

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

See also

netBufLib, **netPoolInit( )**, **netPoolDelete( )** or **netPoolCreate( )**

Name

netPoolDelete( ) - delete a memory pool

Layer

NET_BASE

Synopsis

```
STATUS netPoolDelete
    (
    NET_POOL_ID pNetPool   /* pointer to a net pool */
    )
```

Description

This routine deletes the specified **netBufLib**-managed memory pool.

Returns

OK or **ERROR**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

See also

netBufLib

Name

netMblkFree( ) - free an **mBlk** back to its memory pool

Layer

NET_BASE

Synopsis

```
void netMblkFree
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    M_BLK_ID    pMblk       /* mBlk to free */
    )
```

Description

This routine frees the specified **mBlk** back to the specified memory pool.

Returns

N/A

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_MBLK_INVALID

An invalid or corrupted mBlk was detected

See also

netBufLib, **netMblkGet( )**

Name

netClBlkFree( ) - free a **clBlk**-cluster construct back to the memory pool

Layer

NET_BASE

Synopsis

```
void netClBlkFree
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    CL_BLK_ID   pClBlk      /* pointer to the clBlk to free */
    )
```

Description

This routine decrements the reference counter in the specified **clBlk**. If the reference count falls to zero, this routine frees both the **clBlk** and its associated cluster back to the specified memory pool.

Returns

N/A

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

See also

netBufLib

Name

netClFree( ) - free a cluster back to the memory pool

Layer

NET_BASE

Synopsis

```
void _netClFree
    (
    NET_POOL_ID pNetPool,    /* pointer to the net pool */
    UCHAR *     pClBuf,      /* pointer to the cluster buffer */
    char *      pFuncName,
    char *      pFileName,
    int         lineNum
    )
```

Description

This routine returns the specified cluster buffer back to the specified memory pool.

Returns

N/A

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

See also

netBufLib

Name

netMblkClFree( ) - free an **mBlk**-`clBlk'-cluster construct

Layer

NET_BASE

Synopsis

```
M_BLK_ID netMblkClFree
    (
    M_BLK_ID pMblk   /* pointer to the mBlk */
    )
```

Description

For the specified **mBlk**-`clBlk'-cluster construct, this routine frees the **mBlk** back to the specified memory pool. It also decrements the reference count in the **clBlk** structure. If the reference count falls to zero, no other **mBlk** structure reference this **clBlk**. In that case, this routine also frees the **clBlk** structure and its associated cluster back to the specified memory pool.

Returns

If the specified **mBlk** was part of an **mBlk** chain, this routine returns a pointer to the next **mBlk**. Otherwise, it returns a **NULL**. It is not considered an error if pMblk is **NULL**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_MBLK_INVALID

An invalid or corrupted mBlk was detected

See also

netBufLib

Name

netMblkClChainFree( ) - free a chain of **mBlk**-`clBlk'-cluster constructs

Layer

NET_BASE

Synopsis

```
void netMblkClChainFree
    (
    M_BLK_ID pMblk   /* pointer to the mBlk */
    )
```

Description

For the specified chain of **mBlk**-`clBlk'-cluster constructs, this routine frees all the **mBlk** structures back to the specified memory pool. It also decrements the reference count in all the **clBlk** structures. If the reference count in a **clBlk** falls to zero, this routine also frees that **clBlk** and its associated cluster back to the specified memory pool.

Returns

N/A

Errno

S_netBufLib_MBLK_INVALID

An invalid of corrupt mBlk was detected.

See also

netBufLib

Name

netMblkGet( ) - get an **mBlk** from a memory pool

Layer

NET_BASE

Synopsis

```
M_BLK_ID netMblkGet
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    int         canWait,    /* M_WAIT/M_DONTWAIT */
    UCHAR       type        /* mBlk type */
    )
```

Description

This routine allocates an **mBlk** from the specified memory pool, if available.

pNetPool

Expects a pointer to the pool from which you want an **mBlk**.

canWait

Expects either **M_WAIT** or **M_DONTWAIT**. If no **mBlk** is immediately available, the **M_WAIT** value allows this routine to repeat the allocation attempt after performing garbage collection. It omits these steps when the **M_DONTWAIT** value is used.

type

Expects the type value that you want to associate with the returned **mBlk**.

Warning

The net pool ID pNetPool should not be the pool ID of a pool using the linkBufPool back end, as such a pool cannot safely supply bare M_BLKs. An **M_BLK** from a linkBufPool pool is assumed always attached to its adjacent cluster block. Treating an **M_BLK** obtained from a linkBufPool as bare, for instance by attaching it to another cluster block, can result in a memory leak. Several network drivers use linkBufPool pools, so do not use a driver pool to allocate bare M_BLKs (unless you are absolutely sure the driver uses a pool which supports this). It is generally safest to treat driver pools as private to the driver, and to use a pool such as the network stack data pool (_pNetDpool) or the network stack system pool (_pNetSysPool), which are netBufPool-style pools known capable of providing bare M_BLKs.

Returns

M_BLK_ID or **NULL** if no **mBlk** is available.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_POOL_RELEASED

The netPool is in a release state. This call is prohibited.

See also

netBufLib, **netMblkFree( )**

Name

netClBlkGet( ) - get a **clBlk**

Layer

NET_BASE

Synopsis

```
CL_BLK_ID netClBlkGet
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    int         canWait     /* M_WAIT/M_DONTWAIT */
    )
```

Description

This routine gets a **clBlk** from the specified memory pool.

pNetPool

Expects a pointer to the pool from which you want a **clBlk**.

canWait

Expects either **M_WAIT** or **M_DONTWAIT**. If no **clBlk** is immediately available, the **M_WAIT** value allows this routine to repeat the allocation attempt after performing garbage collection. It omits these steps when the **M_DONTWAIT** value is used.

Returns

CL_BLK_ID or a **NULL** if no **clBlk** was available.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_POOL_RELEASED

The netPool is in a release state. This call is prohibited.

See also

netBufLib

Name

netClusterGet( ) - get a cluster from the specified cluster pool

Layer

NET_BASE

Synopsis

```
char * _netClusterGet
    (
    NET_POOL_ID pNetPool,    /* pointer to the net pool */
    CL_POOL_ID  pClPool,     /* ptr to the cluster pool */
    char *      pFuncName,
```

```
    char *      pFileName,
    int         lineNum
    )
```

Description

This routine gets a cluster from the specified cluster pool pClPool within the specified memory pool pNetPool.

Returns

This routine returns a character pointer to a cluster buffer or **NULL** if none was available.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_POOL_RELEASED

The netPool is in a release state. This call is prohibited.

See also

netBufLib

Name

netMblkClGet( ) - get a **clBlk**-cluster and join it to the specified **mBlk**

Layer

NET_BASE

Synopsis

```
STATUS netMblkClGet
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    M_BLK_ID    pMblk,      /* mBlk to embed the cluster in */
    int         bufSize,    /* size of the buffer to get */
    int         canWait,    /* wait or don't wait */
    BOOL        noSmaller   /* TRUE/FALSE */
    )
```

Description

This routine gets a **clBlk**-cluster pair from the specified memory pool and joins it to the specified **mBlk** structure. The **mBlk**-`clBlk'-cluster triplet it produces is the basic structure for handling data at all layers of the network stack.

pNetPool

Expects a pointer to the memory pool from which you want to get a free **clBlk**-cluster pair.

pMbkl

Expects a pointer to the **mBlk** structure (previously allocated) to which you want to join the retrieved **clBlk**-cluster pair.

bufSize

Expects the size, in bytes, of the cluster in the **clBlk**-cluster pair.

canWait

Expects either **M_WAIT** or **M_DONTWAIT**. If either item is not immediately available, the **M_WAIT** value allows this routine to repeat the allocation attempt after performing garbage collection. It omits those steps when the **M_DONTWAIT** value is used.

noSmaller

Expects either **TRUE** or **FALSE**. If noSmaller is **TRUE**, the routine will fail rather than allocating a smaller cluster than requested, but may return an equal or larger size. If noSmaller is **FALSE**, the routine may allocate a cluster of the closest smaller available size, if a cluster at least as large as requested is not available. noSmaller should be set to **TRUE** whenever a cluster of smaller than requested size is not acceptable. If a cluster cannot be allocated (according to these restrictions), the routine returns **ERROR**.

Returns

OK or **ERROR**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_CLSIZE_INVALID

An invalid Cluster size was used.

S_netBufLib_POOL_RELEASED

The netPool is in a release state. This call is prohibited.

See also

netBufLib

Name

netTupleGet( ) - get an **mBlk**-`clBlk'-cluster

Layer

NET_BASE

Synopsis

```
M_BLK_ID netTupleGet
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    int         bufSize,    /* size of the buffer to get */
```

```
    int         canWait,    /* wait or don't wait */
    UCHAR       type,       /* type of data */
    BOOL        noSmaller   /* TRUE/FALSE */
    )
```

Description

This routine gets an **mBlk**-`clBlk'-cluster triplet from the specified memory pool. The resulting structure is the basic method for accessing data at all layers of the network stack.

pNetPool

Expects a pointer to the memory pool with which you want to build a **mBlk**-`clBlk'-cluster triplet.

bufSize

Expects the size, in bytes, of the cluster in the **clBlk**-cluster pair.

canWait

Expects either **M_WAIT** or **M_DONTWAIT**. If any item in the triplet is not immediately available, the **M_WAIT** value allows this routine to repeat the allocation attempt after performing garbage collection. The **M_DONTWAIT** value prevents those extra steps.

type

Expects the type of data. For example **MT_DATA**, **MT_HEADER**. The various values for this type are defined in **netBufLib.h**.

noSmaller

Expects either **TRUE** or **FALSE**. If noSmaller is **TRUE**, the routine will fail rather than allocating a smaller cluster than requested, but may allocate one of equal or larger size. If noSmaller is **FALSE**, the routine may allocate a cluster of the closest smaller available size, if a cluster at least as large as requested is not available. noSmaller should be set to **TRUE** whenever a cluster of smaller than requested size is not acceptable. If a cluster cannot be allocated (according to these restrictions), the routine returns **ERROR**.

Returns

M_BLK_ID or **NULL**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_CLSIZE_INVALID

Invalid cluster size.

See also

netBufLib, **netTupleFree( )**

Name

netTupleFree( ) - free an **mBlk**-`clBlk'-cluster

Layer

NET_BASE

Synopsis

```
M_BLK_ID netTupleFree
    (
    M_BLK_ID pMblk   /* Tuple */
    )
```

Description

This routine returns an **mBlk**-`clBlk'-cluster [tuple] to the source memory netPool. An optimized netPool may return these resources in to the netPool in a manner that is efficient for re-use. Non optimized netPools may break the Tuple into components for future use.

Warning

This routine returns **NULL** in two different cases:

- pMblk is the only **M_BLK** in the chain;
- an error (invalid arguments, or support for the routine is missing in the **netBufLib** back-end pool implementation).

Returns

M_BLK_ID of next **M_BLK** in the chain (linked by the mBlkHdr.mNext member), or **NULL**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_MBLK_INVALID

The mBlk is invalid or corrupted.

See also

netBufLib, **netTupleGet( )**

Name

netClBlkJoin( ) - join a cluster to a **clBlk** structure

Layer

NET_BASE

Synopsis

```
CL_BLK_ID netClBlkJoin
    (
    CL_BLK_ID     pClBlk,     /* pointer to a cluster Block */
    char *        pClBuf,     /* pointer to a cluster buffer */
    int           size,       /* size of the cluster buffer */
    FUNCPTR       pFreeRtn,   /* pointer to the free routine */
    _Vx_usr_arg_t arg1,       /* argument 1 of the free routine */
    _Vx_usr_arg_t arg2,       /* argument 2 of the free routine */
    _Vx_usr_arg_t arg3        /* argument 3 of the free routine */
    )
```

Description

This routine joins the previously reserved cluster specified by pClBuf to the previously reserved **clBlk** structure specified by pClBlk. The size parameter passes in the size of the cluster referenced in pClBuf. The arguments pFreeRtn, arg1, arg2, arg3 set the values of the **pCLFreeRtn**, **clFreeArg1**, **clFreeArg2**, and **clFreeArg1**, members of the specified **clBlk** structure.

Returns

CL_BLK_ID or **NULL**.

Errno

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

See also

netBufLib

Name

netMblkClJoin( ) - join an **mBlk** to a **clBlk**-cluster construct

Layer

NET_BASE

Synopsis

```
M_BLK_ID netMblkClJoin
    (
    M_BLK_ID  pMblk,   /* pointer to an mBlk */
    CL_BLK_ID pClBlk   /* pointer to a cluster Blk */
    )
```

Description

This routine joins the previously reserved **mBlk** referenced in pMblk to the **clBlk**-cluster construct referenced in pClBlk. Internally, this routine sets the **M_EXT** flag in **mBlk.mBlkHdr.mFlags**. It also and sets the **mBlk.mBlkHdr.mData** to point to the start of the data in the cluster.

Returns

M_BLK_ID or **NULL**.

Errno

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

See also

netBufLib

Name

netClPoolIdGet( ) - return a **CL_POOL_ID** for a specified buffer size

Layer

NET_BASE

Synopsis

```
CL_POOL_ID netClPoolIdGet
    (
    NET_POOL_ID pNetPool,   /* pointer to the net pool */
    int         bufSize,    /* size of the buffer */
    BOOL        noSmaller   /* TRUE/FALSE */
    )
```

Description

This routine returns a **CL_POOL_ID** for a cluster pool containing clusters that match the specified bufSize. If noSmaller is **TRUE**, this routine returns a **CL_POOL_ID** for a pool of clusters of size greater than or equal to bufSize. If noSmaller is **FALSE**, this routine returns a **CL_POOL_ID** for a cluster from whatever cluster pool is available. noSmaller should be set to **TRUE** whenever a pool of clusters of smaller than requested size is not acceptable.

It is not guaranteed that the chosen cluster pool will actually have any clusters in it.

Returns

CL_POOL_ID or **NULL**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_POOL_RELEASED

The netPool is in a release state. This call is prohibited.

See also

netBufLib

Name

netMblkToBufCopy( ) - copy data from an **mBlk** to a buffer

Layer

NET_BASE

Synopsis

```
int netMblkToBufCopy
    (
    M_BLK_ID pMblk,     /* pointer to an mBlk */
    char *   pBuf,      /* pointer to the buffer to copy */
    FUNCPTR  pCopyRtn   /* function pointer for copy routine */
    )
```

Description

This routine copies data from the **mBlk** chain referenced in pMblk to the buffer referenced in pBuf. It is assumed that pBuf points to enough memory to contain all the data in the entire **mBlk** chain. The argument pCopyRtn expects either a **NULL** or a function pointer to a copy routine. The arguments passed to the copy routine are source pointer, destination pointer and the length of data to copy. If pCopyRtn is **NULL**, **netMblkToBufCopy( )** uses a default routine to extract the data from the chain.

Returns

The length of data copied or zero.

Errno

none

See also

netBufLib

Name

netMblkDup( ) - duplicate an **mBlk**

Layer

NET_BASE

Synopsis

```
M_BLK_ID netMblkDup
    (
    M_BLK_ID pSrcMblk,   /* pointer to source mBlk */
```

```
    M_BLK_ID pDestMblk   /* pointer to the destination mBlk */
    )
```

Description

This routine copies the references from a source **mBlk** in an **mBlk**-`clBlk'-cluster construct to a stand-alone **mBlk**. This lets the two **mBlk** structures share the same **clBlk**-cluster construct. This routine also increments the reference count in the shared **clBlk**. The pSrcMblk expects a pointer to the source **mBlk**. The pDescMblk parameter expects a pointer to the destination **mBlk**.

Returns

A pointer to the destination **mBlk** or **NULL** if the source **mBlk** referenced in pSrcMblk is not part of a valid **mBlk**-`clBlk'-cluster construct.

Errno

S_netBufLib_MBLK_INVALID

An invalid or corrupted mBlk was detected

See also

netBufLib

Name

netMblkChainDup( ) - duplicate an **mBlk** chain

Layer

NET_BASE

Synopsis

```
M_BLK_ID netMblkChainDup
    (
    NET_POOL_ID pNetPool,   /* pointer to the pool */
    M_BLK_ID    pMblk,      /* pointer to source mBlk chain*/
    int         offset,     /* offset to duplicate from */
    int         len,        /* length to copy */
    int         canWait     /* M_DONTWAIT/M_WAIT */
    )
```

Description

This routine makes a copy of an **mBlk** chain starting at offset bytes from the beginning of the chain and continuing for len bytes. If len is **M_COPYALL**, then this routine will copy the entire **mBlk** chain from the offset.

This routine copies the references from a source pMblk chain to a newly allocated **mBlk** chain. This lets the two **mBlk** chains share the same **clBlk**-cluster constructs. This routine also increments the reference count in the shared **clBlk**. The pMblk expects a pointer to the source **mBlk** chain. The pNetPool parameter expects a pointer to the netPool from which the new **mBlk** chain is allocated.

The canWait parameter determines the behavior if any required **mBlk** is not immediately available. A value of **M_WAIT** allows this routine to repeat the allocation attempt after performing garbage collection. The **M_DONTWAIT** value prevents those extra steps.

Warning

The net pool ID pNetPool should not be the pool ID of a pool using the linkBufPool back end, as such a pool cannot safely supply the bare M_BLKs needed for the duplicate chain. An **M_BLK** from a linkBufPool is assumed always attached to its adjacent cluster block. Using a linkBufPool pool ID for pNetPool can result in memory leak (usually of tuples or clusters from the original chain). Several network drivers use linkBufPool pools, so driver pools generally should not be used for this purpose. It is safest always to use a global pool such as the network stack data pool (_pNetDpool), a netBufPool-style pool known to provide source of bare M_BLKs.

Returns

A pointer to the newly allocated **mBlk** chain or **NULL**.

Errno

S_netBufLib_NETPOOL_INVALID

An invalid netPool was detected.

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

S_netBufLib_NO_POOL_MEMORY

NetPool is out of available memory for descriptors or clusters

See also

netBufLib, **netMblkDup( )**

Name

netPoolCreate( ) - create a buffer pool

Layer

NET_BASE

Synopsis

```
NET_POOL_ID netPoolCreate
    (
    NETBUF_CFG * pNetBufCfg,   /* Configuration Structure */
    POOL_FUNC *  pFuncTbl      /* Optional plug in function table */
    )
```

Description

This routine creates a net pool having extended feature including a name and attributes. All memory for the control structures and clusters (buffer) is allocated and initialized for the **netPool**. The configuration structure is made up of the following elements:

NETBUF_CFG pNetBufCfg

pName

pool Name is a unique string name. Each **netPool** may have a **NULL** name or a unique name. A unique pool name is required for pools that will support attachment, binding and search functions. The size of the name is limited by **NET_POOL_NAME_SZ**.

attributes

Valid attribute set: **ATTR_AI_SH_ISR**, **ATTR_AC_SH_ISR**, **ATTR_AP_SH_ISR**, **ATTR_AI_ISR**, **ATTR_AC_ISR** or **ATTR_AP_ISR**

The **NB_NO_LEADING_SPACE** bit may also be set in attributes to prevent the addition of default leading pad space to clusters in qualifying driver pools (those with one cluster pool, with cluster size at least 1500 bytes).

pDomain

RTP ID or **NULL** for kernel

ctrlNumber

Number of ctrl structures to pre-allocate. This value determines the amount of memory to allocate for mBlks.

ctrlPartId

Memory Partition for Control structures. A **NULL** value defaults to use the Kernel memory Partition.

bMemExtraSize

Allocate additional memory for runtime clusters (buffers) allocation. Additional memory is placed in a memory block that is usable during runtime operation. Whenever a specific cluster size is unavailable, the cluster is carved from the memory block to complete the caller's request. Therefore, the pool may carve buffers as needed for any cluster size defined in the netPool. The net effect is a pool that self tunes its cluster queues bases on the callers usage. If bMemExtraSize is set zero, runtime carving is not available. This mode is functionally compatible with vxWorks 5.5. bMemExtraSize does not effect clusters that are carved during initialization.

bMemPartId

Memory Partition for clusters (buffers). A **NULL** value defaults to use the Kernel memory Partition.

pClDescTbl

A **NETBUF_CL_DESC** table containing cluster sizes and cluster counts. The table may contain up to 10 entries.

clDescTblNumEnt

Number of entries in the pClDescTbl table.

pFuncTbl

pFuncTbl may be **NULL** or point to a system or user defined netPool plug in. If **NULL**, the generic netPool plug in is used. There are two defined plug ins:

Table Name	Description	pool name
_pNetPoolFuncTbl	Default Backwards compatible netPool	netBufPool
_pLinkPoolFuncTbl	Optimized linkBuf	linkBufPool

Netpool creation and enhancements

The **netBufLib** supports two techniques to create a **netPool**. The **netPoolInit( )** may be called to create a pool that is fully backward compatible with vxWorks 5.5. This call requires the caller to manage memory allocation and the pool may not be suitable for deletion. **netPoolCreate( )** is a simpler technique that provides management of all memory resources. Furthermore, **netPoolCreate( )** provide support for new features that may not be available using a **netPoolInit( )** created pool.

		Created via	**Created via**
Feature	**API**	**netPoolInit( )**	**netPoolCreate( )**
Attaching	**netPoolAttach( )** & **netPoolDetach( )**	No	Yes
Binding	**netPoolBind( )** & **netPoolUnbind( )**	No	Yes
Query	**netPoolIdGet( )** & **netPoolNameGet( )**	No	Yes
Optimized API	**netTupleListGet( )** & **netTupleListFree( )**	Yes / non	Yes
		optimized	Yes
Driver Support	**netTuplePhysAddrMap( )**	Yes	Yes
Pool Destruction	**netPoolDestroy( )**	No	Yes

Returns

netPool ID or **NULL** on error.

Errno

S_netBufLib_INVALID_ARGUMENT

One of the supplied arguments is invalid.

S_netBufLib_NO_SYSTEM_MEMORY

Out of System Memory

S_netBufLib_MEMSIZE_INVALID

Invalid configuration structure

S_netBufLib_CLSIZE_INVALID

Internal Error

S_netBufLib_MEM_UNALIGNED

Internal Error

S_netBufLib_MEMSIZE_UNALIGNED

Internal Error

S_netBufLib_DUPLICATE_POOL

Attempted to create a second pool with an existing name

S_netBufLib_MEMAREA_INVALID

Internal Error

See also

netBufLib, **netPoolInit( )**

Name

netPoolRelease( ) - release a network buffer pool

Layer

NET_BASE

Synopsis

```
STATUS netPoolRelease
    (
    NET_POOL_ID pNetPool,        /* pointer to a net pool */
    int         destroyContext   /* destroy Context */
    )
```

Description

This routine gracefully releases a network buffer pool. It starts a process which ends when all control structures and clusters have been returned to the pool, and the pool and all memory associated with it have been deleted. **netPoolRelease( )** is only supported for pools created using **netPoolCreate( )**, and only when the underlying network pool back-end implementation supports **netPoolRelease( )**.

The netPool is placed in a release state until all resources belonging to it are returned. Any subsequent attempt to allocate from the pool will fail. When all resources have been returned to the pool, the pool is automatically destroyed.

If netPoolRelease is called on a pool that is all ready in a release state, the pool is re-examined to see if it qualifies to be destroyed.

This routine is not callable from interrupt context.

pNetPool

Must be a valid **NET_POOL_ID** for a pool created via **netPoolCreate( )**. **netPoolRelease( )** will fail for a pool created using **netPoolInit( )**.

destroyContext

The destroyContext value must be either **NET_REL_IN_CONTEXT** or **NET_REL_IN_TASK**. **NET_REL_IN_CONTEXT** means the netPool is destroyed either in the **netPoolRelease( )** call itself, or in the context of whatever task returns the last resource from the pool. If it is possible that some resource from the pool may be returned from an ISR, this option should not be used. **NET_REL_IN_TASK** spawns a low priority task netPoolReleaseTask that will destroy the netPool when all resources have been returned to it. Although strongly discouraged, some older drivers free network pool resources within an interrupt context. In this case **NET_REL_IN_TASK** is required to property destroy the pool, since the freeing of the pool memory cannot happen in the ISR.

Returns

OK or **ERROR**

Errno

S_netBufLib_NETPOOL_INVALID

Returned if pool is invalid or if pool was not created via **netPoolCreate( )**.

S_netBufLib_POOL_RELEASE_IGNORE

The requested netPool cannot be released because it has either active attachments or bindings.

See also

netBufLib, **netPoolCreate( )**

Name

netPoolIdGet( ) - returns net pool ID by name

Layer

NET_BASE

Synopsis

```
NET_POOL_ID netPoolIdGet
    (
    const char * pPoolName   /* Pool name */
    )
```

Description

This routine returns the netPool ID by a given name. All netPools are checked against pPoolName for a match. If no match is found, error is returned. This routine is not callable from within an interrupt context.

pPoolName

A string containing the name of the pool to find. The netPool's name is unique and cannot exceed **NET_POOL_NAME_SZ**.

Notes:

1. Pool names are only valid for netPools created using netPoolCreate.
2. For vxWorks 5.5 Backwards compatible mode, pool names are **NULL**.
3. Searching for pools by name is not considered deterministic and may block.

Returns

NET_POOL_ID or **NULL** on error

Errno

S_netBufLib_LIB_NOT_INITIALIZED

netBufLib is not or initialized.

S_netBufLib_INVALID_ARGUMENT

Semaphore ID is invalid.

See also

netBufLib, **netPoolNameGet( )**

Name

netPoolNameGet( ) - returns netPool name

Layer

NET_BASE

Synopsis

```
char * netPoolNameGet
    (
    NET_POOL_ID pNetPool   /* pointer to the pool */
    )
```

Description

This routine returns a pointer to the name of a given netBuf pool. The name is only valid as long as the netPool is active.

pNetPool

Valid netPool ID.

Notes:

1. Pool names are only valid for netPools created using netPoolCreate.
2. For vxWorks 5.5 Backwards compatible mode, pool names are **NULL**.

Returns

Pointer to name or **NULL** on error

Errno

S_netBufLib_NETPOOL_INVALID

Returned if pool is invalid or if pool was not created via **netPoolCreate( )**.

See also

netBufLib, **netPoolIdGet( )**

36. NETLINKLIB

Name

netlinkLib - netlink helper library

Layer

RTNET

Routines

netlinkBufGet( ) - get a netlink buffer **netlinkBufPut( )** - put back a netlink buffer **netlinkBufPush( )** - push data into to netlink message **netlinkBufScopeStart( )** - start a netlink scope **netlinkBufScopeFinish( )** - finish a netlink scope **netlinkHdrScopeRtn( )** - finish routine for netlink message **netlinkBufPut( )** - deliver a netlink message to a socket **netlinkReq( )** - process netlink request **netlinkLibInit( )** - initialize netlink library

Description

This module contain helper functions for to help protocol handlers deal with netlink messages.

The parts relevant for VxWorks of the following RFCs are implemented by this module. /ts RFC | Title -----|------ 3449 | Linux Netlink as an IP Services Protocol /te

Include files

netlinkLib.h

See also

VxWorks 7 Network Stack Programmer's Guide

Name

netlinkBufGet( ) - get a netlink buffer

Layer

RTNET

Synopsis

```
NL_BUF_ID netlinkBufGet (void)
```

Description

This routine get a netlink buffer that may be used to dynamically build arbitrary netlink messages.

Returns

RET_DESC

Errno

N/A

See also

netlinkLib

Name

netlinkBufPut( ) - put back a netlink buffer

Layer

RTNET

Synopsis

```
void netlinkBufPut
    (
    NL_BUF_ID buf   /**< netlink message buffer */
    )
```

Description

This routine is called to tell this library that the caller is done using the buffer.

Returns

N/A

Errno

N/A

See also

netlinkLib

Name

netlinkBufPush( ) - push data into to netlink message

Layer

RTNET

Synopsis

```
void netlinkBufPush
    (
    NL_BUF_ID    buf,      /**< netlink message buffer */
```

```
    const void * data,     /**< data to push to buffer end of */
    UINT32       dataLen   /**< length of data */
    )
```

Description

This routine push a data area into a netlink message, the caller must have initialized all parts of the pushed data.

Returns

N/A

Errno

N/A

See also

netlinkLib

Name

netlinkBufScopeStart( ) - start a netlink scope

Layer

RTNET

Synopsis

```
void netlinkBufScopeStart
    (
    NL_BUF_ID buf,     /**< netlink buffer */
    UINT32    hdrLen   /**< length of header of this scope */
    )
```

Description

This routine start a netlink scope, the length of data pushed up to the point **netlinkBufScopeFinish( )** is called is tracked. Space for the header is reserved in this stage, the header must be written by the routine passed to **netlinkBufScopeFinished( )**.

Returns

N/A

Errno

N/A

See also

netlinkLib

Name

netlinkBufScopeFinish( ) - finish a netlink scope

Layer

RTNET

Synopsis

```
void netlinkBufScopeFinish
    (
    NL_BUF_ID        buf,   /**< netlink buffer */
    NL_BUF_SCOPE_RTN rtn,   /**< netlink scope finish routine */
    ...                     /**< additional argument */
    )
```

Description

This routine finish a netlink scope and call the scope handler that can write the length of the header and payload, all additional header fiels must also be set by the passed routine.

Returns

N/A

Errno

N/A

See also

netlinkLib

Name

netlinkHdrScopeRtn( ) - finish routine for netlink message

Layer

RTNET

Synopsis

```
void netlinkHdrScopeRtn
    (
    void *  ptr,   /**< pointer to netlink header */
    UINT32  len,   /**< total length of message */
    va_list args   /**< message type, NLM_F_xxx flags, sequence numbers */
                   /* and port ID */
    )
```

Description

This routine finish a netlink message by writing down the actual message length in the header. The normal usage of this routine is as argument to **netlinkBufScopeFinish( )** when finishing the outer most scope.

Returns

N/A

Errno

Not Available

See also

netlinkLib, **netlinkBufScopeFinish( )**

Name

netlinkBufPut( ) - deliver a netlink message to a socket

Layer

RTNET

Synopsis

```
void netlinkSockQueue
    (
    const SYS_SOCK_FLOW * flow,    /**< netlink socket flow */
    NL_MSG_BUF *          buf      /**< netlink buffer to queue on socket */
    )
```

Description

This routine deliver a netlink message to the receive buffer of

Returns

N/A

Errno

N/A

See also

netlinkLib

Name

netlinkReq( ) - process netlink request

Layer

RTNET

Synopsis

```
STATUS netlinkReq
    (
    SOCKET *   sock,    /**< netlink socket */
    NL_REQ_RTN reqRtn   /**< netlink topic specific handler  */
    )
```

Description

This routine process netlink requests transmitted to the network stack via an **AF_NETLINK** socket. A response is delivered to the same socket the request is transmitted on if the call fails or if the application explicitly request a response by setting the **NLM_F_ACK** flag in the request.

Returns

Always **OK**

Errno

Not Available

See also

netlinkLib, **netlinkReqDequeue( )**, **netlinkReqEnqueue( )**

Name

netlinkLibInit( ) - initialize netlink library

Layer

RTNET

Synopsis

```
STATUS netlinkLibInit (void)
```

Description

This routine initialize the netlink library.

Returns

Always **OK**

Errno

N/A

See also

netlinkLib

37. NETLINKRTLIB

Name

netlinkRtLib - netlink route library functions

Layer

RTNET

Routines

nlRtAddrLen( ) - return length of Internet address **nlRtAddrPtr( )** - return pointer to Internet address data **nlrtBufAttrPush( )** - push a netlink route attribute **netlinkRtMsgSet( )** - set handler for a netlink route message request **netlinkRtLibInit( )** - initialize netlink route library

Description

This module implements a back-end for the **NETLINK_ROUTE** type of the **AF_NETLINK** socket family. The functions presented are the possibility to dump the current configuration of the FIB (see **fibLib**) and network interface (see **netifLib**).

It is possible to add/modify/delete entries in the FIB, add/modify/delete IP-addresses assigned to network interfaces and dump statistics about transmitted/received packets on network interfaces via this module.

The functions are accessed via the socket calls **send( )**/**recv( )** made on a socket instance created on the **AF_NETLINK** domain with the type set to **SOCK_RAW** and the protocol set to **NETLINK_ROUTE**.

This library is the infrastructure for anyting implementing the **NETLINK_ROUTE**, see **nlRtFibLib**, **nlRtNeighLib**, **nlRtAddrLib** and **nlRtSockLib** for specific functions that register the **NETLINK_ROUTE** messages they handle via **netlinkRtMsgSet( )**.

Include files

netlinkLib.h netlinkRtLib.h private/nlrtLibP.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

nlRtAddrLen( ) - return length of Internet address

Layer

RTNET

Synopsis

```
UINT32 nlrtAddrLen
    (
    UINT8 family   /* AF_INET or AF_INET 6*/
    )
```

Description

This routine return the length of addresses in the specified address family.

Returns

Length in octets.

Errno

N/A

See also

netlinkRtLib

Name

nlRtAddrPtr( ) - return pointer to Internet address data

Layer

RTNET

Synopsis

```
const void * nlrtAddrPtr
    (
    const struct in6_addr * addr   /**< an Internet address */
    )
```

Description

This routine return a pointer to the Internet address data. This network stack store IPv4 addresses as IPv4-mapped-IPv6 addresses.

Returns

Pointer to address data

Errno

N/A

See also

netlinkRtLib

Name

nlrtBufAttrPush( ) - push a netlink route attribute

Layer

RTNET

Synopsis

```
void nlrtBufAttrPush
    (
    NL_BUF_ID    buf,         /**< netlink message buffer */
    UINT16       rtaType,     /**< route attribute type */
    const void * payload,     /**< pointer to payload data */
    UINT32       payloadLen   /**< length of payload data */
    )
```

Description

This routine push a netlink route attribute, including payload, into a netlink message buffer.

Returns

N/A

Errno

N/A

See also

netlinkRtLib

Name

netlinkRtMsgSet( ) - set handler for a netlink route message request

Layer

RTNET

Synopsis

```
STATUS netlinkRtMsgSet
    (
    UINT16     rtaMsg,   /**< netlink route message ID */
    NL_REQ_RTN rtn       /**< request handler for message ID */
    )
```

Description

This routine set a handler for a single netlink route message request type.

Returns

OK if successful, or **ERROR** if out of memory.

Errno

N/A

See also

netlinkRtLib

Name

netlinkRtLibInit( ) - initialize netlink route library

Layer

RTNET

Synopsis

```
STATUS netlinkRtLibInit (void)
```

Description

This routine initialize the netlink route library for the specified network namespace, a handler for tuples (**AF_NETLINK**,**SOCK_DGRAM**/**SOCK_RAW**,**NETLINK_ROUTE**) is registered with **sysSockLib**.

Returns

OK if successful, or **ERROR** otherwise

Errno

Not Available

See also

netlinkRtLib, **sysSockLibAdd( )**

38. NETPOOLSHOW

Name

netPoolShow - network buffer library show routines

Layer

NET_BASE

Routines

netPoolShow( ) - show pool statistics **netStackDataPoolShow( )** - show network stack data pool statistics
netStackSysPoolShow( ) - show network stack system pool statistics **mbufShow( )** - report mbuf statistics

Description

This library contains routines for displaying information about network buffer pools and their usage statistics.

To use this feature, include the following component: **INCLUDE_NETPOOLSHOW**

Include files

netBufLib.h

See also

netBufLib, **netBufAdvLib**

Name

netPoolShow( ) - show pool statistics

Layer

NET_BASE

Synopsis

```
void netPoolShow
    (
    NET_POOL_ID pNetPool
    )
```

Description

This routine displays the distribution of `mBlk's and clusters in a given network pool ID.

Example

```
void endPoolShow
   (
```

```
    char * devName,     /* The interface name: "dc", "ln" ...*/
    int    unit         /* the unit number: usually 0        */
    )
    {
    END_OBJ * pEnd;

    if ((pEnd = endFindByName (devName, unit)) != NULL)
        netPoolShow (pEnd->pNetPool);
    else
        printf ("Could not find device %s\n", devName);
    return;
    }
```

Returns

N/A

Errno

See also

netPoolShow

Name

netStackDataPoolShow( ) - show network stack data pool statistics

Layer

NET_BASE

Synopsis

```
void netStackDataPoolShow (void)
```

Description

This routine displays the distribution of `mBlk's and clusters in a the network data pool. The network data pool is used only for data transfer through the network stack.

The "clusters" column indicates the total number of clusters of that size that have been allocated. The "free" column indicates the number of available clusters of that size (the total number of clusters minus those clusters that are in use). The "usage" column indicates the number of times clusters have been allocated (not, as you might expect, the number of clusters currently in use).

Returns

N/A

Errno

See also

netPoolShow, **netStackSysPoolShow( )**, **netBufLib**

Name

netStackSysPoolShow( ) - show network stack system pool statistics

Layer

NET_BASE

Synopsis

```
void netStackSysPoolShow (void)
```

Description

This routine displays the distribution of `mBlk's and clusters in a the network system pool. The network system pool is used only for system structures such as sockets, routes, interface addresses, protocol control blocks, multicast addresses, and multicast route entries.

The "clusters" column indicates the total number of clusters of that size that have been allocated. The "free" column indicates the number of available clusters of that size (the total number of clusters minus those clusters that are in use). The "usage" column indicates the number of times clusters have been allocated (not, as you might expect, the number of clusters currently in use).

Returns

N/A

Errno

See also

netPoolShow, **netStackDataPoolShow( )**, **netBufLib**

Name

mbufShow( ) - report mbuf statistics

Layer

NET_BASE

Synopsis

```
void mbufShow (void)
```

Description

This routine is a synonym for **netStackDataPoolShow( )**. It displays information about the usage of M_BLKs and clusters in the network stack system pool.

Returns

N/A

Errno

See also

netPoolShow

39. NETVIEWLIB

Name

netViewLib - network stack view library

Layer

RTNET

Routines

netViewAddr( ) - get a view of all the IP-address in the network stack **netViewFib( )** - get a view of all the FIBs in the network stack **netViewLink( )** - get a view of all attached network interfaces **netViewNeigh( )** - get a view of all ARP/NDP entries **netViewSock( )** - get a view of all open sockets **netViewLibInit( )** - initialize the network view library

Description

This module contain helper functions to present a "view" or a "snapshot" of different data-structures within the network stack. This library can be used by user-applications regardless if the RTNET stack is located in the same address space as the user-application or in a different address space, this library use an appropriate IPC mechanism for communication with the network stack.

All the **netViewXxx( )** routines take a callback and some used-defined data. The callback is called for each entry in the database-type being requested. Note that **netViewFib( )** will enumerate ALL FIB entries in the network stack, the callback must perform additional filtering like only process entries for a certain family, **AF_INET** or **AF_INET6**, or only process entries from the "normal" FIB with table ID **RT_TABLE_MAIN**.

The callback is executed in the context of the same task calling the **netViewXxxx( )** routine, no additional task is created by this library.

Include files

netViewLib.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

netViewAddr( ) - get a view of all the IP-address in the network stack

Layer

RTNET

Synopsis

```
STATUS netViewAddr
    (
    NET_VIEW_ADDR_RTN rtn,     /**< IP-address callback routine */
```

```
    void *            usrCtx   /**< user defined context data */
    )
```

Description

This routine call the user-defined routine for each IP-address assigned to all attached network interface, any filtering must be done in the user routine.

Returns

OK if the communication with the network stack is successful, or **ERROR** otherwise.

Errno

The errno from the failing call is passed through to the caller.

See also

netViewLib

Name

netViewFib( ) - get a view of all the FIBs in the network stack

Layer

RTNET

Synopsis

```
STATUS netViewFib
    (
    NET_VIEW_FIB_RTN rtn,     /**< FIB-node callback routine */
    void *           usrCtx   /**< user-defined data */
    )
```

Description

This routine call the user-defined routine for each FIB node in all defined FIBs, any filtering must be done in the user routine

Returns

OK if the communication with the network stack is successful, or **ERROR** otherwise.

Errno

The errno from the failing call is passed through to the caller.

See also

netViewLib

Name

netViewLink( ) - get a view of all attached network interfaces

Layer

RTNET

Synopsis

```
STATUS netViewLink
    (
    NET_VIEW_LINK_RTN rtn,     /**< network interface callback routine */
    void *            usrCtx   /**< user-defined data */
    )
```

Description

This routine call the user-defined routine all for every attached network interfaces, passing link address and other link-specific information. Any filtering must be done in the user routine.

Returns

OK if the communication with the network stack is successful, or **ERROR** otherwise.

Errno

The errno from the failing call is passed through to the caller.

See also

netViewLib

Name

netViewNeigh( ) - get a view of all ARP/NDP entries

Layer

RTNET

Synopsis

```
STATUS netViewNeigh
    (
    NET_VIEW_NEIGH_RTN rtn,     /**< network interface callback routine */
    void *             usrCtx   /**< user-defined data */
    )
```

Description

This routine call the user-defined routine for every neighbor node (ARP/NDP) in the network stack passing IP-address, link-address and state. Any filtering must be done in the user routine.

Returns

OK if the communication with the network stack is successful, or **ERROR** otherwise.

Errno

The errno from the failing call is passed through to the caller.

See also

netViewLib

Name

netViewSock( ) - get a view of all open sockets

Layer

RTNET

Synopsis

```
STATUS netViewSock
    (
    NET_VIEW_SOCK_RTN rtn,      /**< network interface callback routine */
    void *            usrCtx    /**< user-defined data */
    )
```

Description

This routine call the user-defined routine for all open sockets in the network stack, passing local/remote addresses/ports, socket type etc. Any filtering must be done in the user routine.

Returns

OK if the communication with the network stack is successful, or **ERROR** otherwise.

Errno

The errno from the failing call is passed through to the caller.

See also

netViewLib

Name

netViewLibInit( ) - initialize the network view library

Layer

RTNET

Synopsis

```
STATUS netViewLibInit (void)
```

Description

This routine initialize the network view library

Returns

OK if successful, or **ERROR** otherwise.

Errno

N/A

See also

netViewLib

40. NLRTADDRLIB

Name

nlrtAddrLib - netlink route IP-address library functions

Layer

RTNET

Routines

nlrtAddrLibInit( ) - initialize netlink route addrbor library

Description

This module handles the **NETLINK_ROUTE** messages **RTM_GETADDR** messages.

Include files

netlinkLib.h netlinkRtLib.h private/nlrtLibP.h

Name

nlrtAddrLibInit( ) - initialize netlink route addrbor library

Layer

RTNET

Synopsis

```
STATUS nlrtAddrLibInit (void)
```

Description

This routine register netlink route request handlers for **RTM_GETADDR** message.

Returns

OK if successful, or **ERROR** otherwise

Errno

ENOMEM

Could not allocate enough memory

See also

nlrtAddrLib

41. NLRTFIBLIB

Name

nlrtFibLib - netlink route FIB library functions

Layer

RTNET

Routines

nlrtFibLibInit( ) - initialize netlink route FIB library

Description

This module handles the **NETLINK_ROUTE** messages **RTM_NEWROUTE**, **RTM_DELROUTE** and **RTM_GETROUTE** messages.

Include files

netlinkLib.h netlinkRtLib.h private/nlrtLibP.h

Name

nlrtFibLibInit( ) - initialize netlink route FIB library

Layer

RTNET

Synopsis

```
STATUS nlrtFibLibInit (void)
```

Description

This routine register netlink route request handlers for **RTM_NEWROUTE**, **RTM_DELROUTE** and **RTM_GETROUTE** messages.

Returns

OK if successful, or **ERROR** otherwise

Errno

ENOMEM

Could not allocate enough memory

See also

nlrtFibLib

42. NLRTLINKLIB

Name

nlrtLinkLib - netlink route link library functions

Layer

RTNET

Routines

nlrtLinkLibInit( ) - initialize netlink route link library

Description

This module handles the **NETLINK_ROUTE** messages **RTM_GETLINK** messages.

Include files

netlinkLib.h netlinkRtLib.h private/nlrtLibP.h

Name

nlrtLinkLibInit( ) - initialize netlink route link library

Layer

RTNET

Synopsis

```
STATUS nlrtLinkLibInit (void)
```

Description

This routine register netlink route request handlers for **RTM_GETLINK** message.

Returns

OK if successful, or **ERROR** otherwise

Errno

ENOMEM

Could not allocate enough memory

See also

nlrtLinkLib

43. NLRTNEIGHLIB

Name

nlrtNeighLib - netlink route neighbor library functions

Layer

RTNET

Routines

nlrtNeighLibInit( ) - initialize netlink route neighbor library

Description

This module handles the **NETLINK_ROUTE** messages **RTM_GETNEIGH** messages.

Include files

netlinkLib.h netlinkRtLib.h private/nlrtLibP.h

Name

nlrtNeighLibInit( ) - initialize netlink route neighbor library

Layer

RTNET

Synopsis

```
STATUS nlrtNeighLibInit (void)
```

Description

This routine register netlink route request handlers for **RTM_GETNEIGH** message.

Returns

OK if successful, or **ERROR** otherwise

Errno

ENOMEM

Could not allocate enough memory

See also

nlrtNeighLib

44. NLRTSOCKLIB

Name

nlrtSockLib - netlink route socket library functions

Layer

RTNET

Routines

nlrtSockLibInit( ) - initialize netlink route socket library

Description

This module handles the **NETLINK_ROUTE** messages **RTM_GETSOCK** messages.

Include files

netlinkLib.h netlinkRtLib.h private/nlrtLibP.h

Name

nlrtSockLibInit( ) - initialize netlink route socket library

Layer

RTNET

Synopsis

```
STATUS nlrtSockLibInit (void)
```

Description

This routine register netlink route request handlers for **RTM_GETSOCK** message.

Returns

OK if successful, or **ERROR** otherwise

Errno

ENOMEM

Could not allocate enough memory

See also

nlrtSockLib

45. PANICLIB

Name

panicLib - **panic( )** function library

Layer

NET_BASE

Routines

panic( ) - something is very wrong

Description

This library provides the **panic( )** routine that replaces the Unix kernel function that is used in the network code.

Include files

none

Name

panic( ) - something is very wrong

Layer

NET_BASE

Synopsis

```
void panic
    (
    const char *format,
    ...
    )
```

Description

panic simply logs a message to the console prepended with "panic". If the panic occurs at task level, the current task is suspended. The length of the message must fit within a 128 byte buffer.

Returns

Not Available

Errno

Not Available

See also

panicLib

46. PEERINFOLIB

Name

peerInfoLib - peer information library file

Layer

RTNET

Routines

currentGenGet( ) - get current generation data area of a peer information cache **nextGenGet( )** - get next generation data area **peerInfoGet( )** - get peer information and generate a fragmentation ID **peerInfoPathMtuSet( )** - set path MTU for peer **peerInfoNextHopSet( )** - set next hop info from redirect packet **peerInfoInit( )** - initialize peer information cache **peerInfoCreate( )** - create peer information cache

Description

The peer information cache holds the path MTU for peers on other networks than this host belongs to. The cache is populated as the IPv4/IPv6 layer receives ICMPv4/v6 "packet too big" messages from routers on the path to the final destination. The peer information cache also generates fragmentation IDs if the packet size exceeds the path MTU.

Nodes in the peer cache holds the following information: - Destination address - Timeout handle - Fragmentation id - Path MTU

Adding nodes

The cache is implemented as a simple ring buffer. Nodes are added at an index counter which starts at zero and increments for each added node. When the index counter reaches the maximum size of the cache it is reset to zero. When a node is added an aging timer is started.

Disposing nodes

For each added node a size counter is incremented. If the counter reaches the maximum size of the cache the ring buffer is full and new nodes will dispose the oldest entry and replace the node in that entry. Because all nodes have the same lifetime and the timer is never restarted the oldest node will always be the one at the index counter. Nodes will also be disposed when the timer expires which causes the size counter to be decremented.

Mutual exclusion

The peer information cache is protected by a generation lock (see documentation of **genLib**). The generation data consists of two sorted arrays of peer IDs, which implements the current and next generation of the peer information cache. The peer ID is simply the index into the ring buffer where the node is stored and the arrays are sorted based on the node's destination address. The sorted arrays are used to apply binary search inorder to find nodes in the cache based on destination address.

Include files

peerInfoLib.h

Name

currentGenGet( ) - get current generation data area of a peer information cache

Layer

RTNET

Synopsis

```
_WRS_INLINE PEER_INFO_GEN * currentGenGet
    (
    PEER_INFO_GEN * gen,   /**< Generation data array */
    GENERATION      id     /**< Current generation id */
    )
```

Description

This routine gets the current generation data area of a peer information cache.

Returns

Generation data area.

Errno

N/A

See also

peerInfoLib

Name

nextGenGet( ) - get next generation data area

Layer

RTNET

Synopsis

```
_WRS_INLINE PEER_INFO_GEN * nextGenGet
    (
    PEER_INFO_GEN * gen,   /**< Generation data */
    GENERATION      id     /**< Current generation id */
    )
```

Description

This routine gets the next generation data area

Returns

Generation data area.

Errno

N/A

See also

peerInfoLib

Name

peerInfoGet( ) - get peer information and generate a fragmentation ID

Layer

RTNET

Synopsis

```
void peerInfoGet
    (
    PEER_INFO       peerInfo,   /**< Peer information */
    const PKT_CTX * pkt,        /**< Packet context */
    BOOL            forceId,    /**< TRUE to force new ID */
    NEXTHOP_INFO *  nhInfo      /**< Fragment ID, path MTU, next hop address */
    )
```

Description

This routine gets peer information. If the peer does not exist in the cache [pathMtu] is not changed. If the peer exists in the cache [pathMtu] is set to the stored peer's MTU, if it is lower than [pathMtu]. The routine also generates a new fragmentation ID if parameter [forceId] is **TRUE** or the payload size exceeds the MTU.

Returns

N/A

Errno

N/A

See also

peerInfoLib

Name

peerInfoPathMtuSet( ) - set path MTU for peer

Layer

RTNET

Synopsis

```
void peerInfoPathMtuSet
    (
    PEER_INFO        peerInfo,   /**< Peer information */
    const FIB_ATTR * attr,       /**< FIB attributes */
    UINT32           pathMtu,    /**< Path MTU to set */
    UINT32           minMtu      /**< Minimum path MTU to set */
    )
```

Description

This routine sets the path MTU for a peer. If the peer does not exist in the cache a new node is created. The MTU is only set if it is lower than the previously stored value and higher than the minimum allowed MTU.

Returns

N/A

Errno

N/A

See also

peerInfoLib

Name

peerInfoNextHopSet( ) - set next hop info from redirect packet

Layer

RTNET

Synopsis

```
void peerInfoNextHopSet
    (
    PEER_INFO         peerInfo,   /**< Peer information */
    FIB_ATTR *        attr,       /**< FIB attributes */
    struct in6_addr * addr        /**< next hop address */
    )
```

Description

This routine sets next hop for a peer. If the peer does not exist in the cache a new node is created.

Returns

N/A

Errno

N/A

See also

peerInfoLib

Name

peerInfoInit( ) - initialize peer information cache

Layer

RTNET

Synopsis

```
STATUS peerInfoInit
    (
    PEER_INFO peerInfo,   /**< Peer information */
    TMO_DB    tmoDb       /**< Timeout database */
    )
```

Description

This routine initializes the peer information cache. A timeout action handler is bound to each node and the generation lock is initialized.

Returns

OK, or **ERROR** if the cache could not be initialized.

ENOMEM

Not possible to bind all timeouts.

Errno

Not Available

See also

peerInfoLib

Name

peerInfoCreate( ) - create peer information cache

Layer

RTNET

Synopsis

```
PEER_INFO peerInfoCreate
    (
    UINT32 maxNodes,    /**< Max number of peer info nodes */
    UINT32 agingTime,   /**< Aging timeout in seconds */
    TMO_DB tmoDb        /**< Timeout database */
    )
```

Description

This routine creates the peer information cache. Memory is allocated for an array that holds the peer information nodes and a timeout instance is reserved for each node. The routine also creates the generation lock data which comprises two sorted arrays of peer ids. Memeory is allocated for these two arrays.

Returns

Pointer to peer information cache or **NULL** if the cache could not be created.

Errno

N/A

See also

peerInfoLib

47. PING

Name

ping - interactive command to test if a remote IP-node is alive

Layer

RTNET

Routines

pingRawMode( ) - set standard input / shell FD to raw mode **ping4( )** - interactive **ping4** command **ping6( )** - interactive **ping6** command

Description

This module transmit ICMP echo-request messages to a specified IP-node and check if and with what latency responses in the form of ICMP echo-reply is received from that IP-node.

Include files

N/A

Name

pingRawMode( ) - set standard input / shell FD to raw mode

Layer

RTNET

Synopsis

```
void pingRawMode
    (
    PING_CTX * ctx,    /**< ping session */
    BOOL       set     /**< TRUE to enter RAW mode, FALSE to restore original mode */
    )
```

Description

When set is **TRUE**, this routine sets the standard input (or shell file descriptor) to unbuffered mode with no echo, and remembers the previous settings in ctx. In the kernel, it also disables the **tyLib** abort routine.

When called with set equal to **FALSE**, this routine restores the previous standard input / shell FD settings.

Returns

N/A

Errno

N/A

See also

ping

Name

ping4( ) - interactive **ping4** command

Layer

RTNET

Synopsis

```
int ping4
    (
    const char * dstStr,    /**< IP-node to send echo-request to */
    int          count,     /**< number of packets to be sent */
    int          dataSize   /**< the number of ICMP data bytes to be sent */
    )
```

Description

This routine send ICMP echo request and print any responses and their latencies to the console.

Returns

Number of echo-replies received.

Errno

N/A

See also

ping

Name

ping6( ) - interactive **ping6** command

Layer

RTNET

Synopsis

```
int ping6
    (
    const char * dstStr,    /**< IP-node to send echo-request to */
    int          count,     /**< number of packets to be sent */
    int          dataSize   /**< the number of ICMP data bytes to be sent */
    )
```

Description

This routine send ICMPv6 echo request and print any responses and their latencies to the console.

Returns

Number of echo-replies received.

Errno

N/A

See also

ping

48. PINGLIB

Name

pingLib - Packet InterNet Grouper (PING) library

Layer

IPNET_USRSPACE

Routines

pingLibInit( ) - initialize the **ping( )** utility **ping( )** - test that a remote host is reachable

Description

This library contains the **ping( )** utility, which tests whether a remote host is reachable.

The routine **ping( )** is typically called from the VxWorks shell to check the network connection to another VxWorks target or to a UNIX host. **ping( )** may also be used programmatically by applications that require such a test. The remote host must be running TCP/IP networking code that responds to ICMP echo request packets. The **ping( )** routine is re-entrant, thus may be called by many tasks concurrently.

The routine **pingLibInit( )** initializes the **ping( )** utility and allocates resources used by this library. It is called automatically when **INCLUDE_PING** is defined.

Include files

pingLib.h

Name

pingLibInit( ) - initialize the **ping( )** utility

Layer

IPNET_USRSPACE

Synopsis

```
STATUS pingLibInit (void)
```

Description

This routine allocates resources used by the **ping( )** utility. It is called automatically when **INCLUDE_PING** is defined.

Returns

OK

Errno

N/A

See also

pingLib

Name

ping( ) - test that a remote host is reachable

Layer

IPNET_USRSPACE

Synopsis

```
STATUS ping
    (
    char *  host,         /* host to ping */
    int     numPackets,   /* number of packets to receive */
    ulong_t options       /* option flags */
    )
```

Description

This routine tests that a remote host is reachable by sending ICMP echo request packets, and waiting for replies. It may called from the VxWorks shell as follows:

```
   -> ping "remoteSystem", 1, 0
```

where remoteSystem is either a host name that has been previously added to the remote host table by a call to **hostAdd( )**, or an Internet address in dot notation (for example, "90.0.0.2").

The second parameter, numPackets, specifies the number of ICMP packets to receive from the remote host. If numPackets is 1, this routine waits for a single echo reply packet, and then prints a short message indicating whether the remote host is reachable. For all other values of numPackets, timing and sequence information is printed as echoed packets are received. If numPackets is 0, this routine runs continuously.

If no replies are received within a 5-second timeout period, the routine exits. An **ERROR** status is returned if no echo replies are received from the remote host.

The following flags may be given through the options parameter:

PING_OPT_SILENT

Suppress output. This option is useful for applications that use **ping( )** programmatically to examine the return status.

PING_OPT_DONTROUTE

Do not route packets past the local network. This also prevents pinging local addresses (i.e. the IP address of the host itself). The 127.x.x.x addresses will still work however.

PING_OPT_NOHOST

Suppress host lookup. This is useful when you have the DNS resolver but the DNS server is down and not returning host names. When **ping( )** receives a response it tries to convert the IP address of the peer to its hostname so that it can be displayed. If the entry is not available in the local host table, then the DNS server will be contacted. If the server does not respond the first time, the resolver makes multiple attempts to contact the server and then times out if no response is received. By default it makes a maximum of 2 attempts with a starting timeout of 5 seconds which results in a total

timeout period of 15 seconds. If the number of attempts is increased to 4, this timeout can extend upto 75 seconds (because of exponential backoff between retries). During this time it may appear that the ping call has stalled and it will not return. In that case wait a few minutes to see if the call returns. Sometimes the DNS server information specified to the resolver is incorrect, especially if the default settings were not changed at all when the DNS resolver component was included. Check the settings and confirm whether the server is reachable or not.

PING_OPT_DEBUG

Enables debug output.

The following global variables can be set from the target shell or Host based Debug Shell to configure the **ping( )** parameters:

_pingTxLen

Size of the ICMP echo packet (default 64, min 8, max 65516).

_pingSockBufSize

Size of the socket buffer (default 8192).

_pingTxInterval

Packet interval in seconds (default 1 second).

_pingTxTmo

Packet timeout in seconds (default 5 seconds).

Note

If you have included the DNS resolver, not specified the **PING_OPT_NOHOST** option and you notice that **ping( )** is taking a long time (more than the timeout, _pingTxTmo) to return, check your resolver settings to see if the server has been specified correctly. It is also possible that the server is specified correctly but it is down. See the description for the **PING_OPT_NOHOST** option if you see this problem.

Returns

OK, or **ERROR** if the remote host is not reachable.

Errno

EINVAL

numPackets is invalid

S_pingLib_TIMEOUT

Timed out without a response

See also

pingLib

49. PKTCTXLIB

Name

pktCtxLib - network packet context library header file

Layer

RTNET

Routines

pktCtxInit( ) - initialize a packet context **pktCtxNetnsGet( )** - get the network namespace associated with a packet **pktCtxAttrGet( )** - get the IP-protocol attributes associated with a packet **pktCtxAttrConstGet( )** - get the IP-protocol attributes associated with a packet **pktCtxPush( )** - add data to a packet context **pktCtxPop( )** - pop data from a packet context **pktCtxMove( )** - move buffer between packet contexts **pktCtxLenGet( )** - get total length of packet data **pktCtxLenSet( )** - set total data length of a packet **pktCtxIov( )** - get the packet data buffer as an I/O-vector **pktCtxIovLen( )** - get the length of the I/O-vector returned by **pktCtxIov( )** **pktCtxIpHdrGet( )** - get the aligned copied IP-header **pktCtxIpHdrSet( )** - set IP-header pointer **pktCtxIpBaseGet( )** - get IP-header base pointer **pktCtxIpBaseSet( )** - set IP-header base pointer **pktCtxIpBaseReset( )** - reset packet to IP-datagram **pktCtxIpOffset( )** - return offset from IP-header to a higher layer member

Description

Real-time network stack packet manipulation routines.

Include files

N/A

Name

pktCtxInit( ) - initialize a packet context

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxInit
    (
    PKT_CTX * ctx,    /**< packet context */
    NETNS *   netns   /**< network namespace for the packet */
    )
```

Description

This routine initialize a packet context and bind it to a specific network namespace.

Returns

N/A

Errno

Not Available

See also

pktCtxLib

Name

pktCtxNetnsGet( ) - get the network namespace associated with a packet

Layer

RTNET

Synopsis

```
_WRS_INLINE NETNS * pktCtxNetnsGet
    (
    PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine get the network namespace associated with a packet

Returns

Pointer to a network namespace instance.

Errno

N/A

See also

pktCtxLib

Name

pktCtxAttrGet( ) - get the IP-protocol attributes associated with a packet

Layer

RTNET

Synopsis

```
_WRS_INLINE FIB_ATTR * pktCtxAttrGet
    (
```

```
    PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine get the IP-protocol attributes, or meta-data, associated with a packet. The information include information like IP-addresses, network interface ID and next-hop IP-address. The "source" and "destiation" IP-address information is always stored as-if the packet is transmitted by the local node, that mean that even ingress packets will store the address of the remote IP-node as destination IP (returned by **fibAttrDstGet( )**).

Returns

pointer to packet attributes

Errno

N/A

See also

pktCtxLib

Name

pktCtxAttrConstGet( ) - get the IP-protocol attributes associated with a packet

Layer

RTNET

Synopsis

```
_WRS_INLINE const FIB_ATTR * pktCtxAttrConstGet
    (
    const PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine does the same thing as **pktCtxAttrGet( )** except that the attribute pointer returned may only be passed to routines that read attributes, not write them.

Returns

pointer to read-only packet attributes

Errno

N/A

See also

pktCtxLib

Name

pktCtxPush( ) - add data to a packet context

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxPush
    (
    PKT_CTX * ctx,      /**< packet context */
    void *    data,     /**< data to push to the packet, must be valid as */
                        /* long as the packet context is valid */
    UINT32    dataLen   /**< length of the data */
    )
```

Description

This routine add a data-segment to a packet context, the data is added in front of any existing data.

Returns

N/A

Errno

Not Available

See also

pktCtxLib

Name

pktCtxPop( ) - pop data from a packet context

Layer

RTNET

Synopsis

```
_WRS_INLINE const void * pktCtxPop
    (
    PKT_CTX * ctx,      /**< packet context */
    UINT32    dataLen   /**< number of bytes to pop from the packet */
    )
```

Description

This routine move the pointer to the beginning of the packet data and return a pointer to the removed data. The returned data is read-only as the same underlying data buffer might be shared by multiple packet contexts.

Returns

read-only pointer to the data popped of the packet

Errno

N/A

See also

pktCtxLib

Name

pktCtxMove( ) - move buffer between packet contexts

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxMove
    (
    PKT_CTX * to,
    PKT_CTX * from,
    UINT32    dataLen
    )
```

Description

This routine move buffer content from one buffer to another

Returns

N/A

Errno

N/A

See also

pktCtxLib

Name

pktCtxLenGet( ) - get total length of packet data

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 pktCtxLenGet
    (
    const PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine get the sum of the length of all data segments in the packet

Returns

total packet data length

Errno

N/A

See also

pktCtxLib

Name

pktCtxLenSet( ) - set total data length of a packet

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxLenSet
    (
    PKT_CTX * ctx,     /**< packet context */
    UINT32    newLen   /**< new packet length */
    )
```

Description

This routine set the total data length of a packet by trim bytes at the end of the data buffer. It is not allowed/valid to extend the data length through this function, use **pktCtxPush( )** to add data.

Returns

N/A

Errno

N/A

See also

pktCtxLib

Name

pktCtxIov( ) - get the packet data buffer as an I/O-vector

Layer

RTNET

Synopsis

```
_WRS_INLINE const struct iovec * pktCtxIov
    (
    const PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine return an I/O-vector representation of the packet data contained within in a packet context. The data returned may only be read.

Returns

packet data as an read-only I/O-vector

Errno

N/A

See also

pktCtxLib

Name

pktCtxIovLen( ) - get the length of the I/O-vector returned by **pktCtxIov( )**

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 pktCtxIovLen
    (
```

```
    const PKT_CTX * ctx    /**< packet context */
    )
```

Description

This routine return the length of the I/O-vector returned by **pktCtxIov( )**.

Returns

I/O-vector length.

Errno

N/A

See also

pktCtxLib

Name

pktCtxIpHdrGet( ) - get the aligned copied IP-header

Layer

RTNET

Synopsis

```
_WRS_INLINE const void * pktCtxIpHdrGet
    (
    const PKT_CTX * ctx    /**< packet context */
    )
```

Description

This routine get a pointer to the copied IP-header of the packet.

Returns

Pointer to properly aligned copy of the IP-header which is stored in
stack instead of heap, or **NULL** if the packet does not have an IP-header.

Errno

N/A

See also

pktCtxLib

Name

pktCtxIpHdrSet( ) - set IP-header pointer

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxIpHdrSet
    (
    PKT_CTX * ctx,     /**< packet context */
    void *    ipHdr    /**< pointer to IP-header, must be located on correct */
                       /*   alignment */
    )
```

Description

This routine set a stack pointer to the copied IP-header of the packet. This differs from **pktCtxIpBaseSet( )** in that the pointer passed as argument here must be properly aligned.

Returns

N/A

Errno

N/A

See also

pktCtxLib

Name

pktCtxIpBaseGet( ) - get IP-header base pointer

Layer

RTNET

Synopsis

```
_WRS_INLINE const void * pktCtxIpBaseGet
    (
    const PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine get a pointer to where the IP-header is stored in the packet, IP-header base pointer is only set for ingress packets where the buffer is contiguous.

Returns

Pointer to IP-headr, or **NULL** if the packet does not have an IP-header.

Errno

N/A

See also

pktCtxLib

Name

pktCtxIpBaseSet( ) - set IP-header base pointer

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxIpBaseSet
    (
    PKT_CTX * ctx,      /**< packet context */
    void *    ipBase    /**< pointer to IP-header, must be located on the */
                        /*   contiguous pkt buffer */
    )
```

Description

This routine set a pointer to the IP-header of the packet. This differs from **pktCtxIpHdrSet( )** in that the base pointer must be returned from a call to **pktCtxPop( )**.

Returns

N/A

Errno

N/A

See also

pktCtxLib

Name

pktCtxIpBaseReset( ) - reset packet to IP-datagram

Layer

RTNET

Synopsis

```
_WRS_INLINE void pktCtxIpBaseReset
    (
    PKT_CTX * ctx   /**< packet context */
    )
```

Description

This routine reset the packet context to make the IP header be the topmost header again.

Returns

N/A

Errno

N/A

See also

pktCtxLib

Name

pktCtxIpOffset( ) - return offset from IP-header to a higher layer member

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 pktCtxIpOffset
    (
    PKT_CTX * ctx,   /**< packet context */
    void *    ptr    /**< pointer into a buffer returned by pktCtxPop() */
                     /*   for the same 'ctx' as the first argument */
    )
```

Description

This routine returns the offset in octets from the IP-header set as base in **pktCtxIpBaseSet( )** and a pointer into something returned by **pktCtxPop( )** for the same packet context used as first to this routine.

Returns

Offset in octets.

Errno

N/A

See also

pktCtxLib

50. REASSEMBLYLIB

Name

reassemblyLib - IP-datagram reassembly library

Layer

RTNET

Routines

reassemblyDbCreate( ) - create a reassembly database **reassemblyDbInit( )** - initialize a reassembly database **reassemblyInfoInit( )** - initialize fragment information **reassemblyInfoMfSet( )** - mark this as not the last fragment **reassemblyRx( )** - try reassemble IP-datagram **reassemblyRxDone( )** - finish using a reassembled IP-datagram **reassemblyTmoHookSet( )** - set hook function for reassembly timeout action

Description

Reassembles IP-datagram fragments, generic implementation that support both IPv4 and IPv6 protocols.

Fragmenents classified based on the 4-tuple IP-payload type (IP-protocol), source IP-address, destination IP-address and fragmentation ID. Up to the capacity passed to **reassemblyDbCreate( )** individual reassembly chains may be active concurrently. If all reassembly chains are in use, then the oldest chain is reused for any new 4-tuple received.

The granularity for any fragmentation offset is always assumed to be 8 bytes (64-bits).

Synchronization

Each reassembly database instance has a lock for mutual exclusion, this lock must always be held by the task using a reassembled IP-datagram returned by **reassemblyRx( )**. All resources, including the lock, is released by a task by the means of the **reassemblyRxDone( )** routine.

Example

```
void mayInputReassembledDgram
    (
    PKT_CXT * frag
    )
    {
    REASS_OFST fragOffset;       /* data offset in fragment */
    UINT32     fragId;           /* fragmentation ID */
    UINT8      ipProto;          /* upper layer protocol */
    BOOL       moreFragments;    /* whether there is more fragments */
    REASS_INFO nfo;
    REASS_CTX  rCtx;
    PKT_CTX    pkt;

    /*
     @ Fill fragOffset, fragId, ipProto and moreFragments (protocol
     @ specific)
     */

    reassemblyInfoInit (&nfo, frag, ipProto, fragId, fragOffset);
```

```
    if (moreFragments)
        {
        reassemblyInfoMfSet (&nfo);
        }

    rCtx = reassemblyRx (netnsReassDbGet (pkt), &pkt, &nfo, IS_IPV4, NULL);
    if (rCtx != REASS_CTX_NONE)
        {
        /*
         @ Deliver the reassembled datagram contained in 'pkt' to
         @ higher level handler.
         */
         higherLevelHandler (pkt);

        /*
         @ This release the reassembly chain and any other resources
         @ associated by 'pkt', which means that 'pkt' must not be
         @ used after this call.
         */
        reassemblyRxDone (rCtx);
        }
    }
```

Include files

reassemblyLib.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

reassemblyDbCreate( ) - create a reassembly database

Layer

RTNET

Synopsis

```
REASS_DB reassemblyDbCreate
    (
    UINT32 capacity,      /**< maximum number of concurrent reassembly chains */
    UINT32 lifetimeSec,   /**< time before reassembly time out */
    TMO_DB tmoDb          /**< timeout database */
    )
```

Description

This routine create a new reassembly database with a specific capacity with respect to maximum number of IP-datagrams that can be reassembled concurrently

Returns

The new reassembly database, or **NULL** if not enough memory could be allocated

Errno

N/A

See also

reassemblyLib

Name

reassemblyDbInit( ) - initialize a reassembly database

Layer

RTNET

Synopsis

```
STATUS reassemblyDbInit
    (
    REASS_DB db
    )
```

Description

This routine initialize a reassambly database

Returns

OK if successful, or **ERROR** otherwise

Errno

N/A

See also

reassemblyLib

Name

reassemblyInfoInit( ) - initialize fragment information

Layer

RTNET

Synopsis

```
void reassemblyInfoInit
    (
    REASS_INFO * info,      /**< fragment information */
    PKT_CTX *    frag,      /**< fragment */
    UINT8        ipProto,   /**< IP-protocol */
    UINT32       id,        /**< fragment ID */
    UINT32       offset     /**< fragment offset */
    )
```

Description

This routine initialize fragment information

Returns

N/A

Errno

N/A

See also

reassemblyLib

Name

reassemblyInfoMfSet( ) - mark this as not the last fragment

Layer

RTNET

Synopsis

```
void reassemblyInfoMfSet
    (
    REASS_INFO * info   /**< fragment information */
    )
```

Description

This routine set the fragment information to say that there are more fragments after the current fragment.

Returns

N/A

Errno

N/A

See also

reassemblyLib

Name

reassemblyRx( ) - try reassemble IP-datagram

Layer

RTNET

Synopsis

```
REASS_CTX reassemblyRx
    (
    REASS_DB                  db,            /**< reassembly database */
    PKT_CTX *                 ipDgram,       /**< will contain the reassembled IP-datagram */
    REASS_INFO *              info,          /**< fragment information */
    IP_VERSION                ipVer,         /**< IS_IPV4 or IS_IPV6 */
    REASSEMBLY_IPHDR_SET_HOOK ipHdrSetHook   /**< ipHdr set hook */
    )
```

Description

This routine add the fragment to any previously received fragments for the same IP-datagram, the reassembled datagram is returned if possible.

Returns

Contex pointer, or **REASS_CTX_NONE** if more fragments are required.

Errno

Not Available

See also

reassemblyLib, **reassemblyRxDone( )**

Name

reassemblyRxDone( ) - finish using a reassembled IP-datagram

Layer

RTNET

Synopsis

```
void reassemblyRxDone
    (
    REASS_CTX ctx   /**< context handle returned by */
```

```
                    /* reassemblyRx() */
    )
```

Description

This routine release resources associated with a reassembled IP-datagram return by **reassemblyRx( )**

Returns

N/A

Errno

Not Available

See also

reassemblyLib, **reassemblyRx( )**

Name

reassemblyTmoHookSet( ) - set hook function for reassembly timeout action

Layer

RTNET

Synopsis

```
void reassemblyTmoHookSet
    (
    REASSEMBLY_TMO_HOOK tmoHook,   /**< reassembly timeout hook */
    BOOL                isV4       /**< TRUE if IPv4, FALSE if IPv6 */
    )
```

Description

This routine will set hook function for reassembly timeout action.

Returns

N/A

Errno

Not Available

See also

reassemblyLib, **reassemblyTmo( )**

51. REMLIB

Name

remLib - remote command library

Layer

IPNET_USRSPACE

Routines

remLibInit( ) - initialize **remLib** **rcmd_af( )** - execute a shell command on a remote machine **rcmd( )** - execute a shell command on a remote machine **rresvport( )** - open a socket with a privileged port bound to it **rresvport_af( )** - open a socket with a privileged code bound to it **remCurIdGet( )** - get the current remote user name and password **setlogin( )** - set the remote username for use in this RTP **remUserNameSet( )** - set the remote username for use in this RTP **remUserNameGet( )** - get the remote username for use in this RTP **setpassword( )** - set the remote user password for use in this RTP **getpassword_r( )** - get the remote user password for use in this RTP **bindresvport( )** - bind a socket to a privileged IP port **bindresvport_af( )** - bind a socket to a privileged IP port

Description

This library provides routines that support remote command functions. The **rcmd( )** and **rresvport( )** routines use protocols implemented in BSD 4.3; they support remote command execution, and the opening of a socket with a bound privileged port, respectively. **rcmd_af( )** and **rresvport_af( )** can be used for IPv4 as well as IPv6. For more information, see <Unix Network Programming> by W. Richard Stevens. This library also includes routines that authorize network file access via **netDrv**.

All the functions managing user information in this library are strictly about providing VxWorks with information regarding remote users (i.e. users registered on a remote system). None of them apply to VxWorks' native users.

Configuration

To include **remLib** in a VxWorks image, include the **INCLUDE_REMLIB** configuration component. This component contains one parameter, **RSH_STDERR_SETUP_TIMEOUT**. Use this parameter to specify how long an **rcmd( )** or **rcmd_af( )** call should wait for a return from its internal call to **select( )**. Valid values for **RSH_STDERR_SETUP_TIMEOUT** are 0 (**NO_WAIT**), -1 (**WAIT_FOREVER**), or a positive integer from 1 to 2147483647 inclusive. This positive integer specifies the wait in seconds. The default value for **RSH_STDERR_SETUP_TIMEOUT** is -1 (**WAIT_FOREVER**).

Include files

remLib.h

See also

inetLib, "VxWorks 7 Network Stack Programmer's Guide"

Name

remLibInit( ) - initialize **remLib**

Layer

IPNET_USRSPACE

Synopsis

```
void remLibInit
    (
    long timeout   /* RSH_STDERR_SETUP_TIMEOUT */
    )
```

Description

This routine initializes the remote command library by setting the setup timeout value.

In the kernel, the timeout value can be changed using the **RSH_STDERR_SETUP_TIMEOUT** parameter associated with the **INCLUDE_REMLIB** configuration component.

When the remote command library is required in an RTP, this routine must be called when the RTP is initialized. The timeout value is specified when this routine is invoked.

Parameters:

timeout

The setup timeout value.

Returns

N/A

Errno

N/A

See also

remLib

Name

rcmd_af( ) - execute a shell command on a remote machine

Layer

IPNET_USRSPACE

Synopsis

```
int rcmd_af
    (
    char *host,          /* host name or inet address */
    int  remotePort,     /* remote port to connect to (rshd) */
    char *localUser,     /* local user name */
    char *remoteUser,    /* remote user name */
    char *cmd,           /* command */
    int  *fd2p,          /* if this pointer is non-zero, */
                         /* stderr socket is opened and */
```

```
                        /* socket descriptor is filled in */
    int  family         /* address family */
    )
```

Description

This routine uses a remote shell daemon, **rshd**, to execute a command on a remote system. It is analogous to the BSD **rcmd_af( )** routine.

Internally, this **rcmd_af( )** implementation uses a **select( )** call to wait for a response from the **rshd** daemon. If **rcmd_af( )** receives a response within its timeout, **rcmd_af( )** calls **accept( )** and completes by returning a socket descriptor for the data generated on the remote machine.

The default timeout lets the **rcmd_af( )** call wait forever. However, you can change the timeout value using the **RSH_STDERR_SETUP_TIMEOUT** parameter associated with the **INCLUDE_REMLIB** configuration component.

Parameters:

host

The name or address of the host to be contacted.

remotePort

The port number on the remote host.

localUser

The name of the user on the local machine.

remoteUser

The name of the user on the remote machine.

cmd

The command to be executed on the remote machine.

fd2p

if this pointer is non-zero, stderr socket is opened and socket descriptor is filled in

family

The address family to use. This can be **AF_INET**, **AF_INET6** or **AF_UNSPEC**.

Returns

A socket descriptor if the remote shell daemon accepts, or **ERROR** if the remote command fails.

Errno

S_remLib_RSH_ERROR

error received from rsh

S_remLib_RSH_STDERR_SETUP_FAILED

setup failed

S_remLib_RCMD_INPUT_ERROR

invalid input was received for **rcmd_af( )**

See also

remLib

Name

rcmd( ) - execute a shell command on a remote machine

Layer

IPNET_USRSPACE

Synopsis

```
int rcmd
    (
    char *host,          /* host name or inet address */
    int  remotePort,     /* remote port to connect to (rshd) */
    char *localUser,     /* local user name */
    char *remoteUser,    /* remote user name */
    char *cmd,           /* command */
    int  *fd2p           /* if this pointer is non-zero, */
                         /* stderr socket is opened and */
                         /* socket descriptor is filled in */
    )
```

Description

This routine uses a remote shell daemon, **rshd**, to execute a command on a remote system. It is analogous to the BSD **rcmd( )** routine.

Internally, this **rcmd( )** implementation uses a **select( )** call to wait for a response from the **rshd** daemon. If **rcmd( )** receives a response within its timeout, **rcmd( )** calls **accept( )** and completes by returning a socket descriptor for the data generated on the remote machine.

The default timeout lets the **rcmd( )** call wait forever. However, you can change the timeout value using the **RSH_STDERR_SETUP_TIMEOUT** parameter associated with the **INCLUDE_REMLIB** configuration component.

Parameters:

host

The name or address of the host to be contacted.

remotePort

The port number on the remote host.

localUser

The name of the user on the local machine.

remoteUser

The name of the user on the remote machine.

cmd

The command to be executed on the remote machine.

fd2p

if this pointer is non-zero, stderr socket is opened and socket descriptor is filled in

Returns

A socket descriptor if the remote shell daemon accepts, or **ERROR** if the remote command fails.

Errno

S_remLib_RSH_ERROR

error received from rsh

S_remLib_RSH_STDERR_SETUP_FAILED

setup failed

S_remLib_RCMD_INPUT_ERROR

invalid input was received for rcmd

See also

remLib, BSD reference entry for **rcmd( )**

Name

rresvport( ) - open a socket with a privileged port bound to it

Layer

IPNET_USRSPACE

Synopsis

```
int rresvport
    (
    FAST int *alport   /* port number to initially try */
    )
```

Description

This function will reserve and IPv4 port. Use rresvport_af to specify a specific address family.

Parameters:

alport

The port number to try initially

Returns

A socket descriptor, or **ERROR** if either the socket cannot be opened or all ports are in use.

Errno

N/A

See also

remLib, **rresvport_af( )**, UNIX BSD 4.3 manual entry for **rresvport( )**

Name

rresvport_af( ) - open a socket with a privileged code bound to it

Layer

IPNET_USRSPACE

Synopsis

```
int rresvport_af
    (
    FAST int *alport,   /* port number to initially try on */
    int      family     /* address family */
    )
```

Description

This routine opens a socket with a privileged port bound to it. It is analogous to the UNIX routine **rresvport_af( )**. It is a common piece of code that is executed for IPv4 and IPv6. The family argument specifies **AF_INET** or **AF_INET6**.

Parameters:

alport

The port number to try initially.

family

The address family, either **AF_INET** for IPv4 or **AF_INET6** for IPv6

Returns

A socket descriptor, or **ERROR** if either the socket cannot be opened or all ports are in use.

Errno

N/A

See also

remLib, **rresvport( )**, UNIX BSD 4.3 manual entry for **rresvport_af( )**

Name

remCurIdGet( ) - get the current remote user name and password

Layer

IPNET_USRSPACE

Synopsis

```
void remCurIdGet
    (
    char *user,    /* where to return current remote user's name */
    char *passwd   /* where to return current remote user's password */
    )
```

Description

This routine gets the user name and password currently used for remote host access privileges and copies them to user and passwd. Either parameter can be initialized to **NULL**, and the corresponding item will not be passed.

Parameters:

user

Will be filled in with the current remote user's name.

passwd

Will be filled in with the current remote user's password.

Returns

N/A

Errno

N/A

See also

remLib, **iam( )**, **whoami( )**, **remUserNameGet( )**, **remUserNameSet( )**, **getpassword_r( )**, **setpassword( )**

Name

setlogin( ) - set the remote username for use in this RTP

Layer

IPNET_USRSPACE

Synopsis

```
STATUS setlogin
    (
    char * login
    )
```

Description

This routine sets the remote username which is associated with the current RTP.

Note: this routine is obsolete and will be removed in future versions of the product. Use **remUserNameSet( )** instead.

Returns

OK, or **ERROR** if username is not valid

Errno

N/A

See also

remLib

Name

remUserNameSet( ) - set the remote username for use in this RTP

Layer

IPNET_USRSPACE

Synopsis

```
STATUS remUserNameSet
    (
    char * userName   /* user name on remote system */
    )
```

Description

This routine sets the remote username which is associated with the current RTP.

Returns

OK, or **ERROR** if username is not valid

Errno

N/A

See also

remLib

Name

remUserNameGet( ) - get the remote username for use in this RTP

Layer

IPNET_USRSPACE

Synopsis

```
STATUS remUserNameGet
    (
    char * userName,   /* where to store the remote user's name */
    size_t length      /* buffer for remote user's name */
    )
```

Description

This routine gets the remote username which is associated with the current RTP. The length of the provided buffer is passed as a parameter. It is updated with the actual length of the returned userid.

Returns

OK if the username is returned. **ERROR** if the provided buffer was not long enough to hold the username.

Errno

N/A

See also

remLib

Name

setpassword( ) - set the remote user password for use in this RTP

Layer

IPNET_USRSPACE

Synopsis

```
STATUS setpassword
    (
    char * passwd   /* remote user's password to use with RTP */
    )
```

Description

This routine sets the password to be used for a remote user which is associated with the current RTP.

Returns

N/A

Errno

N/A

See also

remLib

Name

getpassword_r( ) - get the remote user password for use in this RTP

Layer

IPNET_USRSPACE

Synopsis

```
STATUS getpassword_r
    (
    char * passwd,   /* where to store the remote user's password used with RTP */
    size_t length    /* size of buffer for the password */
    )
```

Description

This routine gets the current password of the remote user which is associated with the current RTP. The length of the provided buffer is passed as a parameter.

Returns

N/A

Errno

N/A

See also

remLib

Name

bindresvport( ) - bind a socket to a privileged IP port

Layer

IPNET_USRSPACE

Synopsis

```
STATUS bindresvport
    (
    int                     sd,    /* socket to be bound */
    FAST struct sockaddr_in *sin   /* socket address -- value/result */
    )
```

Description

This routine picks a port number between **MIN_RESV_PORT** and **MAX_RESV_PORT** that is not being used by any other programs and binds the socket passed as sd to that port. Privileged IP ports (numbers between and including 1 and 1023) are reserved for privileged programs.

Parameters:

sd

The socket to be bound.

sin

The socket address. This value is returned by the routine.

Returns

OK, or **ERROR** if the address family specified in sin is not supported or the call fails.

Errno

N/A

See also

remLib

Name

bindresvport_af( ) - bind a socket to a privileged IP port

Layer

IPNET_USRSPACE

Synopsis

```
STATUS bindresvport_af
    (
    int                     sd,      /* socket to be bound */
    FAST struct sockaddr * sin,      /* socket address */
```

```
    int                     family   /* AF_INET, AF_INET6 */
    )
```

Description

This routine picks a port number between **MIN_RESV_PORT** and **MAX_RESV_PORT** that is not being used by any other programs and binds the socket passed as sd to that port. Privileged IP ports (numbers between and including 1 and 1023) are reserved for privileged programs. This routine supports both IPv4 and IPv6 addresses. The family argument must be either **AF_INET** or **AF_INET6**.

Parameters:

sd

The socket to be bound.

sin

The socket address. This value is returned by the routine.

family

The address family, either **AF_INET** for IPv4 or **AF_INET6** for IPv6

Returns

OK, or **ERROR** if the address family specified in sin is not supported or the call fails.

Errno

N/A

See also

remLib

52. RLOGLIB

Name

rlogLib - remote login library

Layer

IPNET_USRSPACE

Routines

rlogInit( ) - initialize the remote login facility **rlogind( )** - the VxWorks remote login daemon **rlogin( )** - log in to a remote host

Description

This library provides a remote login facility for VxWorks based on the UNIX **rlogin** protocol (as implemented in UNIX BSD 4.3). On a VxWorks terminal, this command gives users the ability to log in to remote systems on the network.

Reciprocally, the remote login daemon, **rlogind( )**, allows remote users to log in to VxWorks. The daemon is started by calling **rlogInit( )**, which is called automatically when **INCLUDE_RLOGIN** is defined. The remote login daemon accepts remote login requests from another VxWorks or UNIX system, and causes the shell's input and output to be redirected to the remote user.

Internally, **rlogind( )** provides a tty-like interface to the remote user through the use of the VxWorks pseudo-terminal driver **ptyDrv**.

Include files

rlogLib.h

See also

ptyDrv, **telnetLib**, UNIX BSD 4.3 manual entries for **rlogin**, **rlogind**, and **pty**

Name

rlogInit( ) - initialize the remote login facility

Layer

IPNET_USRSPACE

Synopsis

```
STATUS rlogInit (void)
```

Description

This routine initializes the remote login facility in the kernel. It creates a pty (pseudo tty) device and spawns **rlogind( )**. If **INCLUDE_RLOGIN** is included, **rlogInit( )** is called automatically at boot time.

Returns

OK or **ERROR**.

Errno

Not Available

See also

rlogLib, **ptyDrv**

Name

rlogind( ) - the VxWorks remote login daemon

Layer

IPNET_USRSPACE

Synopsis

```
void rlogind (void)
```

Description

This routine provides a facility for remote users to log in to VxWorks over the network. If **INCLUDE_RLOGIN** is defined, **rlogind( )** is spawned by **rlogInit( )** at boot time.

Remote login requests will cause **stdin**, **stdout**, and **stderr** to be directed away from the console. When the remote user disconnects, **stdin**, **stdout**, and **stderr** are restored, and the shell is restarted. The **rlogind( )** routine uses the remote user verification protocol specified by the UNIX remote shell daemon documentation, but ignores all the information except the user name, which is used to set the VxWorks remote identity (see the manual entry for **iam( )**).

The remote login daemon requires the existence of a pseudo-terminal device, which is created by **rlogInit( )** before **rlogind( )** is spawned. The **rlogind( )** routine creates two child processes, **tRlogInTask** and **tRlogOutTask**, whenever a remote user is logged in. These processes exit when the remote connection is terminated.

Returns

N/A

Errno

Not Available

See also

rlogLib, **rlogInit( )**, **iam( )**

Name

rlogin( ) - log in to a remote host

Layer

IPNET_USRSPACE

Synopsis

```
STATUS rlogin
    (
    char *host   /* name of host to connect to */
    )
```

Description

This routine allows users to log in to a remote host. In the kernel it may be called from the VxWorks shell as follows:

```
-> rlogin "remoteSystem"
```

where remoteSystem is either a host name, which has been previously added to the remote host table by a call to **hostAdd( )**, or an Internet address in dot notation (e.g., "90.0.0.2"). The remote system will be logged into with the current user name as set by a call to **iam( )**.

If a different userid is needed to connect to remote host then, **remCurIdSet( )** should be used prior to rlogin call.

```
-> remCurIdSet "user id"
```

In user space, an application can start an rlogin session through a program This can be done as follows:

```
remLibInit (WAIT_FOREVER);
rlogin ("remote hostname or IP address");
```

If a different userid is needed to connect to remote host then, **setlogin( )** should be called prior to **rlogin( )** call. This will set user id for the RTP.

```
setlogin ("user id");
```

The user disconnects from the remote system by typing:

```
~.
```

as the only characters on the line, or by simply logging out from the remote system using **logout( )**.

Warning

This routine is callable only from one task context (i.e., from one shell) at a time. It is, however, not protected against multiple concurrent invocations. Use with caution.

Returns

OK, or **ERROR** if the host is unknown, no privileged ports are available, the routine is unable to connect to the host, or the child process cannot be spawned.

Errno

Not Available

See also

rlogLib, **iam( )**, **logout( )**, **remCurIdSet( )**, **setlogin( )**

53. SOCKLIBSCTPEXT

Name

sockLibSctpExt - generic user-side socket library for sctp

Layer

IPNET_USRSPACE

Routines

sctp_bindx( ) - bind a specific subset of addresses **sctp_connectx( )** - initiate a association with a specific subset of addresses **sctp_send( )** - send SCTP data to a socket **sctp_sendx( )** - send SCTP data to a socket **sctp_sendmsg( )** - send a SCTP message to a socket **sctp_recvmsg( )** - receive a SCTP message from a socket **sctp_getaddrlen( )** - get the length specified family's socket address structure **sctp_getpaddrs( )** - get all peer addresses in an association **sctp_freepaddrs( )** - frees all resources allocated by **sctp_getpaddrs( ) sctp_getladdrs( )** - get all locally bound address(es) on a socket **sctp_freeladdrs( )** - frees all resources allocated by **sctp_getladdrs( ) sctp_opt_info( )** - get SCTP socket options information

Description

This library provides UNIX BSD 4.4 compatible socket calls. Use these calls to open, close, read, and write sockets. These sockets can join processes on the same CPU or on different CPUs between which there is a network connection. The calling sequences of these routines are identical to their equivalents under UNIX BSD 4.4. Except for **connectWithTimeout( )**, all the API are available as system calls.

Configuration

This library requires the **INCLUDE_SC_SOCKLIB** component to be configured into the kernel; errno will be set to **ENOSYS** if this component is not present.

Include files

socket.h, **socketvar.h**

See also

netLib, VxWorks 7 Sockets Programmer's Guide

Name

sctp_bindx( ) - bind a specific subset of addresses

Layer

IPNET_USRSPACE

Synopsis

```
int sctp_bindx
    (
    int              fd,         /* the socket descriptor */
    struct sockaddr *addrs,      /* the socket bind address list */
```

```
    int             addrcnt,   /* the socket bind address count */
    int             flags      /* the flags parameter is formed from the bitwise */
    )
```

Description

This function allows the user to bind a specific subset of addresses or, if the SCTP extension described in [RFC5061] is supported, add or delete specific addresses. If sd is an IPv4 socket, the addresses passed must be IPv4 addresses. If the sd is an IPv6 socket, the addresses passed can either be IPv4 or IPv6 addresses.

Parameters

fd

Specifies the file descriptor for the socket.

*addrs

Specifies the socket bind address list.

addrcnt

Specifies the socket bind address count.

flags

Specifies flags for the socket, formed bitwise. Available bindx flags include:

SCTP_BINDX_ADD_ADDR

Specifies SCTP to add the given addresses to the association.

SCTP_BINDX_REM_ADDR

Specifies SCTP to remove the given addresses from the association.

Example

The following example demonstrates the use of the **sctp_bindx( )** function:

```
int create_socket (int family, int type, int proto)
{
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

int do_bindx (int fd)
{

#define       ADDRS_COUNT     2

#define       PORT    1234
```

```
      struct sockaddr_in addrs[ADDRS_COUNT];
      int flags;
      int ret;

      /* the addr specifies the address family , IP address and port
       for the socket that is being bound */
      memset (addrs, 0, ADDRS_COUNT*sizeof(struct sockaddr_in));
      addrs[0].sin_family = AF_INET;
      addrs[0].sin_addr.s_addr = inet_addr("192.168.200.1");
      addrs[0].sin_port = htons(PORT);
      addrs[0].sin_len = sizeof(struct sockaddr_in);


      addrs[1].sin_family = AF_INET;
      addrs[1].sin_addr.s_addr = inet_addr("192.168.100.1");
      addrs[1].sin_port = htons(PORT);
      addrs[1].sin_len = sizeof(struct sockaddr_in);

      /* the addrs must use the same port */

      /* set this flag */
      flags = SCTP_BINDX_ADD_ADDR;

      /* bind the socket */
      if ((ret=sctp_bindx (fd, (struct sockaddr*)addrs, ADDRS_COUNT, flags)) == ERROR)
      {
              close(fd);
              printf ("sctp_bindx error: %s",strerror(errno));
      }
      return ret;
}

void test ()
{
      int fd;

      if ((fd = create_socket (AF_INET, SOCK_STREAM, IPPROTO_SCTP)) < 0)
              return;

      if (do_bindx (fd) < 0)
              return;
      /* action */
      close (fd);

}
```

Returns

OK, or **ERROR**.

Errno

Not Available

See also

sockLibSctpExt

Name

sctp_connectx( ) - initiate a association with a specific subset of addresses

Layer

IPNET_USRSPACE

Synopsis

```
int sctp_connectx
    (
    int             fd,         /* the socket descriptor */
    struct sockaddr *addrs,     /* an array of addresses */
    int             addrcnt,    /* the number of addresses in the array */
    sctp_assoc_t    *id         /* the association identification for the newly created associa
tion (if successful) */
    )
```

Description

Much like **sctp_bindx( )** this call allows a caller to specify multiple addresses at which a peer can be reached. The way the SCTP stack uses the list of addresses to set up the association is implementation dependent. This function only specifies that the stack will try to make use of all the addresses in the list when needed.

Note

the list of addresses passed in is only used for setting up the association. It does not necessarily equal the set of addresses the peer uses for the resulting association. If the caller wants to find out the set of peer addresses, it must use **sctp_getpaddrs( )** to retrieve them after the association has been set up.

Parameters

fd

Specifies the file descriptor for the socket.

*addrs

Specifies an array of addresses.

addrcnt

Specifies the addresses array count.

*id

Specifies the association identification for the newly created association.

Example

The following example demonstrates the use of the **sctp_connectx( )** function:

```
int create_socket (int family, int type, int proto)
{
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

int do_connectx (int fd)
{

#define       ADDRS_COUNT     2

#define       PORT    1234

      struct sockaddr_in paddrs[ADDRS_COUNT];
      sctp_assoc_t id;
      int ret;

      /* specify the addresses to be connected */
      memset (paddrs, 0, ADDRS_COUNT*sizeof(struct sockaddr_in));
      paddrs[0].sin_family = AF_INET;
      paddrs[0].sin_addr.s_addr = inet_addr("192.168.200.1");
      paddrs[0].sin_port = htons(PORT);
      paddrs[0].sin_len = sizeof(struct sockaddr_in);


      paddrs[1].sin_family = AF_INET;
      paddrs[1].sin_addr.s_addr = inet_addr("192.168.100.1");
      paddrs[1].sin_port = htons(PORT);
      paddrs[1].sin_len = sizeof(struct sockaddr_in);

      /* connect  the peer */
      if ((ret=sctp_connectx(fd, (struct sockaddr*)paddrs, ADDRS_COUNT, &id)) == ERROR)
      {
              close(fd);
              printf ("sctp_connectx error: %s",strerror(errno));
      }
      return ret;
}

void test ()
{
      int fd;

      if ((fd = create_socket (AF_INET, SOCK_STREAM, IPPROTO_SCTP)) < 0)
              return;

      if (do_connectx (fd) < 0)
              return;

      /* action */
      close (fd);
```

```
}
```

Returns

OK, or **ERROR**.

Errno

Not Available

See also

sockLibSctpExt, **sctp_getpaddrs( )**, **sctp_bindx( )**

Name

sctp_send( ) - send SCTP data to a socket

Layer

IPNET_USRSPACE

Synopsis

```
ssize_t sctp_send
    (
    int                         fd,       /* the socket descriptor */
    const void                  *pbuf,    /* the message to be sent */
    size_t                      len,      /* the length of the message */
    const struct sctp_sndrcvinfo *sinfo,   /* used a sendmsg call */
    int                         flags     /* the same flags as used by the sendmsg call flags */
    )
```

Description

Provide another alternative function or system call to assist an application with the sending of data without the use of the CMSG header structures.

The flags value is the same as used by the sendmsg call flags (e.g. **MSG_DONTROUTE**).

Parameters

fd

Specifies the file descriptor for the socket.

*pbuf

Specifies the name of the message to be transmitted.

len

Specifies the length of the message.

*sinfo

Specifies a pointer to a sctp_sndrcvinfo structure to be filled upon receipt of the message.

flags

Specifies a data transmission flag value. You may OR the following values into the flags parameter with this operation:

MSG_OOB (0x1)

Out-of-band data. Unsupported for **SOCK_SEQPACKET**-type sockets.

MSG_DONTROUTE (0x4)

Send without using routing tables.

Returns

The number of bytes sent, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt, **sendmsg( )**

Name

sctp_sendx( ) - send SCTP data to a socket

Layer

IPNET_USRSPACE

Synopsis

```
ssize_t sctp_sendx
    (
    int                     fd,         /* the socket descriptor */
    const void              *msg,       /* the message to be sent */
    size_t                  len,        /* the length of the message */
    struct sockaddr         *addrs,     /* an array of addresses */
    int                     addrcnt,    /* the number of addresses in the array */
    struct sctp_sndrcvinfo *sinfo,      /* used for a sendmsg call */
    int                     flags       /* The same flags as used by the sendmsg call flags */
    )
```

Description

call to assist an application with the sending of data without the use of the CMSG header structures that also gives a list of addresses. The list of addresses is provided for implicit association setup. In such a case the list of addresses serves the same purpose as the addresses given in **sctp_connectx( )**

The flags value is the same as used by the sendmsg call flags (e.g. **MSG_DONTROUTE**).

Note

on return from this call the sinfo structure will have changed in that the sinfo_assoc_id will be filled in with the new association id.

Parameters

fd

Specifies the file descriptor for the socket.

*msg

Specifies the message to be sent.

len

Specifies the length of the message to be sent.

*addrs

Specifies an array of addresses.

addrcnt

Specifies the number of addresses in the array.

*sinfo

Specifies a pointer to a sctp_sndrcvinfo structure to be filled upon receipt of the message.

flags

Specifies a data transmission flag value. You may OR the following values into the flags parameter with this operation:

MSG_OOB (0x1)

Out-of-band data. Unsupported for **SOCK_SEQPACKET**-type sockets.

MSG_DONTROUTE (0x4)

Send without using routing tables.

Example

The following example demonstrates the use of the **sctp_sendx( )** function:

```
int create_socket (int family, int type, int proto)
{
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

void sndmsgx (int fd)
{
```

```
#define       ADDRS_COUNT     2

#define       PORT    1234


      struct sockaddr_in addrs[ADDRS_COUNT];
      int ret;
      struct sctp_sndrcvinfo info;

      char buf [] = "windriver";
      /* the addr specifies the address family , IP address and port
       for the socket that is being bound */
      memset (addrs, 0, ADDRS_COUNT*sizeof(struct sockaddr_in));
      addrs[0].sin_family = AF_INET;
      addrs[0].sin_addr.s_addr = inet_addr("192.168.200.1");
      addrs[0].sin_port = htons(PORT);
      addrs[0].sin_len = sizeof(struct sockaddr_in);

      addrs[1].sin_family = AF_INET;
      addrs[1].sin_addr.s_addr = inet_addr("192.168.100.1");
      addrs[1].sin_port = htons(PORT);
      addrs[1].sin_len = sizeof(struct sockaddr_in);

      memset (&info, 0, sizeof(info));

      ret = sctp_sendx (fd, buf, sizeof(buf), (struct sockaddr*)addrs, ADDRS_COUNT, &info, 0);

      if (ret == ERROR)
              printf ("sctp_sendx error:%s\n", strerror(errno));
      else if (ret!=sizeof (buf))
              printf ("not all data was sent\n");
      else
              printf ("send succefully\n");
}

void test ()
{
      int fd;

      if ((fd = create_socket (AF_INET, SOCK_SEQPACKET, IPPROTO_SCTP)) < 0)
              return;
      sndmsgx (fd);
      /* action */

      close (fd);
}
```

Returns

The number of bytes sent, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt, **sctp_connectx( )**, **sendmsg( )**

Name

sctp_sendmsg( ) - send a SCTP message to a socket

Layer

IPNET_USRSPACE

Synopsis

```
ssize_t sctp_sendmsg
    (
    int                     fd,          /* the socket descriptor */
    const void              *pbuf,       /* the message to be sent */
    size_t                  len,         /* the length of the message */
    const struct sockaddr *to,           /* the destination address of the message */
    socklen_t               tolen,       /* the length of the destination address */
    uint32_t                ppid,        /* payload protocol id */
    uint32_t                flags,       /* flags for sendmsg() and recvmsg() */
    uint32_t                stream_no,   /* number of streams */
    uint32_t                pr_value,    /* depends of the PR-SCTP policy specified by the sinfo
_pr_policy field */
    uint32_t                context      /* context data for sendmsg() only */
    )
```

Description

Sending a message using **sctp_sendmsg( )** is atomic unless explicit EOR marking is enabled on the socket specified by sd.

Note

On return from this call the sinfo structure will have changed in that the sinfo_assoc_id will be filled in with the new association id.

Parameters

fd

Specifies the file descriptor for the socket.

*pbuf

Specifies the message to be sent.

len

Specifies the length of the message to be sent.

*to

Specifies the destination address of the message.

tolen

Specifies the length of the destination address.

ppid

Specifies the payload protocol id.

flags

Specifies a data transmission flag value. You may OR the following values into the flags parameter with this operation:

MSG_OOB (0x1)

Out-of-band data. Unsupported for **SOCK_SEQPACKET**-type sockets.

MSG_DONTROUTE (0x4)

Send without using routing tables.

stream_no

Specifies the number of streams.

pr_value

Specifies the PR-SCTP policy as per the sinfo_pr_policy field.

context

Specifies the context data for **sendmsg( )** only.

Example

The following example demonstrates the use of the **sctp_sendmsg( )** function:

```
int create_socket (int family, int type, int proto)
{
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

void sctp_snd (int fd)
{

#define       PORT    1234
      char buf[] = "windriver";
      struct sockaddr_in paddr;
      int ret;

      memset (&paddr, 0, sizeof(struct sockaddr_in));
      paddr.sin_family = AF_INET;
      paddr.sin_addr.s_addr = inet_addr("192.168.200.1");
      paddr.sin_port = htons(PORT);
      paddr.sin_len = sizeof(struct sockaddr_in);
```

```
      /* send message to the peer over stream 0 */
      ret = sctp_sendmsg (fd, buf, sizeof(buf), (struct sockaddr*)&paddr, sizeof(paddr), 0, 0
, 0, 0, 0);
      if (ret == ERROR)
              printf ("sctp_sendmsg error: %s\n", strerror(errno));
      else if (ret != sizeof(buf))
              printf ("not all data was sent\n");
      else
              printf ("sctp_sendmsg completely\n");
}


void test ()
{
      int fd;

      if ((fd = create_socket (AF_INET, SOCK_SEQPACKET, IPPROTO_SCTP)) < 0)
              return;
      sctp_snd (fd);
      /* action */

      close (fd);

}
```

Returns

The number of characters sent, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt, **sendmsg( )**

Name

sctp_recvmsg( ) - receive a SCTP message from a socket

Layer

IPNET_USRSPACE

Synopsis

```
ssize_t sctp_recvmsg
    (
    int                     fd,          /* the socket descriptor */
    void                    *pbuf,       /* the message buffer to be filled */
    size_t                  len,         /* the length of the message buffer */
    struct sockaddr         *from,       /* a address to be filled with the sender of this mess
```

```
ages address */
    socklen_t              *fromlen,    /* an in/out parameter describing the from length */
    struct sctp_sndrcvinfo *sinfo,      /* a pointer to a sctp_sndrcvinfo structure to be fill
ed upon receipt of the message */
    int                    *msg_flags   /* a pointer to a integer to be filled with any messag
e flags(e.g.  MSG_NOTIFICATION) */
    )
```

Description

The **sctp_sendmsg( )** system call is used to transmit a message to another SCTP endpoint.

The msg_flags is a pointer to a integer to be filled with any message flags (e.g. **MSG_NOTIFICATION**). Note that this field is an in-out field. Options for the receive may also be passed into the value (e.g. **MSG_PEEK**). On return from the call, the msg_flags value will be different than what was sent in to the call. If implemented via a **recvmsg( )** call, the msg_flags should only contain the value of the flags from the **recvmsg( )** call.

Note

In order for the sctp_sndrcvinfo structure to be filled in by **sctp_recvmsg( )** the caller must enable the sctp_data_io_events with the **SCTP_EVENTS** option. the setting of the **SCTP_USE_EXT_RCVINFO** will effect this function as well, causing the sctp_sndrcvinfo information to be extended.

Parameters

fd

Specifies the file descriptor for the socket.

*pbuf

Specifies the message buffer to be filled.

len

Specifies the length of the message buffer.

*from

Specifies an address to be filled with the sender of the message address.

*fromlen

Specifies the from length.

*sinfo

Specifies a pointer to a sctp_sndrcvinfo structure to be filled upon receipt of the message.

*msg_flags

Specifies a pointer to a integer to be filled with any message flags(e.g. **MSG_NOTIFICATION**).

Example

The following example demonstrates the use of the **sctp_recvmsg( )** function:

```
int create_socket (int family, int type, int proto)
{
```

```
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

int do_bindx (int fd)
{

#define       ADDRS_COUNT     2

#define       PORT    1234

      struct sockaddr_in addrs[ADDRS_COUNT];
      int flags;
      int ret;

      /* the addr specifies the address family , IP address and port
       for the socket that is being bound */
      memset (addrs, 0, ADDRS_COUNT*sizeof(struct sockaddr_in));
      addrs[0].sin_family = AF_INET;
      addrs[0].sin_addr.s_addr = inet_addr("192.168.200.1");
      addrs[0].sin_port = htons(PORT);
      addrs[0].sin_len = sizeof(struct sockaddr_in);

      addrs[1].sin_family = AF_INET;
      addrs[1].sin_addr.s_addr = inet_addr("192.168.100.1");
      addrs[1].sin_port = htons(PORT);
      addrs[1].sin_len = sizeof(struct sockaddr_in);

      /* the addrs must use the same port */

      /* must set this flag */
      flags = SCTP_BINDX_ADD_ADDR;

      /* bind the socket */
      if ((ret=sctp_bindx (fd, (struct sockaddr*)addrs, ADDRS_COUNT, flags)) == ERROR)
      {
              close(fd);
              printf ("sctp_bindx error: %s",strerror(errno));
      }
      return ret;
}

void sctp_rcv (int fd)
{

      /* infinite loop */
#define       MAX_BUFFER_SIZE 100
      char rbuf[MAX_BUFFER_SIZE];
      int flag;
      struct sctp_sndrcvinfo info;
      socklen_t len;
      struct sockaddr_in peer;
```

```
	int ret;
	for(;;)
	{
		ret = sctp_recvmsg (fd, rbuf, MAX_BUFFER_SIZE, (struct sockaddr*)&peer, &len, &i
nfo, &flag);
		if (ret == ERROR)
		{
			printf ("recv error: %s\n", strerror(errno));

		}
		else
		{
			if (ret==0)
			{
				printf ("peer closed\n");
				break;
			}
			else
			{
				printf ("peer: %s\n", inet_ntoa (peer.sin_addr));
				printf ("stream id: %d\n", info.sinfo_stream);
				printf ("%d bytes message: %s\n", ret, rbuf);
			}
		}
	}
}

void start ()
{
	int fd;

	if ((fd = create_socket (AF_INET, SOCK_SEQPACKET, IPPROTO_SCTP)) < 0)
		return;

	if (do_bindx (fd) < 0)
		return;
	if (listen (fd, 5) < 0)
		printf ("listen failed\n");
	sctp_rcv (fd);

	/* action */
	close (fd);
	/* never reach here */
}
```

Returns

The number of bytes received, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt, **sendmsg( )**, **recvmsg( )**

Name

sctp_getaddrlen( ) - get the length specified family's socket address structure

Layer

IPNET_USRSPACE

Synopsis

```
int sctp_getaddrlen
    (
    int domain   /* the address domain */
    )
```

Description

For application binary portability it is sometimes desirable to know what the kernel thinks is the length of a socket address family.

Parameters

domain

Specifies the socket domain address family.

Returns

The length of family's address structure, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt

Name

sctp_getpaddrs( ) - get all peer addresses in an association

Layer

IPNET_USRSPACE

Synopsis

```
int sctp_getpaddrs
    (
    int             fd,       /* the socket descriptor */
    sctp_assoc_t    id,       /* an association identification to get */
```

```
    struct sockaddr **addrs   /* an (packed) array of peer addresses */
    )
```

Description

sctp_getpaddrs( ) returns all peer addresses in an association.

Parameters

fd

Specifies the file descriptor for the socket.

id

Specifies the association identification to get.

**addrs

Specifies a packed array of peer addresses to get.

Example

The following example demonstrates the use of the **sctp_getpaddrs( )** function:

```
int create_socket (int family, int type, int proto)
{
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

int do_connectx (int fd)
{

#define       ADDRS_COUNT     2

#define       PORT    1234

      struct sockaddr_in paddrs[ADDRS_COUNT];
      sctp_assoc_t id;
      int ret;

      /* specify the addresses to be connected */
      memset (paddrs, 0, ADDRS_COUNT*sizeof(struct sockaddr_in));
      paddrs[0].sin_family = AF_INET;
      paddrs[0].sin_addr.s_addr = inet_addr("192.168.200.1");
      paddrs[0].sin_port = htons(PORT);
      paddrs[0].sin_len = sizeof(struct sockaddr_in);

      paddrs[1].sin_family = AF_INET;
      paddrs[1].sin_addr.s_addr = inet_addr("192.168.100.1");
      paddrs[1].sin_port = htons(PORT);
```

```
      paddrs[1].sin_len = sizeof(struct sockaddr_in);

      /* connect  the peer */
      if ((ret=sctp_connectx(fd, (struct sockaddr*)paddrs, ADDRS_COUNT, &id)) == ERROR)
      {
              close(fd);
              printf ("sctp_connectx error: %s",strerror(errno));
      }

      return ret;
}

void getpeeraddrs (int fd)
{
      struct sockaddr *paddrs = NULL;
      int count;
      if ((count = sctp_getpaddrs(fd, 0, &paddrs)) == ERROR)
              printf ("no peer address Available on the socket %d\n", fd);
      else
      {

              int i;
              struct sockaddr *ptmp;
              printf ("peer address(es):\n");
              for (i=0; i<count; i++)
              {
                      ptmp = (struct sockaddr *) ((char *) paddrs + (i * sizeof (struct sockad
dr_in)));

                      printf ("%s ",inet_ntoa (((struct sockaddr_in*)ptmp)->sin_addr));
              }
              printf ("\n");
              sctp_freepaddrs (paddrs);
      }

}

void test ()
{
      int fd;

      if ((fd = create_socket (AF_INET, SOCK_STREAM, IPPROTO_SCTP)) < 0)
              return;

      if (do_connectx (fd) < 0)
              return;
      getpeeraddrs (fd);
      /* action */
      sleep(10);
      close (fd);

}
```

Returns

The number of peer addresses in the association, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt, **sctp_freepaddrs( )**

Name

sctp_freepaddrs( ) - frees all resources allocated by **sctp_getpaddrs( )**

Layer

IPNET_USRSPACE

Synopsis

```
void sctp_freepaddrs
    (
    struct sockaddr *addrs   /* an (packed) array of addresses */
    )
```

Description

Frees all resources allocated by **sctp_getpaddrs( )**.

Parameters

*addrs

Specifies a packed array of addresses.

Returns

N/A

Errno

Not Available

See also

sockLibSctpExt, **sctp_getpaddrs( )**

Name

sctp_getladdrs( ) - get all locally bound address(es) on a socket

Layer

IPNET_USRSPACE

Synopsis

```
int sctp_getladdrs
    (
    int             fd,       /* the socket descriptor */
    sctp_assoc_t    id,       /* an association identification to get */
    struct sockaddr **addrs   /* an (packed) array of locally bound addresses */
    )
```

Description

sctp_getladdrs( ) returns all locally bound address(es) on a socket.

Parameters

fd

Specifies the file descriptor for the socket.

<id.

Specifies an association identification to get.

**addrs

Specifies a packed array of locally bound addresses to get.

Example

The following example demonstrates the use of the **sctp_getladdrs( )** function:

```
int create_socket (int family, int type, int proto)
{
      /* create a socket */
      int fd = socket (family, type, proto);
      if (fd == ERROR)
              /* show the error information */
              printf ("create socket error: %s",strerror(errno));

      return fd;
}

int do_connectx (int fd)
{

#define       ADDRS_COUNT     2

#define       PORT    1234

      struct sockaddr_in paddrs[ADDRS_COUNT];
      sctp_assoc_t id;
      int ret;

      /* specify the addresses to be connected */
      memset (paddrs, 0, ADDRS_COUNT*sizeof(struct sockaddr_in));
      paddrs[0].sin_family = AF_INET;
      paddrs[0].sin_addr.s_addr = inet_addr("192.168.200.1");
```

```
      paddrs[0].sin_port = htons(PORT);
      paddrs[0].sin_len = sizeof(struct sockaddr_in);


      paddrs[1].sin_family = AF_INET;
      paddrs[1].sin_addr.s_addr = inet_addr("192.168.100.1");
      paddrs[1].sin_port = htons(PORT);
      paddrs[1].sin_len = sizeof(struct sockaddr_in);

      /* connect  the peer */
      if ((ret=sctp_connectx(fd, (struct sockaddr*)paddrs, ADDRS_COUNT, &id)) == ERROR)
      {
              close(fd);
              printf ("sctp_connectx error: %s",strerror(errno));
      }

      return ret;
}

void getlocaladdrs (int fd)
{
      struct sockaddr *laddrs = NULL;
      int count;
      if ((count = sctp_getladdrs(fd, 0, &laddrs)) ==ERROR)
              printf ("unable to get local addresses on the socket %d\n", fd);
      else
      {

              int i;
              struct sockaddr *ptmp;
              printf ("local address(es):\n");
              for (i=0; i<count; i++)
              {
                      ptmp = (struct sockaddr *) ((char *) laddrs + (i * sizeof (struct sockad
dr_in)));

                      printf ("%s ",inet_ntoa (((struct sockaddr_in*)ptmp)->sin_addr));
              }
              printf ("\n");
              sctp_freeladdrs (laddrs);
      }
}

void test ()
{
      int fd;

      if ((fd = create_socket (AF_INET, SOCK_STREAM, IPPROTO_SCTP)) < 0)
              return;

      if (do_connectx (fd) < 0)
              return;
      getlocaladdrs (fd);
      /* action */

      close (fd);
```

```
}
```

Returns

The number of local addresses in the association, or unbound on this socket, or **ERROR** if the call fails.

Errno

Not Available

See also

sockLibSctpExt, **sctp_freeladdrs( )**

Name

sctp_freeladdrs( ) - frees all resources allocated by **sctp_getladdrs( )**

Layer

IPNET_USRSPACE

Synopsis

```
void sctp_freeladdrs
    (
    struct sockaddr *addrs   /* an (packed) array of locally bound addresses */
    )
```

Description

Frees all resources allocated by **sctp_getladdrs( )**.

Parameters

*addrs

Specifies a packed array of addresses.

Returns

N/A

Errno

Not Available

See also

sockLibSctpExt, **sctp_getladdrs( )**

Name

sctp_opt_info( ) - get SCTP socket options information

Layer

IPNET_USRSPACE

Synopsis

```
int sctp_opt_info
    (
    int          fd,        /* a socket descriptor */
    sctp_assoc_t id,        /* an association identification */
    int          optname,   /* the option name */
    void         *optval,   /* data to set for the option */
    socklen_t    *optlen    /* the length of the buffer poited to by optval */
    )
```

Description

The **sctp_opt_info( )** call is a replacement for **getsockopt( )** only and will not set any options associated with the specified socket. A **setsockopt( )** must be used to set any writeable option.

Parameters

s

Specifies the file descriptor for the socket.

id

Specifies an association identification.

optname

Specifies socket option to get.

optval

Specifies the socket option value.

optlen

Specifies the length of socket option value.

Option list

The following sections discuss options available.

SCTP_RTOINFO -- Retransmission Timeout Parameters

The protocol parameters used to initialize and bound retransmission timeout (RTO) are tunable.

SCTP_ASSOCINFO -- Association Parameters

This option is used to examine various association and endpoint parameters.

SCTP_DEFAULT_SEND_PARAM -- Default Send Parameters

Applications that wish to use the **sendto( )** system call may wish to specify a default set of parameters that would normally be supplied through the inclusion of ancillary data. This socket option allows such an application to get the default sctp_sndrcvinfo structure.

SCTP_GET_PEER_ADDR_INFO -- Peer Address Information

Applications can retrieve information about a specific peer address of an association, including its reachability state, congestion window, and retransmission timer values.

SCTP_PRIMARY_ADDR -- Primary Address

Requests that the local SCTP stack use the enclosed peer address as the association primary. The enclosed address must be one of the association peer's addresses.

SCTP_PEER_ADDR_PARAMS -- Peer Address Parameters

Applications can enable or disable heartbeats for any peer address of an association, modify an address's heartbeat interval, force a heartbeat to be sent immediately, and adjust the address's maximum number of retransmissions sent before an address is considered unreachable, this option can get these information.

SCTP_STATUS -- Association Status

Applications can retrieve current status information about an association, including association state, peer receiver window size, number of unacked data chunks, and number of data chunks pending receipt.

SCTP_CONTEXT -- The Default Context

The context field in the sctp_sndrcvinfo structure is normally only used when a failed message is retrieved holding the value that was sent down on the actual send call.

Returns

N/A

Errno

Not Available

See also

sockLibSctpExt, **getsockopt( )**, **setsockopt( )**

54. SOCKSHOW

Name

sockShow - C-interpreter socket show command

Layer

RTNET

Routines

sockShowProtoGet( ) - get protocol name **sockShowUnspecAddrFill( )** - fill unspecified address **sockShowAddrFill( )** - fill address string representation **sockShowAddrGet( )** - get socket address string representation **sockShow( )** - show open sockets **sock4Show( )** - show open IPv4 sockets **sock6Show( )** - show open IPv6 sockets

Description

This module contain a routine that print all open socket in the system.

Include files

N/A

Name

sockShowProtoGet( ) - get protocol name

Layer

RTNET

Synopsis

```
const char * sockShowProtoGet
    (
    char * strBuf,   /**< buffer for protocol name */
    UINT8  family,   /**< address family */
    UINT8  proto     /**< protocol */
    )
```

Description

This routine get protocol name for socket

Returns

A protocol name

Errno

N/A

See also

sockShow

Name

sockShowUnspecAddrFill( ) - fill unspecified address

Layer

RTNET

Synopsis

```
void sockShowUnspecAddrFill
    (
    char * strBuf,
    UINT8  family
    )
```

Description

This routine fill a string representation of the unspecified address for an address family.

Returns

N/A

Errno

N/A

See also

sockShow

Name

sockShowAddrFill( ) - fill address string representation

Layer

RTNET

Synopsis

```
void sockShowAddrFill
    (
    char *            strBuf,   /**< buffer for address */
    struct sockaddr * sa        /**< socket address */
    )
```

Description

This routine create and fill a string buffer with a string representation of a socket address.

Returns

N/A

Errno

N/A

See also

sockShow

Name

sockShowAddrGet( ) - get socket address string representation

Layer

RTNET

Synopsis

```
const char * sockShowAddrGet
    (
    char *          strBuf,   /**< buffer for address */
    struct rtattr * ska,      /**< socket attribute */
    UINT8           family    /**< socket family */
    )
```

Description

This routine get a string representation for a socket address.

Returns

A string representation of a socket address.

Errno

N/A

See also

sockShow

Name

sockShow( ) - show open sockets

Layer

RTNET

Synopsis

```
INT32 sockShow (void)
```

Description

This routine show all open sockets in the network stack.

Returns

Number of sockets printed.

Errno

N/A

See also

sockShow

Name

sock4Show( ) - show open IPv4 sockets

Layer

RTNET

Synopsis

```
INT32 sock4Show (void)
```

Description

This routine show all open IPv4 sockets in the network stack.

Returns

Number of sockets printed.

Errno

N/A

See also

sockShow

Name

sock6Show( ) - show open IPv6 sockets

Layer

RTNET

Synopsis

```
INT32 sock6Show (void)
```

Description

This routine show all open IPv6 sockets in the network stack.

Returns

Number of sockets printed.

Errno

N/A

See also

sockShow

55. SYSSOCKLIB

Name

sysSockLib - socket infrastructure library

Layer

RTNET

Routines

sysSockBackendIsMatch( ) - check if a backend match a lookup key **sysSockCanRepresentAsUint8( )** - check if an integer can be represented as **sysSockCanRepresentAsUint16( )** - check if an integer can be represented as **sysSockRxEpilogue( )** - socket receive epilogue **sysSockGetsockopt( )** - get socket option handler **sysSockSetsockopt( )** - set socket option handler **mayQueue( )** - check if it is possible to queue to socket send buffer **sysSockSin6MapperSet( )** - set address mapper for a domain/address family **sysSockOpsInit( )** - initialize all socket operation **sysSockLibAdd( )** - add a socket protocol back-end **sysSockFdGet( )** - get descriptor for a socket **sysSockFromFdGet( )** - get socket pointer **sysSockAnyFromFdGet( )** - get socket pointer for open or closed socket **sysSockFromFlowGet( )** - get socket matching a flow **sysSock5TupleGet( )** - get a new socket with a specific 5-tuple **sysSockPut( )** - put back socket to socket library. **sysSockIsReadable( )** - check if a socket can be read from **sysSockIsWritable( )** - check if a socket can be written to **sysSockNotify( )** - notify tasks pending on the socket about event **sysSockErrorSet( )** - set error condition on socket **sysSockRefAdd( )** - add a reference to a socket **sysSockRefDel( )** - delete a reference to a socket **sysSockRxDgram( )** - deliver data to datagram socket receive buffer **sysSockRxStream( )** - deliver data to stream socket receive buffer **sysSockTxStreamPeek( )** - add socket stream data to a packet **sysSockRxStreamPoke( )** - store out-of-order stream data in socket receive buffer **sysSockInit( )** - initialize **sockLib** backend functions **sysSockLibInit( )** - initialize **sockLib** backend **sysSockForEach( )** - run an user-define routine for each open socket **sysSockbufOrderSet( )** - sets socket buffer order value

Description

This module implements a socket infrastructure that provides a back-end to **sockLib** and also functions for underlying protocol handlers to fetch and deliver data from/to sockets.

This library must be initialized by calling **sysSockLibInit( )** before any other function in the library is called. The function must be passed a pointer to the default network namespace that will be assigned to sockets when created. The **rtnetStackInit( )** routine will register this library as a **sockLib** back end for **AF_INET**, **AF_INET6**, **AF_NETLINK** and optionally for **AF_SOCKET** regardless of whether the network stack resides in the kernel or in an RTP.

The **sysSockInit( )** function returns a pointer to a **SOCK_FUNC** structure containing all the BSD-socket function handlers. This function is used both when registering this library as a **sockLib** back end, and when initializing the socket proxy dispatch servie running in an RTP.

Protocol handler helper functions

A protocol handler that wants to get the socket matching a packet should initialize an instance of type **SYS_SOCK_FLOW** and call **sysSockFromFlowGet( )** to get the matching socket. Another way of acquiring a socket reference is through a socket handle and a call to **sysSockFromFdGet( )**. The handle for a socket is returned by **sysSockFdGet( )**.

When successful, both **sysSockFromFlowGet( )** and **sysSockFromFdGet( )** return an acquired socket with its lock held, in order to protect the socket's state. The protocol handler must therefore call **sysSockPut( )** to unlock any socket returned via those functions when done with it.

A socket will be released to the free-pool when the last reference to the socket is deleted. Sockets normally get a reference count of one, but the reference count can be manipulated by protocol handlers via **sysSockRefAdd( )** and **sysSockRefDel( )**. **sysSockRefDel( )** must be treated as a delete operation of the socket, which mean that the socket must not be used after a call to **sysSockRefDel( )**.

A protocol handler that want to queue data on a socket must use **sysSockRxDgram( )** or **sysSockRxStream( )** depending on whether the protocol is datagram- or stream-based. The content and relevant meta-data of the packet context structure is copied to the socket receive buffer and no reference is kept on the packet context structure.

```
    ...
    const PKT_CTX * pktCtx = ...
    SOCKET * sock = sysSockFromFlowGet (&flowDesc);
    if (sock != NULL)
        {
        bytesQueued = sysSockRxStream (sock, pktCtx);
        sysSockPut (sock);
        }
    ...
```

Required packet meta-data is nothing for stream sockets while datagram socket must at least have the source address included and ideally also ingress network interface ID and destination address (which is one of the local addresses on the node). Meta-data is set as packet context attributes, a pointer to the attributes is returned by **pktCtxAttrGet( )**.

Transmitting data is done via the functions **sysSockTxDgram( )** or **sysSockTxStreamPeek( )**, they differ slightly in semantics as the function for stream protocols leave the data on the socket buffer. Note that the socket lock must be held for the lifetime of the packet context passed to **sysSockTxDgram( )**/**sysSockTxStreamPeek( )** as the data is not copied, the packet context is pointing directly into the socket buffer.

sysSockTxDgram( ) return the destination transport layer port as argument and include the destination address as packet meta-data.

```
    ...
    SOCKET * sock = ...
    UINT16BE remotePort;
    PKT_CTX  pktCtx;

    pktCtxInit (&pktCtx, pNetworkNamespace);
    /* Include the next datagram from the socket send buffer */
    sysSockTxDgram (sock, &pktCtx, &remotePort);
    doProtocolProcessing (sock, &pktCtx);
    sysSockPut (sock);
    /* The packet context is no longer valid after sysSockPut() */
    ...
```

Stream-based protocol handler must explicitly remove data from the socket send buffer when it is safe to do so. As an example, TCP may remove the data when an valid acknowledgment has been received. Data is removed by adding one for every byte to remove to the SOCKET member ´snd.cons´.

```
streamSockSndBufRemove(SOCKET * sock, UINT32 bytesToRemove)
{
    sock->snd.cons += bytesToRemove;
}
```

Include files

sysSockLib.h

Name

sysSockBackendIsMatch( ) - check if a backend match a lookup key

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL sysSockBackendIsMatch
    (
    const SYS_SOCK_BACKEND * backend,   /**< a socket backend instance */
    UINT32                   key        /**< backend lookup key */
    )
```

Description

This routine check if a lookup key matches the passed socket backend instance.

Returns

TRUE if the lookup key matches, or **FALSE** otherwise.

Errno

N/A

See also

sysSockLib

Name

sysSockCanRepresentAsUint8( ) - check if an integer can be represented as

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL sysSockCanRepresentAsUint8
    (
    int val
    )
```

Description

8-bit unsigned number

This routine check if an integer value can be represented as an 8-bit unsigned number

Returns

OK if possible to represent as 8-bit unsigned integer, or **FALSE** otherwise.

Errno

N/A

See also

sysSockLib

Name

sysSockCanRepresentAsUint16( ) - check if an integer can be represented as

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL sysSockCanRepresentAsUint16
    (
    int val
    )
```

Description

16-bit unsigned number

This routine check if an integer value can be represented as an 16-bit unsigned number

Returns

OK if possible to represent as 16-bit unsigned integer, or **FALSE** otherwise.

Errno

N/A

See also

sysSockLib

Name

sysSockRxEpilogue( ) - socket receive epilogue

Layer

RTNET

Synopsis

```
_WRS_INLINE void sysSockRxEpilogue
    (
```

```
    SOCKET *   sock,   /**< a socket where data has been queued */
    SOCK_BUF * rcv     /**< socket receive buffer */
    )
```

Description

This routine check whether the socket is closed for reading, if so then drop all data in the receive buffer as the application cannot read it.

Returns

N/A

Errno

N/A

See also

sysSockLib

Name

sysSockGetsockopt( ) - get socket option handler

Layer

RTNET

Synopsis

```
STATUS sysSockGetsockopt
    (
    struct socket * so,        /**< socket instance */
    int             level,     /**< socket option level */
    int             optName,   /**< socket option name */
    void *          optVal,    /**< pointer to socket option value */
    socklen_t *     optLen     /**< will hold length of value */
    )
```

Description

This routine first call the socket back-end specific socket option get handler, if that call return that the socket name is unknown then this routine call the generic socket option set routine **sysSockOptGet( )**.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOPROTOOPT

The option is unknown at the level indicated.

See also

sysSockLib

Name

sysSockSetsockopt( ) - set socket option handler

Layer

RTNET

Synopsis

```
STATUS sysSockSetsockopt
    (
    struct socket * so,        /**< socket instance */
    int             level,     /**< socket option level */
    int             optName,   /**< socket option name */
    const void *    optVal,    /**< pointer to socket option value */
    socklen_t       optLen     /**< length of value to set */
    )
```

Description

This routine first call the socket back-end specific socket option set handler, if that call return that the socket name is unknown then this routine call the generic socket option set routine **sysSockOptSet( )**.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOPROTOOPT

The option is unknown at the level indicated.

EINVAL

The option value got a unexpected length.

See also

sysSockLib

Name

mayQueue( ) - check if it is possible to queue to socket send buffer

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL mayQueue
    (
    UINT32 * bQueue,        /**< initially number of bytes that ideally */
                            /* is queued, actual number of bytes that can */
                            /* be queued when the routine return */
    UINT32   sndBufAvail,   /**< socket send buffer capacity */
    UINT8    sockType       /**< socket type */
    )
```

Description

This routine check if there is currently possible to queue a specified amount of bytes to a socket send buffer, the amount of bytes that can be queued might be revised.

Returns

TRUE if data can be queued, or **FALSE** otherwise.

Errno

N/A

See also

sysSockLib

Name

sysSockSin6MapperSet( ) - set address mapper for a domain/address family

Layer

RTNET

Synopsis

```
void sysSockSin6MapperSet
    (
    UINT8                   domain,   /**< address domain/family, i.e. AF_INET */
    SYS_SOCK_TO_SIN6_RTN    toRtn,    /**< map domain address to in6_addr */
    SYS_SOCK_FROM_SIN6_RTN  fromRtn   /**< map domain address from in6_addr */
    )
```

Description

This routine set socket address mapper routines for the specified domain/address family. All addresses are stored as **struct in6_addr** in the socket, these functions map to/from the actual domain.

Returns

N/A

Errno

N/A

See also

sysSockLib

Name

sysSockOpsInit( ) - initialize all socket operation

Layer

RTNET

Synopsis

```
void sysSockOpsInit
    (
    SYS_SOCK_OPS * ops
    )
```

Description

This routine initialize all socket operation except the send function to a default behavior. The send function must be set by every backend.

Returns

N/A

Errno

N/A

See also

sysSockLib

Name

sysSockLibAdd( ) - add a socket protocol back-end

Layer

RTNET

Synopsis

```
STATUS sysSockLibAdd
    (
    UINT8                domain,          /**< address domain, AF_xxx */
    UINT8                type,            /**< socket type, SOCK_xxx */
    UINT8                sockProto,       /**< socket protocol, IPPROTO_xxx */
    UINT8                netProto,        /**< network protocol, IPPROTO_xxx */
    UINT8                behaviorFlags,   /**< behavior flags, SB_xxx */
    const SYS_SOCK_OPS * ops              /**< backend socket options */
    )
```

Description

This routine add a socket protocol back-end for a specific address domain, socket type and protocol ID combination.

Returns

OK, or **ERROR** if the back-end could not be added.

Errno

EEXIST

A back-end is already registered for the (domain, type, protocol) tuple.

ENOMEM

Cannot allocate enough memory.

See also

sysSockLib

Name

sysSockFdGet( ) - get descriptor for a socket

Layer

RTNET

Synopsis

```
int sysSockFdGet
    (
    const SOCKET * so   /**< a socket */
    )
```

Description

This routine get the descriptor number for a specific socket instance.

Returns

Socket descriptor.

Errno

Not Available

See also

sysSockLib

Name

sysSockFromFdGet( ) - get socket pointer

Layer

RTNET

Synopsis

```
SOCKET * sysSockFromFdGet
    (
    int fd   /**< socket handle */
    )
```

Description

This routine gets the socket instance pointer matching the socket descriptor.

Returns

Pointer to socket, or **NULL** if the descriptor value is out-of-range or the socket is not open.

Errno

Not Available

See also

sysSockLib, **sysSockFromFlowGet( )**, **sysSockPut( )**

Name

sysSockAnyFromFdGet( ) - get socket pointer for open or closed socket

Layer

RTNET

Synopsis

```
SOCKET * sysSockAnyFromFdGet
    (
    int fd   /**< socket handle */
    )
```

Description

This routine gets the socket instance pointer matching the socket descriptor. It allows matching a socket that has been closed (or never opened) by the application; it is assumed the calling protocol still holds a reference to the socket in this case. (This routine does not attempt to check this condition.)

Returns

Pointer to socket, or **NULL** if the descriptor value is out-of-range.

Errno

Not Available

See also

sysSockLib, **sysSockFromFdGet( )**, **sysSockFromFlowGet( )**, **sysSockPut( )**

Name

sysSockFromFlowGet( ) - get socket matching a flow

Layer

RTNET

Synopsis

```
SOCKET * sysSockFromFlowGet
    (
    const SYS_SOCK_FLOW * flow   /**< flow specification */
    )
```

Description

This routine get the socket that match the specified flow, the caller must return the socket with a call to **sysSockPut( )**.

Returns

A socket pointer, or **NULL** if no matching socket exist.

Errno

Not Available

See also

sysSockLib, **sysSockFromFdGet( )**, **sysSockPut( )**

Name

sysSock5TupleGet( ) - get a new socket with a specific 5-tuple

Layer

RTNET

Synopsis

```
SOCKET * sysSock5TupleGet
    (
    SOCKET *                  parent,   /**< a parent socket */
    const struct sockaddr_in6 * name,     /**< local address of new socket */
    const struct sockaddr_in6 * to        /**< remote address of new socket */
    )
```

Description

This routine create a new socket and associate it with a specified parent socket. Note that the parent socket lock is released if this function fails. Both the parent socket and the returned socket are locked if the routine is successful, both sockets need to be released with a call to **sysSockPut( )**.

Returns

Pointer to a new socket, or **NULL** if the operation failed

Errno

Not Available

See also

sysSockLib, **sysSockFromFdGet( )**, **sysSockFromFlowGet( )**, **sysSockPut( )**

Name

sysSockPut( ) - put back socket to socket library.

Layer

RTNET

Synopsis

```
void sysSockPut
    (
    SOCKET * sock   /**< a socket */
    )
```

Description

This routine is called when a caller is done using a socket received via **sysSockGet( )**, **sysSockFromFdGet( )** or **sysSockFromFlowGet( )**.

Returns

N/A

Errno

Not Available

See also

sysSockLib, **sysSockFromFdGet( )**, **sysSockFromFlowGet( )**

Name

sysSockIsReadable( ) - check if a socket can be read from

Layer

RTNET

Synopsis

```
BOOL sysSockIsReadable
    (
    const SOCKET * sock   /**< socket to check if readable */
    )
```

Description

This routine check if a socket can be read from without blocking.

Returns

TRUE if the socket can be read from, **FALSE** otherwise

Errno

Not Available

See also

sysSockLib

Name

sysSockIsWritable( ) - check if a socket can be written to

Layer

RTNET

Synopsis

```
BOOL sysSockIsWritable
    (
    const SOCKET * sock   /**< socket to check if writable */
    )
```

Description

This routine check if a socket can be written to without blocking.

Returns

TRUE if the socket can be written to, **FALSE** otherwise

Errno

Not Available

See also

sysSockLib

Name

sysSockNotify( ) - notify tasks pending on the socket about event

Layer

RTNET

Synopsis

```
void sysSockNotify
    (
    SOCKET *       sock,   /**< a socket */
    SYS_SOCK_READY what    /**< type of event */
    )
```

Description

This routine notify tasks pending on the socket about event

Returns

N/A

Errno

N/A

See also

sysSockLib

Name

sysSockErrorSet( ) - set error condition on socket

Layer

RTNET

Synopsis

```
void sysSockErrorSet
    (
    SOCKET * sock,   /**< a socket */
    UINT32   error   /**< socket error to set */
    )
```

Description

This routine set an error condition on a socket

Returns

N/A

Errno

N/A

See also

sysSockLib

Name

sysSockRefAdd( ) - add a reference to a socket

Layer

RTNET

Synopsis

```
void sysSockRefAdd
    (
```

```
    SOCKET * sock   /**< socket to add reference to */
    )
```

Description

This routine add a reference to a socket received via **sysSockFromFdGet( )** or **sysSockFromFlowGet( )**. It must be called before the call to **sysSockPut( )** on the socket pointer.

Returns

N/A

Errno

Not Available

See also

sysSockLib, **sysSockFromFdGet( )**, **sysSockFromFlowGet( )**, **sysSockPut( )**

Name

sysSockRefDel( ) - delete a reference to a socket

Layer

RTNET

Synopsis

```
void sysSockRefDel
    (
    SOCKET * sock   /**< socket to release reference on */
    )
```

Description

This routine delete a reference to a socket, the socket is freed if the reference count goes to zero. This call must be called after a call to **sysSockPut( )** on the socket.

Returns

N/A

Errno

Not Available

See also

sysSockLib, **sysSockFromFdGet( )**, **sysSockFromFlowGet( )**, **sysSockPut( )**

Name

sysSockRxDgram( ) - deliver data to datagram socket receive buffer

Layer

RTNET

Synopsis

```
UINT32 sysSockRxDgram
    (
    SOCKET *        sock,   /**< a socket to queue packet data on */
    const PKT_CTX * pkt,    /**< a packet */
    UINT16BE        rPort   /**< port of remote peer */
    )
```

Description

This routine queue the packet payload in a datagram socket receive buffer.

Returns

Number of bytes queued.

Errno

N/A

See also

sysSockLib

Name

sysSockRxStream( ) - deliver data to stream socket receive buffer

Layer

RTNET

Synopsis

```
UINT32 sysSockRxStream
    (
    SOCKET *        sock,   /**< a socket to queue packet data on */
    const PKT_CTX * pkt     /**< a packet in a stream */
    )
```

Description

This routine queue the packet payload in a stream socket receive buffer.

Returns

Number of bytes queued.

Errno

N/A

See also

sysSockLib

Name

sysSockTxStreamPeek( ) - add socket stream data to a packet

Layer

RTNET

Synopsis

```
UINT32 sysSockTxStreamPeek
    (
    SOCKET *  so,      /**< stream socket to fetch data for transmission */
    PKT_CTX * pkt,     /**< packet the user-data should be added to */
    UINT32    mss,     /**< stream session maximum segment size */
    UINT32    offset   /**< offset of first byte to peek */
    )
```

Description

This routine fetch data from a stream socket send buffer and put it into a packet context structure, data is NOT removed from socket send buffer.

Returns

Number of bytes added to the packet context.

Errno

N/A

See also

sysSockLib

Name

sysSockRxStreamPoke( ) - store out-of-order stream data in socket receive buffer

Layer

RTNET

Synopsis

```
UINT32 sysSockRxStreamPoke
    (
    SOCKET *        so,      /**< stream socket to store out-of-order segment data */
    const PKT_CTX * pkt,     /**< packet containing the out-of-order data */
    UINT32          offset   /**< offset past current producer index at which to store data */
    )
```

Description

This routine stores the data in pkt in the receive buffer of so at offset bytes beyond the current producer index location. The producer index is not modified, nor is any reader application notified.

This is used by TCP to store out-of-order (but in-window) segment data.

This routine copies only that portion of the packet's data that fits in the available space of the RX socket buffer beyond offset bytes past the producer index.

Returns

Number of bytes copied.

Errno

N/A

See also

sysSockLib

Name

sysSockInit( ) - initialize **sockLib** backend functions

Layer

RTNET

Synopsis

```
SOCK_FUNC * sysSockInit (void)
```

Description

This routine assigns functions to each function pointer in the **sockLib** backend socket function array.

Returns

Pointer to a socket function array.

Errno

N/A

See also

sysSockLib

Name

sysSockLibInit( ) - initialize **sockLib** backend

Layer

RTNET

Synopsis

```
STATUS sysSockLibInit
    (
    NETNS * defaultNs
    )
```

Description

This function initializes the global **sysSockLib** data (sysSockData).

Returns

OK, or **ERROR**.

Errno

N/A

See also

sysSockLib

Name

sysSockForEach( ) - run an user-define routine for each open socket

Layer

RTNET

Synopsis

```
void sysSockForEach
    (
    SYS_SOCK_FOR_EACH_RTN rtn,      /**< user supplied routine that is called for */
                                    /* each open socket */
    void *                usrCtx    /**< user data passed to each invocation of */
                                    /* 'rtn' */
    )
```

Description

This routine call a user-defined callback routine for each open socket.

Returns

N/A

Errno

N/A

See also

sysSockLib

Name

sysSockbufOrderSet( ) - sets socket buffer order value

Layer

RTNET

Synopsis

```
void sysSockbufOrderSet
    (
    UINT8 order   /**< max capacity of socket buffer */
    )
```

Description

This routine sets the value of socket order.

Returns

N/A

Errno

N/A

See also

sysSockLib

56. SYSSOCKOPTLIB

Name

sysSockOptLib - socket option library

Layer

RTNET

Routines

sysSockOptGet( ) - get value of a socket option **sysSockOptSet( )** - set value of a socket option **ipsockOptDomainSpecSet( )** - set the IPv4 or IPv6 sock opt domain spec

Description

This module is an example of the Wind River Systems C coding convetions.

Include files

sysSockOptLib.h

Name

sysSockOptGet( ) - get value of a socket option

Layer

RTNET

Synopsis

```
STATUS sysSockOptGet
    (
    SOCKET *    sock,      /**< socket instance */
    int         level,     /**< socket option level */
    int         optName,   /**< socket option name */
    void *      optVal,    /**< buffer for socket option value */
    socklen_t * optLen     /**< number of bytes copied into optVal */
    )
```

Description

This routine get the value of a socket option, all the generic socket options are handled in this routine, protocol specific socket options is handled via the socket back-end getsockopt routine.

Returns

OK if successful, or **ERROR** otherwise

Errno

EINVAL

Invalid length of socket option value

ENOPROTOOPT

Socket option not know at this level

See also

sysSockOptLib

Name

sysSockOptSet( ) - set value of a socket option

Layer

RTNET

Synopsis

```
STATUS sysSockOptSet
    (
    SOCKET *     sock,      /**< socket instance */
    int          level,     /**< socket option level */
    int          optName,   /**< socket option name */
    const void * optVal,    /**< socket option value */
    socklen_t    optLen     /**< size of socket option value */
    )
```

Description

This routine set the value of a socket option, all the generic socket options are handled in this routine, protocol specific socket options is handled via the socket back-end setsockopt routine.

Returns

OK if successful, or **ERROR** otherwise

Errno

EINVAL

Invalid length of socket option value

ENOPROTOOPT

Socket option not know at this level

See also

sysSockOptLib

Name

ipsockOptDomainSpecSet( ) - set the IPv4 or IPv6 sock opt domain spec

Layer

RTNET

Synopsis

```
void ipsockOptDomainSpecSet
    (
    BOOL                isV4,   /**< TRUE if IPv4, FALSE if IPv6 */
    SYS_SOCK_OPT_SPEC * spec,   /**< spec pointer */
    UINT32              size    /**< spec array size */
    )
```

Description

This routine set the IPv4 or IPv6 sock opt domain spec

Returns

N/A

Errno

N/A

See also

sysSockOptLib

57. TCPLIB

Name

tcpLib - transmission control protocol library

Layer

RTNET

Routines

seqLe( ) - compare if the first sequence number is less or equal to the second **seqLt( )** - compare if the first sequence number is less than the second **seqGe( )** - compare if the first sequence number is greater than the second **seqGt( )** - compare if the first sequence number is greater or equal to the **tcpMin( )** - return the smaller of two values **tcpMax( )** - return the larger of two values **tcpDistance( )** - return the absolute difference between two values **tcpCbGet( )** - return the TCP control block for a TCP socket **tcpLwGet( )** - get TCP loss window size **tcpFlightSizeGet( )** - get the current number of text bytes in-flight **tcpBufCapGet( )** - get TCP socket buffer capacity **tcpHdrLenGet( )** - get length of TCP header and options **tcpHdrOptsLenGet( )** - get length of TCP options **tcpIsIdleTmo( )** - return the generic timeout use **tcpSockIsIdleTmo( )** - return the generic timeout use **tcpClose( )** - close a TCP socket **tcpTsAccept( )** - accept this time-stamp **tcpTextAck( )** - check if an immediate acknowledgement must be used **tcpCsumVerify( )** - Verify TCP checksum **tcpLibCreate( )** - reserve timeouts for TCP module **tcpRxGet( )** - get the TCP reception handler pointer **tcpLibInit( )** - initialize TCP module

Description

Implementation of the The transmission control protocol, TCP, for the VxWorks RTNET stack. The protocol handlers are registered both for the socket layer to allow creation of sockets with address family **AF_INET** and **AF_INET6**, type **SOCK_STREAM** and protocol **IPPROTO_TCP** and 0 (i.e. TCP is the default protocol for sockets of type **SOCK_STREAM**).

TCP is the most commonly used protocol on the Internet. TCP is a transport layer in the OSI-model and sit on top of the network layer in the OSI-model, which on the Internet mean that TCP sit on top of IPv4 or IPv6.

TCP provides reliable, ordered, and error-checked delivery of a stream of octets between applications running on Internet protocol nodes. Each TCP-session is identified by a 5-tuple consisting of the Internet protcol type which is always **IPPROTO_TCP**(6), a remote and a local Internet address, and a remote and a local 16-bit "port" that is stored within the TCP header.

The TCP-specific portions of the following RFCs is implemented by **tcpLib** /ts RFC | Title -----|------ 0793 | Transmission Control Protocol 0879 | The TCP Maximum Segment Size and Related Topics 1122 | Requirements for Internet Hosts -- Communication Layers 3390 | Increasing TCP's Initial Window 5681 | TCP Congestion Control 6298 | Computing TCP's Retransmission Timer 6528 | Defending against Sequence Number Attacks 7323 | TCP Extensions for High Performance /te

Ingress segments processing

The overall structure is to match the ingress segment against sockets, this is performed by the **tcpSockGet( )** routine. A matching socket returned by **tcpSockGet( )** has the socket lock acquired. A passively open socket, i.e. a socket created in the context of a listen socket and returned via **accept( )**, is returned with both the lock for the listen socket and the socket itself acquired (in right order, the listen socket lock must be acquired first).

A socket returned by **tcpSockGet( )** must always be released with a call to **tcpSockDone( )**.

The TCP layer must have access to the IP-layer source and destination addresses, all ingress packets must therefore have the addres source/destination attribute specified.

Socket call processing

A socket pointer passed in the socket backend calls always got the socket lock acquired and it must not be released during in the context of the routine.

Include files

tcpLib.h

Name

seqLe( ) - compare if the first sequence number is less or equal to the second

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL seqLe
    (
    UINT32 a,   /**< a TCP sequence number */
    UINT32 b    /**< another TCP sequence number */
    )
```

Description

This routine compare if the first TCP sequence number is less or equal to the second TCP sequence number.

Returns

TRUE if the first sequence number is less or equal the second, or **FALSE** otherwise

Errno

Not Available

See also

tcpLib

Name

seqLt( ) - compare if the first sequence number is less than the second

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL seqLt
    (
    UINT32 a,   /**< a TCP sequence number */
    UINT32 b    /**< another TCP sequence number */
    )
```

Description

This routine compare if the first TCP sequence number is less than the second TCP sequence number.

Returns

TRUE if the first sequence number is less than the second, or **FALSE** otherwise

Errno

Not Available

See also

tcpLib

Name

seqGe( ) - compare if the first sequence number is greater than the second

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL seqGe
    (
    UINT32 a,   /**< a TCP sequence number */
    UINT32 b    /**< another TCP sequence number */
    )
```

Description

This routine compare if the first TCP sequence number is greater than the second TCP sequence number.

Returns

TRUE if the first sequence number is greater than the second, or **FALSE** otherwise

Errno

Not Available

See also

tcpLib

Name

seqGt( ) - compare if the first sequence number is greater or equal to the

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL seqGt
    (
    UINT32 a,   /**< a TCP sequence number */
    UINT32 b    /**< another TCP sequence number */
    )
```

Description

second

This routine compare if the first TCP sequence number is greater or equal to the second TCP sequence number.

Returns

TRUE if the first sequence number is greater or equal to second, or **FALSE** otherwise

Errno

Not Available

See also

tcpLib

Name

tcpMin( ) - return the smaller of two values

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 tcpMin
    (
    UINT32 a,   /**< a value */
    UINT32 b    /**< another value */
    )
```

Description

This routine return the smaller of two values

Returns

The smaller of two values

Errno

Not Available

See also

tcpLib

Name

tcpMax( ) - return the larger of two values

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 tcpMax
    (
    UINT32 a,   /**< a value */
    UINT32 b    /**< another value */
    )
```

Description

This routine return the larger of two values

Returns

The larger of two values

Errno

Not Available

See also

tcpLib

Name

tcpDistance( ) - return the absolute difference between two values

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 tcpDistance
    (
    UINT32 a,   /**< a value */
    UINT32 b    /**< another value */
    )
```

Description

This routine return the absolute difference between two values

Returns

Distance between two values

Errno

N/A

See also

tcpLib

Name

tcpCbGet( ) - return the TCP control block for a TCP socket

Layer

RTNET

Synopsis

```
_WRS_INLINE TCP_CB * tcpCbGet
    (
    SOCKET * sock   /**< TCP socket */
    )
```

Description

This routine return the TCP control block for a TCP socket.

Returns

Pointer to TCP control block

Errno

Not Available

See also

tcpLib

Name

tcpLwGet( ) - get TCP loss window size

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 tcpLwGet
    (
    const TCP_CB * tcb   /**< TCP control block */
    )
```

Description

This routine get the TCP loss window size.

Returns

Loss window size

Errno

N/A

See also

tcpLib

Name

tcpFlightSizeGet( ) - get the current number of text bytes in-flight

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 tcpFlightSizeGet
    (
```

```
    const TCP_CB * tcb    /**< TCP control block */
    )
```

Description

This routine get the current number of text bytesn transmitted but not yet acknowledged.

Returns

Number of text bytes in flight.

Errno

N/A

See also

tcpLib

Name

tcpBufCapGet( ) - get TCP socket buffer capacity

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT32 tcpBufCapGet
    (
    const SOCK_BUF * buf    /**< socket buffer */
    )
```

Description

This routine get the TCP socket buffer capacity

Returns

TCP socket buffer capacity

Errno

N/A

See also

tcpLib

Name

tcpHdrLenGet( ) - get length of TCP header and options

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT8 tcpHdrLenGet
    (
    const TCP_HDR * th   /**< TCP header */
    )
```

Description

This routine get length of the TCP header and any included options

Returns

Length in bytes, is multiple of 4.

Errno

N/A

See also

tcpLib

Name

tcpHdrOptsLenGet( ) - get length of TCP options

Layer

RTNET

Synopsis

```
_WRS_INLINE UINT8 tcpHdrOptsLenGet
    (
    const TCP_HDR * th   /**< TCP header */
    )
```

Description

This routine get the length of the options following the TCP header.

Returns

Length in bytes, is multiple of 4. 0 is returned if no option is present.

Errno

N/A

See also

tcpLib

Name

tcpIsIdleTmo( ) - return the generic timeout use

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL tcpIsIdleTmo
    (
    const TCP_CB * tcb   /**< TCP control block */
    )
```

Description

This routine return whether the generic timeout is used to detect idle connections or used for **TIME_WAIT**.

Returns

TRUE if the generic timeout is used to detect idle connections

Errno

N/A

See also

tcpLib

Name

tcpSockIsIdleTmo( ) - return the generic timeout use

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL tcpSockIsIdleTmo
    (
    const TCP_CB * tcb,
    const SOCKET * sock
    )
```

Description

This routine returns **TRUE** if the generic timeout is used to detect idle connections. When it returns **FALSE**, the generic timeout moves the TCP state to CLOSED.

This routine has special handling of SYN-SENT and SYN-RECEIVED state sockets to account for an immediate timeout meant to move such a socket to closed state after the user application closed the socket. (In this case, a SYN-RECEIVED socket corresponds to a simultaneous open.)

Returns

TRUE if the generic timeout is used to detect idle connections

Errno

N/A

See also

tcpLib

Name

tcpClose( ) - close a TCP socket

Layer

RTNET

Synopsis

```
_WRS_INLINE void tcpClose
    (
    SOCKET * sock   /**< TCP socket */
    )
```

Description

This routine close a TCP socket, the socket must not be used after this call

Returns

N/A

Errno

Not Available

See also

tcpLib

Name

tcpTsAccept( ) - accept this time-stamp

Layer

RTNET

Synopsis

```
_WRS_INLINE void tcpTsAccept
    (
    TCP_CB * tcb   /**< transmission control block */
    )
```

Description

This routine update the "recent" time-stamp of the session peer, this value is echoed back in any TCP segment transmitted.

Returns

N/A

Errno

Not Available

See also

tcpLib

Name

tcpTextAck( ) - check if an immediate acknowledgement must be used

Layer

RTNET

Synopsis

```
_WRS_INLINE BOOL tcpTextAck
    (
    TCP_CB *        tcb,   /**< TCP control block */
```

```
    const TCP_HDR * th      /**< TCP header */
    )
```

Description

This routine check if acknowledgement of the last received piece of text can be delayed up to 200 msec or if the acknowledgement must be transmitted now. This routine start a delayed ack timer to ensure that an acknowledgment is eventually transmitted if the routine returns **FALSE**.

Returns

TRUE if acknowledgment must be transmitted now, or **FALSE** if an delayed acknowledgment is acceptable.

Errno

N/A

See also

tcpLib

Name

tcpCsumVerify( ) - Verify TCP checksum

Layer

RTNET

Synopsis

```
_WRS_INLINE void tcpCsumVerify
    (
    PKT_CTX *       pkt,      /**< TCP packet */
    const TCP_HDR * th,       /**< TCP header */
    UINT32          tcpLen    /**< TCP length */
    )
```

Description

Verify TCP checksum if the hardware has not already done it. Set cSumOK field in the packet when checksum is **OK**.

Returns

N/A

Errno

Not Available

See also

tcpLib

Name

tcpLibCreate( ) - reserve timeouts for TCP module

Layer

RTNET

Synopsis

```
void tcpLibCreate
    (
    TMO_DB tmoDb   /**< timeout database for TCP */
    )
```

Description

This routine reserve timeouts for the TCP module

Returns

N/A

Errno

Not Available

See also

tcpLib

Name

tcpRxGet( ) - get the TCP reception handler pointer

Layer

RTNET

Synopsis

```
PKT_RX_RTN tcpRxGet(void)
```

Description

This routine get and return the TCP reception handler pointer.

Returns

the TCP reception handler pointer

Errno

See also

tcpLib

Name

tcpLibInit( ) - initialize TCP module

Layer

RTNET

Synopsis

```
STATUS tcpLibInit(void)
```

Description

This routine initialize the TCP module.

Returns

OK if successful, or **ERROR** otherwise.

Errno

EEXIST

A back-end is already registered for the (domain, type, protocol) tuple.

ENOMEM

Cannot allocate enough memory.

See also

tcpLib

58. TIMEOUTLIB

Name

timeoutLib - timeout library

Layer

RTNET

Routines

timeoutDbCreate( ) - create a new timeout database **timeoutDbReserve( )** - reserve timeout instances **timeoutDbInit( )** - initialize a timeout database **timeoutDbUpdate( )** - run all expired timeouts **timeoutBind( )** - bind action handler to timeout instance **timeoutMod( )** - modify the expiration time of a timeout **timeoutCancel( )** - cancel a timeout **timeoutIsValid( )** - check if a call the the action handler is non-spurious **timeoutNow( )** - return the currrent time for this database **timeoutTaskIdGet( )** - get task driving this database

Description

The definition of a "timeout" used here is a type of timer that is optimized for the case where the timer is canceled or modified (usually into the future) before the expiration time is reached. This type of use-case is very common for timers used within a TCP/IP network stack implementation.

This library does not internally create a separate task or similar to drive the timeout state-machine, instead this library must be called by an external task. The function that must be called is **timeoutDbUpdate( )** and **timeoutLib** will provide hints as to when in time the next call should be made via an user-supplied function to **timeoutDbInit( )**.

The initialization of **timeoutLib** is done in three steps which all must occur at system initialization since dynamic memory allocation is not permitted in certified systems under normal operation:

timeoutDbCreate( )

creates a new timeout database instance.

timeoutDbReserve( )

is called by all subsystems that are going to use timeouts, maximum number of timeouts that may ever be used must be reserved.

timeoutDbInit( )

finalizes the initialization by allocating memory for all reserved timeouts.

The usage of timeout instances is slightly more restricted compared to most other timer infrastructures. Each timeout instance must be bound to a specific timeout database, expiation handler and user-defined expiration handler argument. The binding is done via **timeoutBind( )**, this must only be done once per timeout instance and must be done prior to using the timeout instance.

The action handler must have the following outline:

```
myTmoActionFn(TMO * tmo, UINT32 userArg)
    {
    /* Acquire lock for accessing the subsystem this is a handler for */
    if (timeoutIsValid(tmo))
        {
        /* Deal with timeout */
```

```
            }
        /* Release lock */
        }
```

This is required for two reasons 1. **timeoutLib** protects the internal state of the DB with a mutex, and the locking order for the subsystem using the timeout and **timeoutLib** must be well defined; it is always first the subsystem, then **timeoutLib**. 2. Due to 1., a concurrently running task might have called **timeoutMod( )** or **timeoutCancel( )** before the action handler is run, **timeoutIsValid( )** will detect such cases and return **FALSE** if the handler should not be run at this time.

The subsystem lock has to be acquired as the `reqTimeout` and `isReqRunning` members of a `TMO` instance are logically protected by the subsystem lock to allow calls to **timeoutMod( )** which move the timeout forward to happen without acquiring the lock **timeoutLib** use to protect the timeout database. `curTimeout` cannot be modified and `heapIdx` cannot be set to zero unless owning BOTH the susbsystem lock and the **timeoutLib** lock. This allows **timeoutIsValid( )** to handle the normal case, i.e. action handler should be run, without acquiring the **timeoutLib** lock.

Generation lock

Additional care need to be taken when using **timeoutLib** on any data that is protected by a generation lock. Such data cannot be referred to by a C-style pointer, as the storage address is altered each generation. The action handler must first acquire the generation lock in write mode, call **timeoutIsValid( )**, then get a pointer to the instance for example via the information supplied as user-argument, do the processing and finally release the generation lock. A new generation does not have to be created if no state is changed in the protected data.

Also note that the `TMO` instance cannot be embedded into data protected by a generation lock as the storage location for such data might change. It is possible to store a pointer to a `TMO` instance as the pointer value is copied across generations.

Include files

timeoutLib.h

See also

VxWorks 7 Real-Time Network Stack Programmer's Guide

Name

timeoutDbCreate( ) - create a new timeout database

Layer

RTNET

Synopsis

```
TMO_DB timeoutDbCreate (void)
```

Description

This routine create a new timeout database, no timers can be allocated/started until **timeoutDbInit( )** has been run.

Returns

A new timeout databaes, or **NULL** if out of memory.

Errno

Not Available

See also

timeoutLib, **timeoutDbInit( )**, **timeoutDbReserve( )**

Name

timeoutDbReserve( ) - reserve timeout instances

Layer

RTNET

Synopsis

```
void timeoutDbReserve
    (
    TMO_DB db,    /**< Timeout database */
    UINT32 cnt    /**< Number of timeout instances to reserve */
    )
```

Description

This routine reserve a specific amount of timeout instances, the timeout instances must be allocated and initialized via **timeoutBind( )** after the timeout database has finish initialization, i.e. after **timeoutDbInit( )** is called.

Returns

N/A

Errno

Not Available

See also

timeoutLib, **timeoutDbCreate( )**, **timeoutDbInit( )**, **timeoutBind( )**

Name

timeoutDbInit( ) - initialize a timeout database

Layer

RTNET

Synopsis

```
STATUS timeoutDbInit
    (
    TMO_DB     db,         /**< Timeout database */
    UINT32     tickHz,     /**< System tick frequency of this system */
    TMO_UPDATE updateFn,   /**< Function to request future call to */
                           /* timeoutDbUpdate() */
    TASK_ID    tid         /**< Task running timeout handlers */
    )
```

Description

This routine initialize a timeout database, **timeoutBind( )** cannot be called until this routine has completed successfully.

Returns

OK, or **ERROR** if the initialization fail.

Errno

ENOSPC

Too many **TMO_ID** instances has been reserved, the definition of **TMO_ID** must be changed to allow more timeout instances.

ENOMEM

Not enough memory for the requested number of timeout instances.

See also

timeoutLib, **timeoutDbCreate( )**, **timeoutDbReserve( )**, **timeoutBind( )**

Name

timeoutDbUpdate( ) - run all expired timeouts

Layer

RTNET

Synopsis

```
void timeoutDbUpdate
    (
    TMO_DB db   /**< Timeout database */
    )
```

Description

This routine run all expired timeouts and update the timeout database perception of current time.

Returns

N/A

Errno

Not Available

See also

timeoutLib

Name

timeoutBind( ) - bind action handler to timeout instance

Layer

RTNET

Synopsis

```
TMO timeoutBind
    (
    TMO_DB     db,          /**< Timeout database */
    TMO_ACTION actionFn,    /**< Timeout action handler */
    TMO_ARG *  actionArg    /**< User argument to action handler */
    )
```

Description

This routine bind an action handler and user argument to a timeout instance.

Returns

Timeout instance, or **NULL** if not enough timeouts was reserved.

Errno

Not Available

See also

timeoutLib, **timeoutDbReserve( )**

Name

timeoutMod( ) - modify the expiration time of a timeout

Layer

RTNET

Synopsis

```
void timeoutMod
    (
    TMO      tmo,       /**< Timeout instance */
    TMO_MSEC duration   /**< Duration until expiration of the timeout */
    )
```

Description

This routine modify the expiration time of a timeout, the timeout is activated if not already running.

Returns

N/A

Errno

Not Available

See also

timeoutLib

Name

timeoutCancel( ) - cancel a timeout

Layer

RTNET

Synopsis

```
void timeoutCancel
    (
    TMO tmo   /**< Timout instance */
    )
```

Description

This routine cancel a timeout, it is allowed to cancel a non-active timeout. Note that the action handler might still be called after this function complete, **timeoutIsValid( )** will return **FALSE** in that case.

Returns

N/A

Errno

Not Available

See also

timeoutLib, **timeoutIsValid( )**

Name

timeoutIsValid( ) - check if a call the the action handler is non-spurious

Layer

RTNET

Synopsis

```
BOOL timeoutIsValid
    (
    TMO tmo   /**< Timout instance */
    )
```

Description

This routine check whether a call the timeout action handler is a spurious call nor not. Spurious call might happen as part of how the internals of the timeout library deals with cancellation and extension of timeouts.

Returns

RET_DESC

Errno

N/A

See also

timeoutLib

Name

timeoutNow( ) - return the currrent time for this database

Layer

RTNET

Synopsis

```
TMO_MSEC timeoutNow
    (
    TMO_DB db   /**< timeout database */
    )
```

Description

This routine return the current time, in millisecond, for the specified timeout database. The returned time start from some unspecified absolute time and different timeout databases does not necessary use the same start time and may therefore not be compared to each other.

Returns

Absolute time in milliseconds

Errno

N/A

See also

timeoutLib

Name

timeoutTaskIdGet( ) - get task driving this database

Layer

RTNET

Synopsis

```
TASK_ID  timeoutTaskIdGet
    (
    TMO_DB db   /**< timeout database */
    )
```

Description

This routine get the task that run the timeout handlers when a timer in this database expires.

Returns

Task ID

Errno

N/A

See also

timeoutLib

59. UDPLIB

Name

udpLib - user datagram protocol library

Layer

RTNET

Routines

udpCsumVerify( ) - Verify UDP checksum **udpRxGet( )** - get the UDP reception handler pointer **udpLibInit( )** - initialize the user datagram protocol layer

Description

Implementation of the user datagram protocol, UDP, for the VxWorks RTNET stack. The protocol handlers are registered both for the socket layer to allow creation of sockets with address family **AF_INET** and **AF_INET6**, type **SOCK_DGRAM** and protocol **IPPROTO_UDP** and 0 (i.e. UDP is the default protocol for socket of type **SOCK_DGRAM**).

The handlers are register via the **udpLibInit( )** function, no other public functions is defined by this library as egress packet goes via the socket layer and ingress packet goes via the network layer, both are called through function pointers registered by **udpLibInit( )**.

UDP is best-effort, non-guaranteed ordered delivery of octet records. The benefit over TCP is largely that UDP can provide lower latency, especially in cases where packet-drop occur as TCP need to hold deliver of data until all previous data has been received while UDP deliver data as it is received.

Each UDP-session is identified by a 5-tuple consisting of the Internet protcol type which is always **IPPROTO_UDP**(17), a remote and a local Internet address, and a remote and a local 16-bit "port" that is stored within the UDP header.

/ts RFC | Title -----|----- 0768 | User Datagram Protocol 1122 | Requirements for Internet Hosts -- Communication Layers /te

Include files

udpLib.h

Name

udpCsumVerify( ) - Verify UDP checksum

Layer

RTNET

Synopsis

```
_WRS_INLINE void udpCsumVerify
    (
    PKT_CTX *       pkt,      /**< UDP packet */
    const UDP_HDR * udpHdr,   /**< UDP header */
    UINT32          udpLen    /**< UDP length */
    )
```

Description

Verify UDP checksum if the hardware has not already done it and it was not disabled in the UDP header. Set cSumOK field in the packet when checksum is **OK**.

Returns

N/A

Errno

Not Available

See also

udpLib

Name

udpRxGet( ) - get the UDP reception handler pointer

Layer

RTNET

Synopsis

```
PKT_RX_RTN udpRxGet(void)
```

Description

This routine get and return the UDP reception handler pointer.

Returns

the UDP reception handler pointer

Errno

See also

udpLib

Name

udpLibInit( ) - initialize the user datagram protocol layer

Layer

RTNET

Synopsis

```
STATUS udpLibInit
    (
    NETNS * ns   /**< network namespace */
    )
```

Description

This routine registers a handler for the user datagram protocol, UDP, into both the socket layer for transmission and the transport layer for reception of UDP packets

Returns

OK, or **ERROR** if **udpLib** could not register with sysSocklib.

Errno

EEXIST

A socket back-end is already registered for the (domain, type, protocol) tuple.

ENOMEM

Cannot allocate enough memory.

See also

udpLib

60. VETHIFLIB

Name

vethIfLib - virtual Ethernet library file

Layer

RTNET

Routines

vethIfVswitchAttach( ) - attach to virtual Ethernet switch **vethIfMuxAttach( )** - attach to MUX device

Description

Virtual Ethernet library is an implemenation of an Ethernte network device that is connected to the VxWorks virtual Ethernet switch, **vswitchLib**.

The only function exported by the library is **vethAttach( )** which creates a new virtual Ethernet device instance and attach it to the virtual switch. The returned NETIF handle can then be used with **netifLib** to add IP-addresses and perform other control tasks.

Include files

netifLib.h vethIfLib.h vswitchLib.h

Name

vethIfVswitchAttach( ) - attach to virtual Ethernet switch

Layer

RTNET

Synopsis

```
STATUS vethIfVswitchAttach
    (
    NETIF *      pNetif,    /**< will hold the network interface handle */
                            /* if the call is successful */
    NETNS *      ns,        /**< network namespace */
    const char * devName,   /**< network device name */
    UINT32       devUnit,   /**< network device unit */
    UINT8        portId     /**< vswitch port to connect to */
    )
```

Description

This routine creates a new virtual Ethernet network interface, connect it to the in-kernel virtual Ethernet switch and attach it to the network stack.

Returns

OK if successful, or **ERROR** otherwise.

Errno

N/A

See also

vethIfLib

Name

vethIfMuxAttach( ) - attach to MUX device

Layer

RTNET

Synopsis

```
STATUS vethIfMuxAttach
    (
    NETIF *      pNetif,
    NETNS *      ns,
    const char * devName,
    UINT32       devUnit
    )
```

Description

This routine creates a new virtual Ethernet network interface, connect it to the in-kernel MUX device matching the device name and unit.

Returns

OK if successful, or **ERROR** otherwise.

Errno

N/A

See also

vethIfLib

61. VSWITCHLIB

Name

vswitchLib - virtual Ethernet-switch library file

Layer

RTNET

Routines

vswitchEventIdGet( ) - get event ID from port ID **vswitchLibInit( )** - initialize a virtual Ethernet switch

Description

This module is implements an in-kernel virtual Ethernet switch to which virtual Ethernet devices, see **vethLib**, can be connected.

The switch will learn Ethernet source addresses behind each port and deliver any frame targeting that MAC to the port. Ethernet multicast frames and unicast frames not matching the MAC of any port will be forward to all ports except the ingress port.

It does not make sens to have a switch with less than two ports and the current implementation does not support more than 24 ports. **vswitchLibInit( )** therefore require a capacity between 2 to 24 ports.

This picture illustrates two stack instances running in separate RTPs, RTP-A and RTP-B. Those two stack instances are normally completly isolated from each other, it is not possible to directly send a packet from one network stack instance to the other. Both instances has created and attached a virtual Ethernet device, the device is attached to a virtual switch. This system now logically operate as if both stack instances had one physical Ethernet each where both ports where connected to an external Ethernet switch.

The network cable is replaced with a **msgQLib** channel, the physical switch is replaced with a kernel task moving frames between ports.

```
/cs     RTP-A   +---------+   |        |   |  NET#0  |>- veth0 --+            Vx-kernel   |        |          |           +---------+   +---------+          |
          |        |                    + -- port0 -<|        |                                | vswitch |     RTP-B            + -- port1 -<|        |
  +---------+          |          |        |   |        |          |          +---------+   |  NET#1  |>- veth0 --+   |        |   +---------+ /ce
```

Include files

vswitchLib.h

Name

vswitchEventIdGet( ) - get event ID from port ID

Layer

RTNET

Synopsis

```
_WRS_INLINE _Vx_event_t vswitchEventIdGet
    (
```

```
    UINT8 portId   /**< virtual Ethernet switch */
    )
```

Description

This routine return the event ID used by the specified virtual Ethernet port

Returns

Event ID

Errno

N/A

See also

vswitchLib

Name

vswitchLibInit( ) - initialize a virtual Ethernet switch

Layer

RTNET

Synopsis

```
STATUS vswitchLibInit
    (
    UINT8 capacity   /**< port capacity of switch */
    )
```

Description

This routine initialize a virtual Ethernet switch to hold a specific number of ports.

Returns

OK if successful, or **ERROR** otherwise.

Errno

EINVAL

Invalid port capacity specified, must be [2..24]

ENOMEM

Cound not allocate memory for the virtual switch.

S_memLib_NOT_ENOUGH_MEMORY

There is not enough memory to spawn the task.

See also

vswitchLib

62. VXBETHERAVBIEEE1588

Name

vxbEtherAvbIeee1588 - etherAVB IEEE1588 device driver

Layer

END

Routines

avbPtpTimerInt( ) - handle PTP interrupts **avb1588Ioctl( )** - the ioctl interface for the etherAvb IEEE1588 timer.

Description

This module implements a driver for the Renesas IEEE1588 device. For a etherAvb, the 1588 related functions are integrated with the MAC chips, which share the resources(regester space and ISR) with the MAC. For the modulization purpose, the 1588 functions are split into this dedicated driver.

Include files

none

See also

vxBus, **ifLib**, **endLib**

Name

avbPtpTimerInt( ) - handle PTP interrupts

Layer

END

Synopsis

```
void avbPtpTimerInt
    (
    IEEE1588_HARDWARE * p1588DevInfo   /* 1588 timer related */
    )
```

Description

This function is invoked whenever the EthernetAVB.ch22 interrupt is asserted by the AVTP Presentation Time Match 0.

Returns

N/A

Errno

Not Available

See also

vxbEtherAvbIeee1588

Name

avb1588Ioctl( ) - the ioctl interface for the etherAvb IEEE1588 timer.

Layer

END

Synopsis

```
int avb1588Ioctl
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    int                 cmd,
    caddr_t             data
    )
```

Description

This function is the ioctl interface for the etherAvb IEEE1588 timer.

Returns

OK, **EINVAL** or **ENOTSUP**

Errno

N/A

See also

vxbEtherAvbIeee1588

63. VXBGEIIEEE1588

Name

vxbGeiIeee1588 - GEI IEEE1588 device driver

Layer

END

Routines

gei1588TsTxOffsetConfig( ) - configure the 1588 device Tx time stamp **gei1588TsRxFilter( )** - configure the 1588 device Rx time stamp filter **gei1588IntHandle( )** - interrupt handler for the gei 1588 device **gei1588Ioctl( )** - the ioctl interface for the GEI IEEE1588 device. **gei1588Start( )** - initialize GEI IEEE1588 device's registers.

Description

This module implements a driver for the Intel IEEE1588 device. For a GEI NIC like i210/i350, the 1588 related functions are integrated with the MAC chips, which share the resources(regester space and ISR) with the MAC. For the modulization purpose, the 1588 functions are split into this dedicated driver.

Include files

none

See also

vxBus, **ifLib**, **endLib**

Name

gei1588TsTxOffsetConfig( ) - configure the 1588 device Tx time stamp

Layer

END

Synopsis

```
STATUS gei1588TsTxOffsetConfig
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    UINT32              uConfig
    )
```

Description

1-step offset features

This function is used to configure the 1588 device Tx time stamp features.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbGeileee1588

Name

gei1588TsRxFilter( ) - configure the 1588 device Rx time stamp filter

Layer

END

Synopsis

```
STATUS gei1588TsRxFilter
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    UINT32              uFilter
    )
```

Description

This function is used to configure the 1588 device Rx time stamp filter.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbGeileee1588

Name

gei1588IntHandle( ) - interrupt handler for the gei 1588 device

Layer

END

Synopsis

```
void gei1588IntHandle
    (
    IEEE1588_HARDWARE * p1588DevInfo
    )
```

Description

This is the ISR for interrupt handle for the gei 1588 device.

Returns

N/A.

Errno

See also

vxbGeileee1588

Name

gei1588Ioctl( ) - the ioctl interface for the GEI IEEE1588 device.

Layer

END

Synopsis

```
int gei1588Ioctl
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    int                 cmd,
    caddr_t             data
    )
```

Description

This function is the ioctl interface for the GEI IEEE1588 device.

Returns

OK or **EINVAL**

Errno

N/A

See also

vxbGeileee1588

Name

gei1588Start( ) - initialize GEI IEEE1588 device's registers.

Layer

END

Synopsis

```
void gei1588Start
    (
    IEEE1588_HARDWARE * p1588DevInfo
    )
```

Description

This function is called by geiEndStart and used to initialize IEEE1588 device's registers.

Returns

OK or **EINVAL**

Errno

N/A

See also

vxbGeileee1588

64. VXBIEEE1588

Name

vxbleee1588 - IEEE1588 library file for vxBus

Layer

END

Routines

ieee1588SysTimeDelayGet( ) - get the time cost of reading the system time of **ieee1588SysTimeIntervalGet( )** - get the time offset of master to slave **ieee1588SysTimeIntervalSet( )** - set the time interval to the master of **ieee1588SysTimeAdd( )** - add an offset to the system time of the IEEE1588 device. **ieee1588TimersMaxGet( )** - retrieve the max 1588 timers of END device **ieee1588Tsc2Time( )** - convert the timestamp counts of 1588 timer to the real time **ieee1588VxbTimerRegister( )** - enable the 1588 timer

Description

This library contains the IEEE1588 bus support routines.

Target-specific parameters

Include files

vxbleee1588.h

Name

ieee1588SysTimeDelayGet( ) - get the time cost of reading the system time of

Layer

END

Synopsis

```
STATUS ieee1588SysTimeDelayGet
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    long *              TimeDelay
    )
```

Description

the IEEE1588 device.

This function is used to get time cost of reading the system time of the IEEE1588 device.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbleee1588

Name

ieee1588SysTimeIntervalGet( ) - get the time offset of master to slave

Layer

END

Synopsis

```
STATUS ieee1588SysTimeIntervalGet
    (
    IEEE1588_HARDWARE *   p1588MainTimer,
    IEEE1588_TIMER_SYNC * sync
    )
```

Description

in boundary clocks

This function is used to get the time offset of master to slave in boundary clocks

Returns

OK or **ERROR**

Errno

N/A

See also

vxbleee1588

Name

ieee1588SysTimeIntervalSet( ) - set the time interval to the master of

Layer

END

Synopsis

```
STATUS ieee1588SysTimeIntervalSet
    (
    IEEE1588_HARDWARE *   p1588MainTimer,
    IEEE1588_TIMER_SYNC * sync
    )
```

Description

a boundary clock.

This function is used to set the time interval to the master of a boundary clock.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbleee1588

Name

ieee1588SysTimeAdd( ) - add an offset to the system time of the IEEE1588 device.

Layer

END

Synopsis

```
STATUS ieee1588SysTimeAdd
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    INT64 *             counts
    )
```

Description

This function is used to fast the convergence.

Returns

OK or **ERROR**

Errno

N/A

See also

vxbleee1588

Name

ieee1588TimersMaxGet( ) - retrieve the max 1588 timers of END device

Layer

END

Synopsis

```
STATUS ieee1588TimersMaxGet
    (
    void *   pCookie,
    UINT32 * pTimerMax
    )
```

Description

This function is to find out how many 1588 timers exists in the END device . The max 1588 timer units is returned in **pUnitsMax** parameter.

Returns

OK or **ERROR** if the parameter is invalid.

Errno

See also

vxbleee1588

Name

ieee1588Tsc2Time( ) - convert the timestamp counts of 1588 timer to the real time

Layer

END

Synopsis

```
void ieee1588Tsc2Time
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    UINT64              nsCounts,
    struct timespec *   ts
    )
```

Description

This function converts the timestamp counts of 1588 timer to the real time.

Returns

N/A

Errno

N/A

See also

vxbleee1588

Name

ieee1588VxbTimerRegister( ) - enable the 1588 timer

Layer

END

Synopsis

```
void ieee1588VxbTimerRegister
    (
    struct vxbTimerFunctionality * pTimerFunc
    )
```

Description

This is the function called to enable the 1588 timer.

Returns

OK or **ERROR** if the parameter is invalid.

Errno

See also

vxbleee1588

65. VXBZYNQGEMIEEE1588

Name

vxbZynqGemIeee1588 - GEM IEEE1588 device driver

Layer

END

Routines

gem1588Ioctl( ) - the ioctl interface for the GEM IEEE1588 device.

Description

This module implements a driver for the Xilinx IEEE1588 device. For an onboard GEM NIC like gem on ZCU102, the 1588 related functions are integrated with the MAC chips, which share the resources(regester space and ISR) with the MAC. For the modulization purpose, the 1588 functions are split into this dedicated driver.

Include files

none

See also

vxBus, **ifLib**, **endLib**

Name

gem1588Ioctl( ) - the ioctl interface for the GEM IEEE1588 device.

Layer

END

Synopsis

```
int gem1588Ioctl
    (
    IEEE1588_HARDWARE * p1588DevInfo,
    int                 cmd,
    caddr_t             data
    )
```

Description

This function is the ioctl interface for the GEM IEEE1588 device.

Returns

OK or **EINVAL**

Errno

N/A

See also

vxbZynqGemIeee1588

66. WEBSOCKLIB

Name

websockLib - web socket library

Layer

NET_BASE

Routines

wsCreate( ) - create a websocket **wsDelete( )** - delete a websocket session **wsMsg( )** - transmit web-socket message **wsSend( )** - send a web-socket message **wsClose( )** - close a web-socket **wsIsControlFrame( )** - check if a frame is a control frame **wsProcess( )** - process data received on the web-socket **wsUsrCtxGet( )** - get user defined context **wsReadyStateGet( )** - get web-socket session state **wsSockGet( )** - get socket descriptor

Description

This module is an implementation of "RFC6455 - The WebSocket Protocol" as a server. Websockets allow asynchronous full duplex communication with web-browser without any plugins as this is a standard feature in HTML5.

Use of this module require that _WRS_CONFIG_HASH is included in the VSB. This module depends on **httpBaseLib** and the two libraries should be used toghether to create a basic webserver that at least is capable of filtering out websocket requests. A very simple such filter is just to call the **wsCreate( )** routine on all requests, that routine will return **NULL** if the HTTP request is not a valid upgrade-to-websocket request.

The **HTTP_REQ** instance passed to the **onOpen( )** callback is guaranteed to always return a non **NULL** value for header extension **HTTP_HF_ORIGIN** as the RFC6455 require that the websocket server must be able to figure out the origin of the script generating the websocket connection attempt.

Include files

websockLib.h, Skeleton layout for **websockLib** usage, \cs, LOCAL WS_OPEN_CODE onOpen, (, WS_ID ws, /* websocket session */, const HTTP_REQ req, /* HTTP request parameters */, const char * subprotocol /* protocol the client want to use, may be */, /* NULL */,), {, WS_OPEN_CODE oc = WS_OC_OK;, /*, @ Determine if the request should be accepted or not, return one of the, @ WS_OPEN_CODE constants., */, return oc;, }, LOCAL void onMsg, (, WS_ID ws, /* websocket session */, const void * data, /* data received */, size_t dataLen, /* length of data */, WS_MSG_TYPE type /* type of data, i.e. text or binary */,), {, /*, @ Do something with data., */, }, LOCAL void onErr, (, WS_ID ws, /* websocket session */, UINT32 error /* errno error code */,), {, /*, @ Handle error., */, }, LOCAL void onClose, (, WS_ID ws, /* websocket session */, WS_STATUS_CODE statusCode /* errno error code */,), {, /*, @ Deal with close request., */, /*, @ Almost all cases should end this handler with a close of the local side, @ of the websocket session., */, wsClose(ws);, }, void httpLibReqRtn, (, const HTTP_REQ req, HTTP_RES res,), {, WS_ID wsock = wsCreate (req, res, onOpen, onMsg, onErr, onClose, usrCtx);, if (wsock != NULL), {, httpDoneSet (httpIdGet (res));, /*, @ websocket created, 'httpLibReqRtn' will not be called again for, @ this session, start using 'wsock' instead that now use the socket, @ that this HTTP session used up until now, */, }, }, \ce

Name

wsCreate( ) - create a websocket

Layer

NET_BASE

Synopsis

```
WS_ID wsCreate
    (
    const HTTP_REQ req,         /**< HTTP request context */
    HTTP_RES       res,         /**< HTTP response context */
    WS_OPEN_RTN    onOpen,      /**< websocet handshake routine */
    WS_MSG_RTN     onMessage,   /**< websocket message routine */
    WS_ERR_RTN     onError,     /**< websocket error routine */
    WS_CLOSE_RTN   onClose,     /**< websocket session close routine */
    void *         usrCtx       /**< used defined data */
    )
```

Description

This routine create a new websocket and send server handshake to client.

Returns

a websocket session or **NULL**

Errno

Not Available

See also

websockLib, **wsDelete( )**, **wsUserCtxGet( )**

Name

wsDelete( ) - delete a websocket session

Layer

NET_BASE

Synopsis

```
void wsDelete
    (
    WS_ID ws   /**< web-socket session */
    )
```

Description

This routine delete a websocket session, the user-defined close routine is called if the session has not been closed prior to this call.

Returns

N/A

Errno

Not Available

See also

websockLib, **wsCreate( )**, **wsClose( )**

Name

wsMsg( ) - transmit web-socket message

Layer

NET_BASE

Synopsis

```
STATUS wsMsg
    (
    WS_ID        ws,        /**< web-socket session */
    const void * data,      /**< payload to transmit */
    size_t       dataLen,   /**< length of payload */
    UINT8        op         /**< type of frame */
    )
```

Description

This routine transmit a web-socket framed message

Returns

OK if successful, or **ERROR** otherwise

Errno

N/A

See also

websockLib

Name

wsSend( ) - send a web-socket message

Layer

NET_BASE

Synopsis

```
STATUS wsSend
    (
    WS_ID        ws,        /**< web-socket session */
    const void * data,      /**< data to transmit */
    size_t       dataLen,   /**< length of data */
    WS_MSG_TYPE  type       /**< message type */
    )
```

Description

This routine add web-socket framing to a message and transmit it to the socket associated with the web-socket session

Returns

OK if successful, or **ERROR** otherwise

Errno

ECONNRESET

Underlying socket sessions is reset

See also

websockLib

Name

wsClose( ) - close a web-socket

Layer

NET_BASE

Synopsis

```
STATUS wsClose
    (
    WS_ID ws   /**< web-socket session */
    )
```

Description

This routine close a web-socket session

Returns

OK if successful, or **ERROR** if the close message cannot be transmitted.

Errno

EBADF

Socket associated with web-socket is no longer valid

See also

websockLib

Name

wsIsControlFrame( ) - check if a frame is a control frame

Layer

NET_BASE

Synopsis

```
_WRS_INLINE BOOL wsIsControlFrame
    (
    UINT8 op   /**< message opcode */
    )
```

Description

This routine check if the opcode in the frame state that this is a control frame.

Returns

TRUE if this is a control frame, or **FALSE** otherwise

Errno

N/A

See also

websockLib

Name

wsProcess( ) - process data received on the web-socket

Layer

NET_BASE

Synopsis

```
STATUS wsProcess
    (
```

```
    WS_ID ws   /**< websocket session */
    )
```

Description

This routine pull all data of the socket associated with the web-socket session. Each full message will trigger an **onMsg( )**, **onClose( )** or **onError( )** events.

Returns

OK if successful, or **ERROR** otherwise.

Errno

ENOMEM

Out of memory

ECONNRESET

Socket session is reset

EBADF

The socket associated with the web-socket session is no longer valid, closed?

See also

websockLib

Name

wsUsrCtxGet( ) - get user defined context

Layer

NET_BASE

Synopsis

```
void * wsUsrCtxGet
    (
    WS_ID ws   /**< websocket session */
    )
```

Description

This routine get the user defined context set in the **wsCreate( )** routine.

Returns

user defined context

Errno

N/A

See also

websockLib

Name

wsReadyStateGet( ) - get web-socket session state

Layer

NET_BASE

Synopsis

```
WS_READY_STATE wsReadyStateGet
    (
    const WS_ID ws   /**< websocket session */
    )
```

Description

This routine get the state of the web-socket session.

Returns

one of the constants defined by the **WS_READY_STATE** enumeration

Errno

N/A

See also

websockLib

Name

wsSockGet( ) - get socket descriptor

Layer

NET_BASE

Synopsis

```
int wsSockGet
    (
```

```
    const WS_ID ws   /**< websocket session */
    )
```

Description

This routine get the socket descriptor associated with the web-socket context.

Returns

socket descriptor

Errno

N/A

See also

websockLib